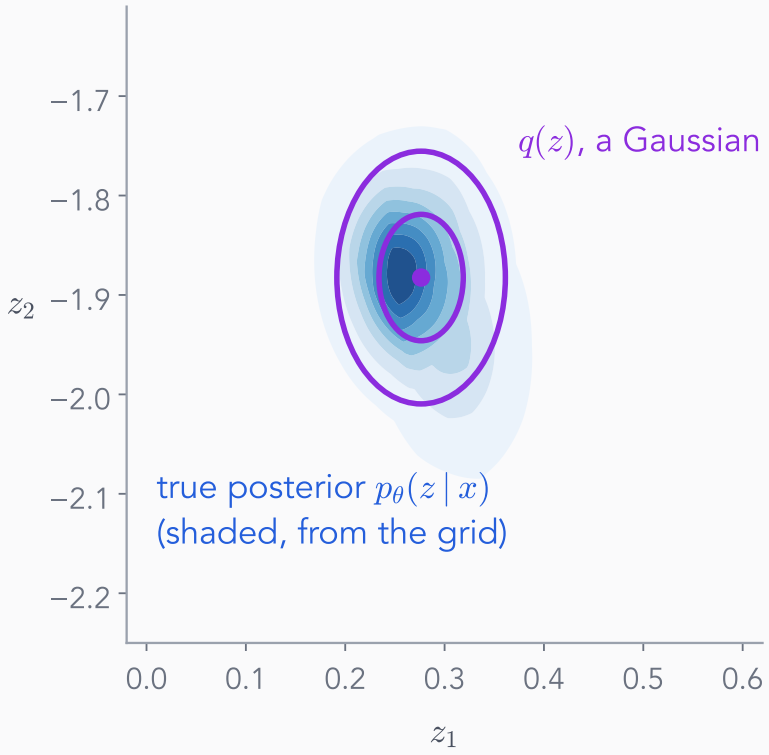
SESSION 2 · WHERE WE STOPPED

# Last time: we need $q$ close to a posterior we cannot compute




A decoder plus a prior is a probability model: $p(z)\,p_\theta(x \mid z)$.

Training it needs $p_\theta(x) = \int p(z)\,p_\theta(x \mid z)\,dz$: intractable.

So does the posterior $p_\theta(z \mid x)$, the shaded patch for our 3.

**Variational idea:** a simple Gaussian $q$ (purple), pulled close to it.

**Today: how to measure "close" (KL), the loss that does the pulling (the ELBO), and the VAE.**

PART 4 OF 7

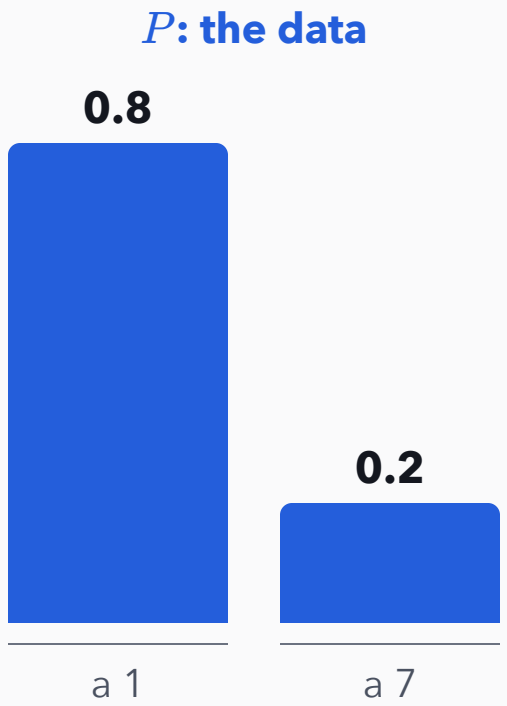
# KL divergence: how far apart are two distributions?

We want $q$ **close** to the posterior. Before two clouds in a plane, the simplest possible pair: two bar charts over the digits 1 and 7. Slowly, from a betting game.


✓ Distributions and fitting
✓ Autoencoders
✓ The probabilistic view
4 **KL divergence**
5 Variational inference and the ELBO
6 Reparameterisation
7 The VAE on MNIST

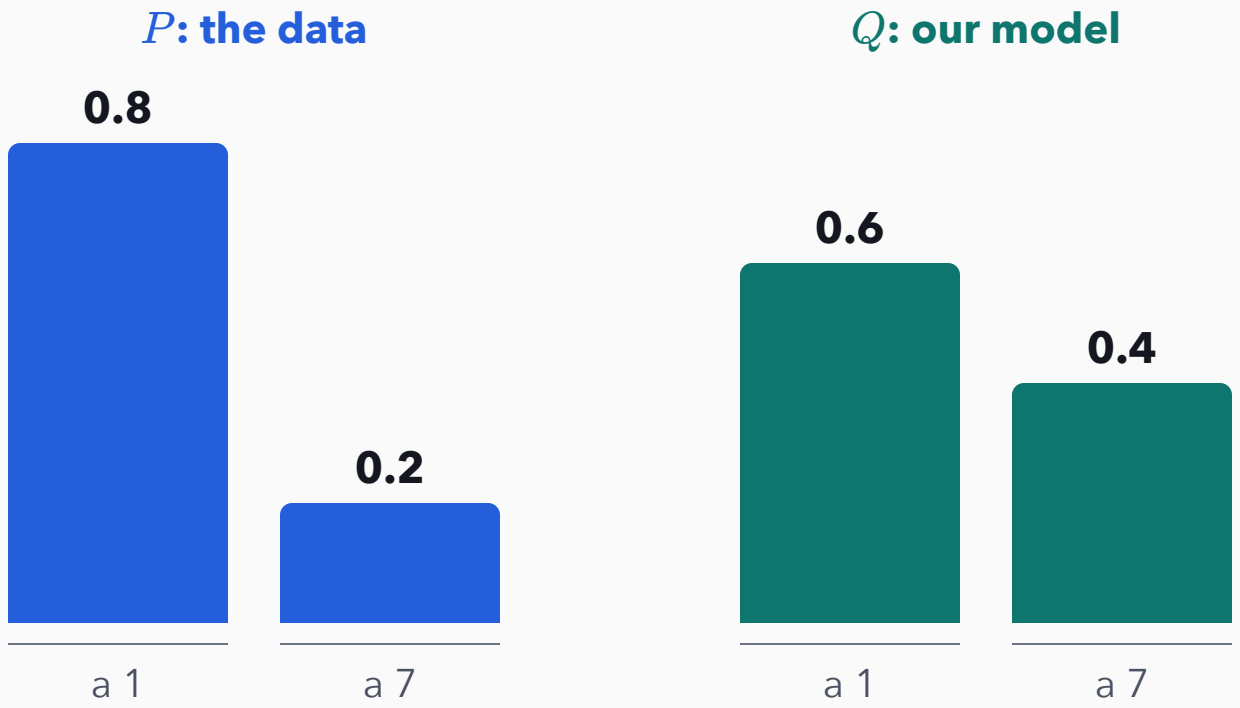
TWO DISTRIBUTIONS OVER THE SAME TWO OUTCOMES

# The data says 80/20. The model says 60/40.



TWO DISTRIBUTIONS OVER THE SAME TWO OUTCOMES

# The data says 80/20. The model says 60/40.




A dataset of 1s and 7s, 80% 1s. The model believes 60/40. How wrong is it, as one number?

A BETTING GAME

# Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| $P$ **(the truth)** | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 |

A BETTING GAME

# Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

| $P$ (the truth) | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | **1** | **1** | **1** | **7** | **1** | **1** | **1** | **1** | **7** | **1** |
| | 0.8 | 0.8 | 0.8 | 0.2 | 0.8 | 0.8 | 0.8 | 0.8 | 0.2 | 0.8 |

$$0.8^8 \times 0.2^2 = 0.0067$$

A BETTING GAME

# Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $P$ **(the truth)** | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 |
| $Q$ **(our model)** | **1** 0.6 | **1** 0.6 | **1** 0.6 | **7** 0.4 | **1** 0.6 | **1** 0.6 | **1** 0.6 | **1** 0.6 | **7** 0.4 | **1** 0.6 |

$$0.8^8 \times 0.2^2 = 0.0067$$

A BETTING GAME

# Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $P$ **(the truth)** | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 |
| $Q$ **(our model)** | **1** 0.6 | **1** 0.6 | **1** 0.6 | **7** 0.4 | **1** 0.6 | **1** 0.6 | **1** 0.6 | **1** 0.6 | **7** 0.4 | **1** 0.6 |

$$0.8^8 \times 0.2^2 = 0.0067$$

$$0.6^8 \times 0.4^2 = 0.0027$$

A BETTING GAME

# Which model makes the real data more likely?

Ten digits from the data: eight 1s, two 7s. Each model gives each digit a probability; multiply them.

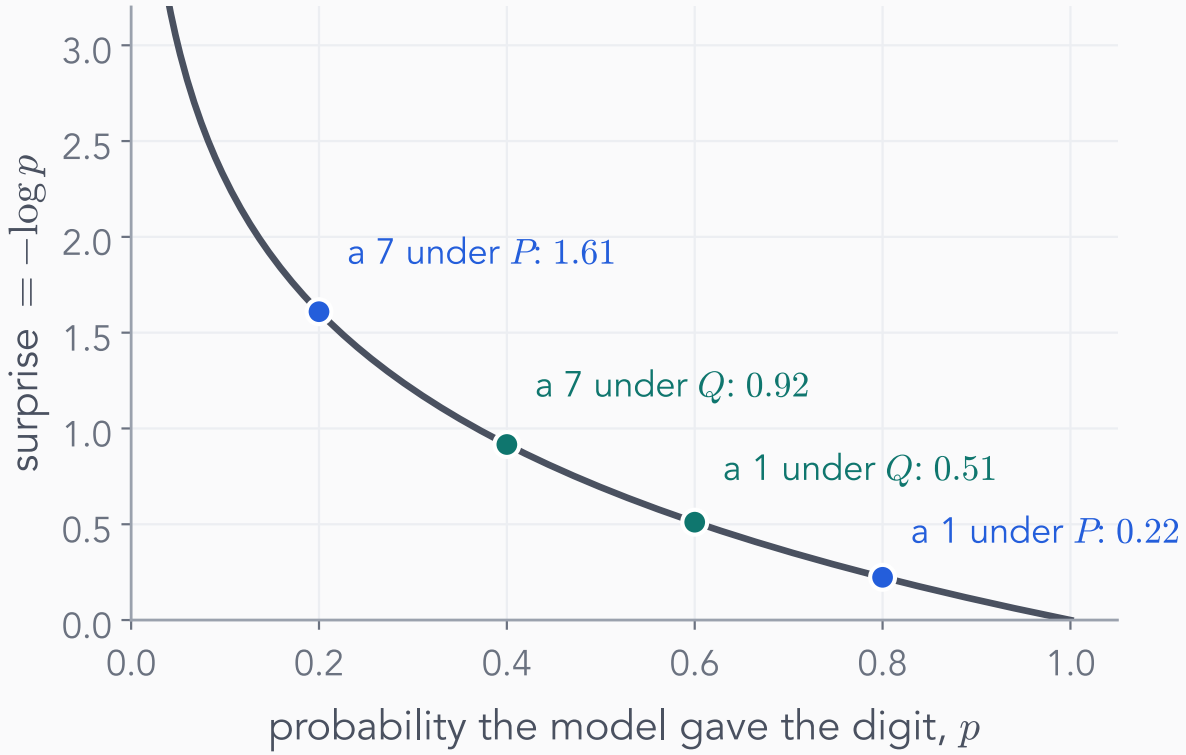
| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| $P$ **(the truth)** | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **1** 0.8 | **7** 0.2 | **1** 0.8 |
| $Q$ **(our model)** | **1** 0.6 | **1** 0.6 | **1** 0.6 | **7** 0.4 | **1** 0.6 | **1** 0.6 | **1** 0.6 | **1** 0.6 | **7** 0.4 | **1** 0.6 |

$$0.8^8 \times 0.2^2 = 0.0067$$

$$0.6^8 \times 0.4^2 = 0.0027$$

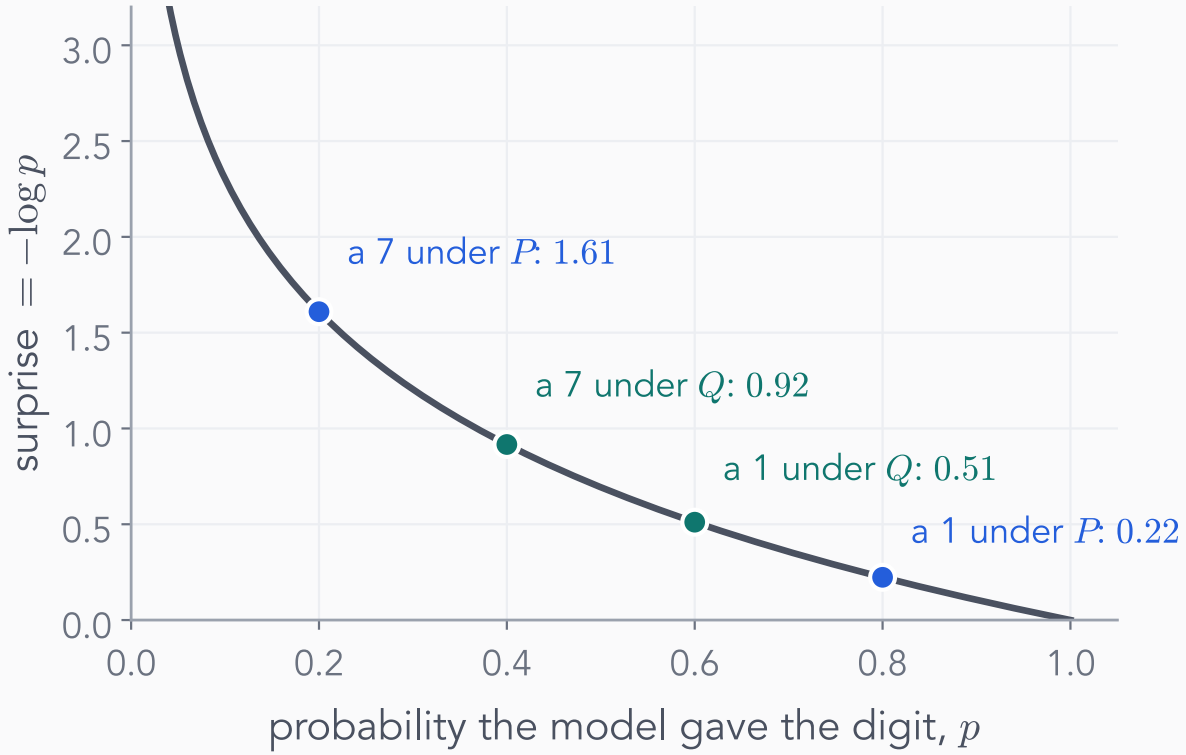
**The truth wins, by a factor of $0.0067/0.0027 = 2.5$. KL will measure this gap, per digit.**

# Surprise $= -\log p$, and why the log




What should "surprise" do?

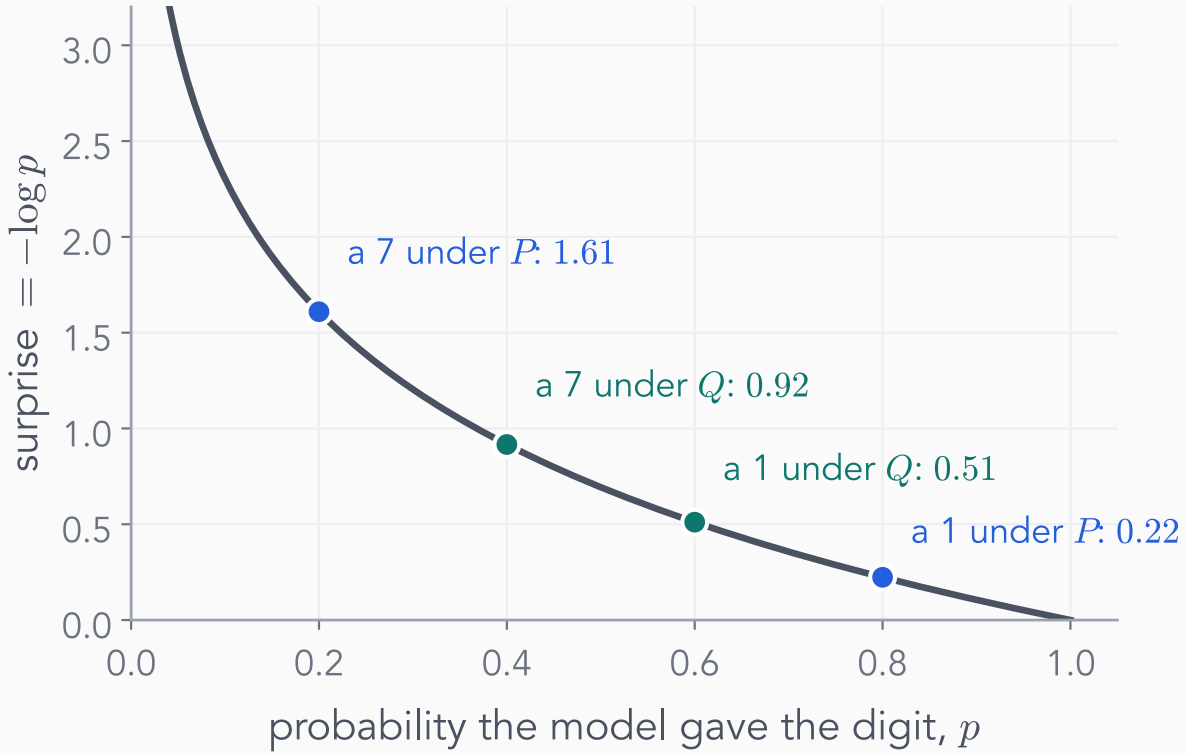
# Surprise $= -\log p$, and why the log




What should "surprise" do?

1 · a certain event: no surprise. $-\log 1 = 0$.

# Surprise $= -\log p$, and why the log

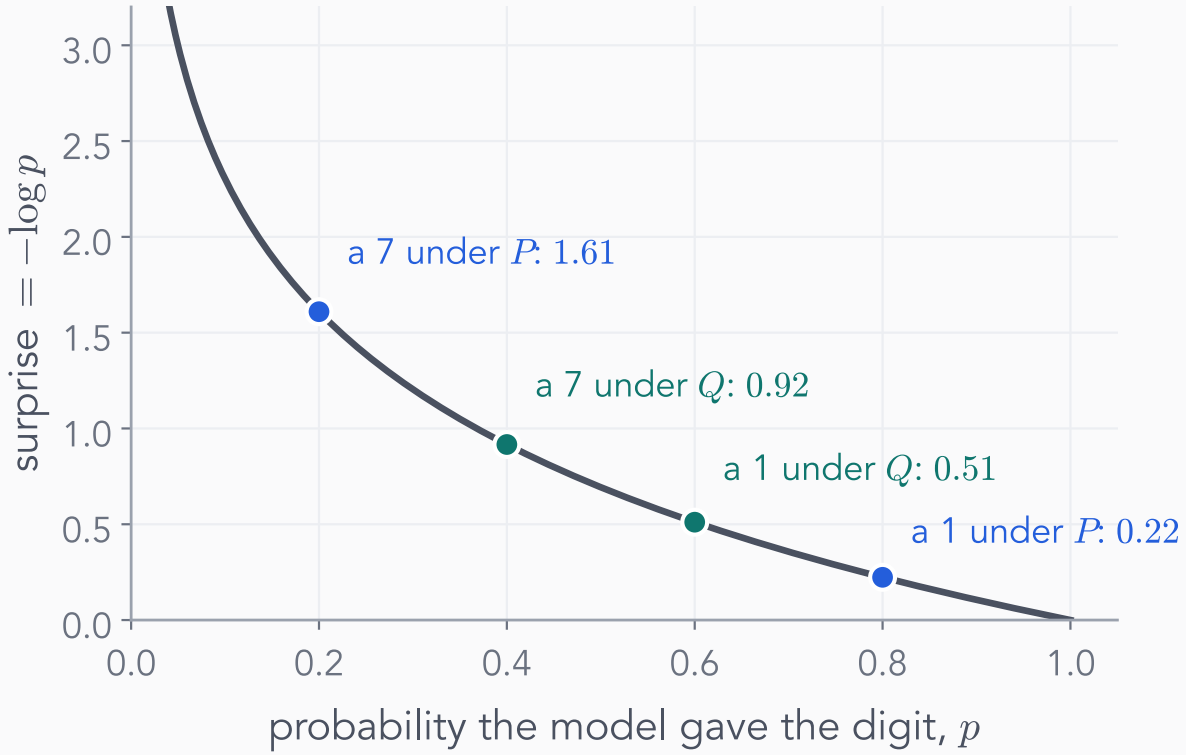



What should "surprise" do?

1 · a certain event: no surprise. $-\log 1 = 0$.

2 · rarer means more surprising. $-\log p$ grows as $p$ shrinks.

# Surprise $= -\log p$, and why the log




What should "surprise" do?

1 · a certain event: no surprise. $-\log 1 = 0$.

2 · rarer means more surprising. $-\log p$ grows as $p$ shrinks.

3 · two independent digits: probabilities **multiply**, surprises should **add**. Only the log turns products into sums.

> **So the betting-game product becomes a sum of surprises, one per digit. Natural log, so the unit is the nat.**

OUR TWO DIGITS, SCORED BOTH WAYS

# The surprise table

| outcome | $P$ | $Q$ | surprise if you believe $P$ | surprise if you believe $Q$ |
|---|---|---|---|---|
| a 1 | 0.8 | 0.6 | $-\log 0.8 = 0.223$ | |
| a 7 | 0.2 | 0.4 | $-\log 0.2 = 1.609$ | |

OUR TWO DIGITS, SCORED BOTH WAYS

# The surprise table

| outcome | $P$ | $Q$ | surprise if you believe $P$ | surprise if you believe $Q$ |
|---|---|---|---|---|
| a 1 | 0.8 | 0.6 | $-\log 0.8 = 0.223$ | $-\log 0.6 = 0.511$ |
| a 7 | 0.2 | 0.4 | $-\log 0.2 = 1.609$ | $-\log 0.4 = 0.916$ |

OUR TWO DIGITS, SCORED BOTH WAYS

# The surprise table

| outcome | $P$ | $Q$ | surprise if you believe $P$ | surprise if you believe $Q$ |
|---|---|---|---|---|
| a 1 | 0.8 | 0.6 | $-\log 0.8 = 0.223$ | $-\log 0.6 = 0.511$ |
| a 7 | 0.2 | 0.4 | $-\log 0.2 = 1.609$ | $-\log 0.4 = 0.916$ |

Each row is one kind of digit. The next two slides average each column, weighted by how often the digit really appears.

Believing $Q$, a 1 surprises us more than it should ($0.511$ vs $0.223$), a 7 less ($0.916$ vs $1.609$).

AVERAGE SURPRISE, WHEN YOU BELIEVE THE TRUTH

# Entropy: $H(P) = 0.500$ nats per digit

$$H(P) = \sum_i P_i \left(-\log P_i\right) = \underbrace{0.8}_{\text{how often a 1}} \times 0.223 + \underbrace{0.2}_{\text{how often a 7}} \times 1.609 = 0.500$$

AVERAGE SURPRISE, WHEN YOU BELIEVE THE TRUTH

# Entropy: $H(P) = 0.500$ nats per digit

$$H(P) = \sum_i P_i \left(-\log P_i\right) = \underbrace{0.8}_{\text{how often a 1}} \times 0.223 + \underbrace{0.2}_{\text{how often a 7}} \times 1.609 = 0.500$$

The weights are how often each digit really appears: eight in ten, two in ten. The batch of ten digits from the game, averaged.

**Even the perfect model is surprised by 0.500 nats per digit on average: the data are random.**

SAME DATA, A DIFFERENT SCORER

## Cross-entropy: $H(P, Q) = 0.592$ nats per digit

$$H(P, Q) = \sum_i P_i \left(-\log Q_i\right) = 0.8 \times 0.511 + 0.2 \times 0.916 = 0.592$$

SAME DATA, A DIFFERENT SCORER

# Cross-entropy: $H(P, Q) = 0.592$ nats per digit

$$H(P, Q) = \sum_i P_i \left(-\log Q_i\right) = 0.8 \times 0.511 + 0.2 \times 0.916 = 0.592$$

**Still weighted by $P$**

the digits still come from the real data: eight 1s, two 7s.

**Scored by $Q$**

each digit's surprise now uses the model's probability.

**Believing the wrong model costs more surprise on the same data: 0.592 instead of 0.500.**

THE EXTRA SURPRISE

# KL divergence = cross-entropy − entropy

$$H(P) = 0.500$$

unavoidable: the data are random

together: $H(P, Q) = 0.592$ (more precisely $0.5004 + 0.0915 = 0.5919$)

THE EXTRA SURPRISE

# KL divergence = cross-entropy − entropy

$H(P) = 0.500$
unavoidable: the data are random

$0.092$
KL

together: $H(P, Q) = 0.592$ (more precisely $0.5004 + 0.0915 = 0.5919$)

THE EXTRA SURPRISE

# KL divergence = cross-entropy − entropy

$H(P) = 0.500$
unavoidable: the data are random

$0.092$
KL

together: $H(P, Q) = 0.592$ (more precisely $0.5004 + 0.0915 = 0.5919$)

$$\mathrm{KL}(P \,\|\, Q) = H(P, Q) - H(P) = \sum_i P_i \log \frac{P_i}{Q_i} = 0.092 \text{ nats}$$

**KL is the part of the surprise that comes only from using the wrong model.**

# Where the 0.092 comes from

| digit | how often $P_i$ | extra surprise $\log(P_i/Q_i)$ | contribution |
|---|---|---|---|
| a 1 | 0.8 | $\log(0.8/0.6) = 0.2877$ | $0.2301$ |

DIGIT BY DIGIT, AND BACK TO THE GAME

# Where the 0.092 comes from

| digit | how often $P_i$ | extra surprise $\log(P_i/Q_i)$ | contribution |
|---|---|---|---|
| a 1 | 0.8 | $\log(0.8/0.6) = 0.2877$ | $0.2301$ |
| a 7 | 0.2 | $\log(0.2/0.4) = -0.6931$ | $-0.1386$ |

DIGIT BY DIGIT, AND BACK TO THE GAME

# Where the 0.092 comes from

| digit | how often $P_i$ | extra surprise $\log(P_i/Q_i)$ | contribution |
|---|---|---|---|
| a 1 | 0.8 | $\log(0.8/0.6) = 0.2877$ | $0.2301$ |
| a 7 | 0.2 | $\log(0.2/0.4) = -0.6931$ | $-0.1386$ |
| **KL** | | | $0.0915$ |

# Where the 0.092 comes from

| digit | how often $P_i$ | extra surprise $\log(P_i/Q_i)$ | contribution |
|---|---|---|---|
| a 1 | 0.8 | $\log(0.8/0.6) = 0.2877$ | $0.2301$ |
| a 7 | 0.2 | $\log(0.2/0.4) = -0.6931$ | $-0.1386$ |
| **KL** | | | $0.0915$ |

$$\text{the game:} \quad \tfrac{1}{10} \log \frac{0.0067}{0.0027} = \tfrac{1}{10} \log 2.497 = 0.0915$$

> One row is negative: $Q$ over-predicts 7s, so a 7 surprises it less. The total is still positive, and it is exactly the betting-game gap, per digit.

NOTATION WE WILL USE ALL LECTURE

# $\mathbb{E}$ means "average over"

$$\mathbb{E}_{x \sim P}\big[f(x)\big] = \sum_x P(x)\, f(x)$$

exact, for a sum over outcomes

## NOTATION WE WILL USE ALL LECTURE

# $\mathbb{E}$ means "average over"

$$\mathbb{E}_{x \sim P}\big[f(x)\big] = \sum_{x} P(x)\, f(x)$$

exact, for a sum over outcomes

$$\mathbb{E}_{z \sim q}\big[f(z)\big] = \int q(z)\, f(z)\, dz$$

exact, for a continuous variable

NOTATION WE WILL USE ALL LECTURE

# $\mathbb{E}$ means "average over"

$$\mathbb{E}_{x \sim P}\big[f(x)\big] = \sum_{x} P(x)\, f(x)$$

exact, for a sum over outcomes

$$\mathbb{E}_{z \sim q}\big[f(z)\big] = \int q(z)\, f(z)\, dz$$

exact, for a continuous variable

$$\mathbb{E}_{x \sim P}\big[f(x)\big] \approx \frac{1}{S} \sum_{s=1}^{S} f(x_s), \quad x_s \sim P$$

estimated from samples

NOTATION WE WILL USE ALL LECTURE

# $\mathbb{E}$ means "average over"

$$\mathbb{E}_{x\sim P}\big[f(x)\big] = \sum_{x} P(x)\, f(x)$$

exact, for a sum over outcomes

$$\mathbb{E}_{z\sim q}\big[f(z)\big] = \int q(z)\, f(z)\, dz$$

exact, for a continuous variable

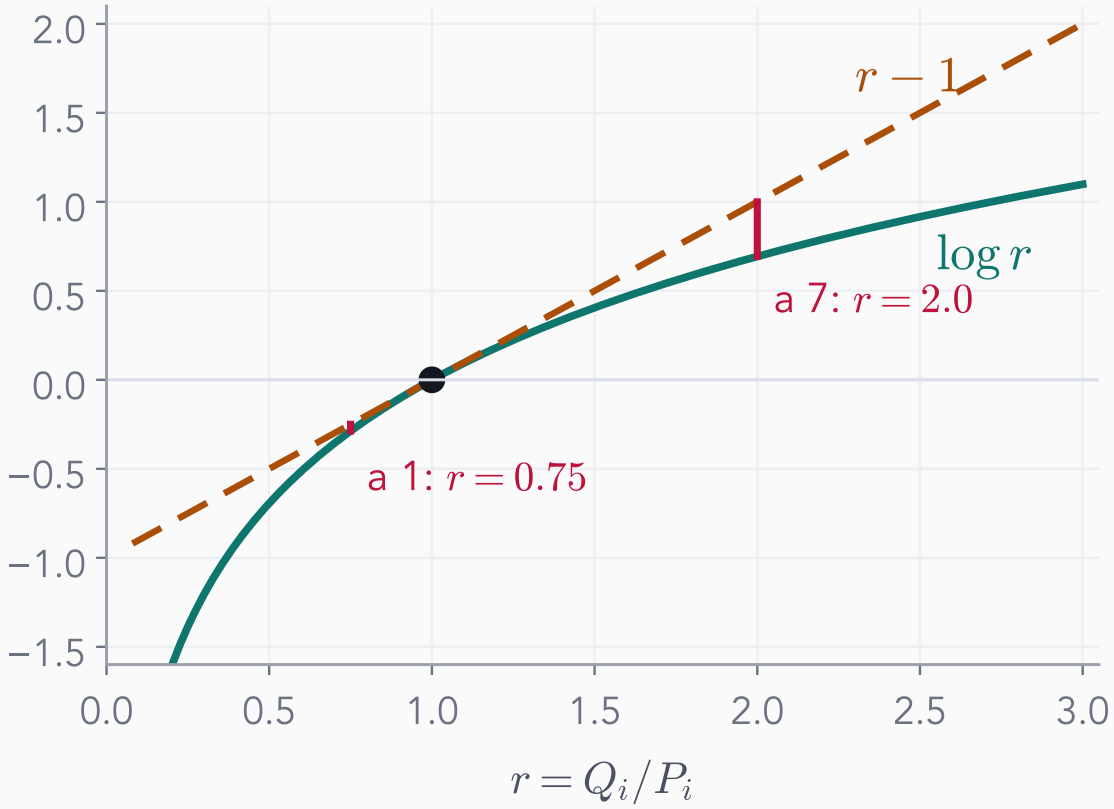
$$\mathbb{E}_{x\sim P}\big[f(x)\big] \approx \frac{1}{S}\sum_{s=1}^{S} f(x_s), \quad x_s \sim P$$

estimated from samples

KL is an average: $\mathbf{KL}(P\|Q) = \mathbb{E}_{x\sim P}\big[\log \frac{P(x)}{Q(x)}\big]$. Five draws 1, 1, 7, 1, 1 give $\frac{1}{5}(4 \times 0.288 - 0.693) = 0.092$.

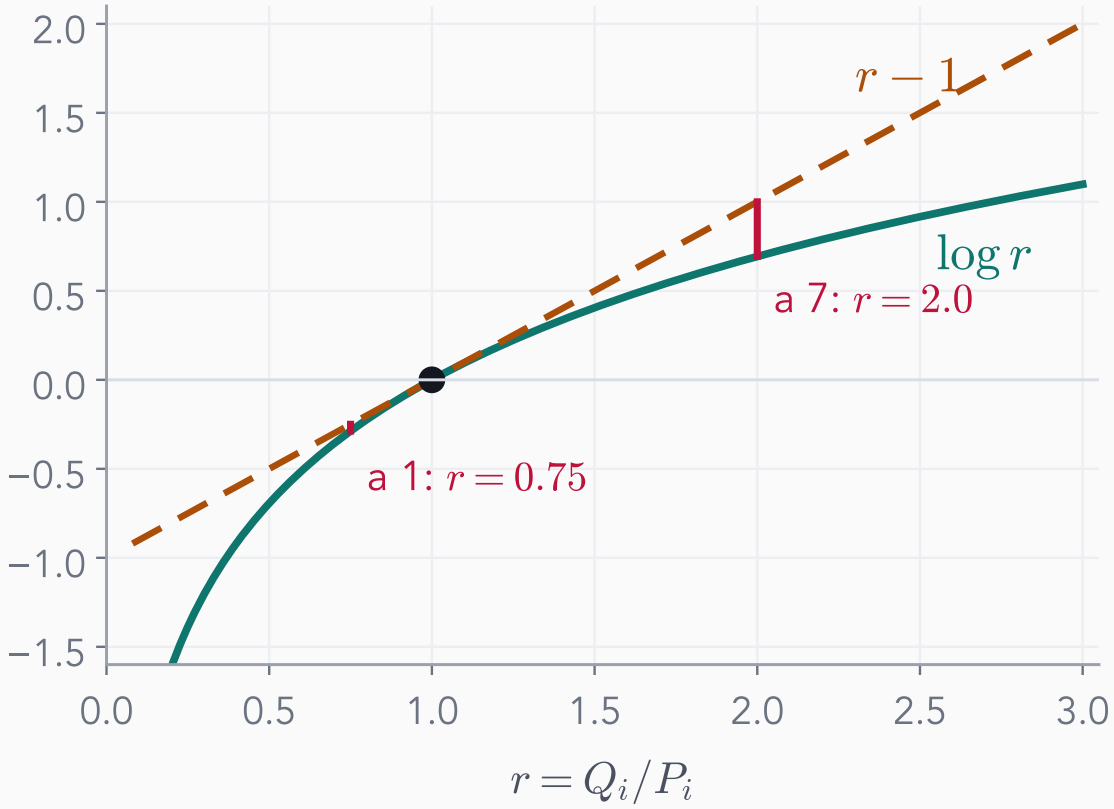
WHY KL CAN NEVER BE NEGATIVE

# One picture: $\log r$ lies under its tangent




Let $r = Q_i/P_i$. The curve $\log r$ never rises above the line $r - 1$.

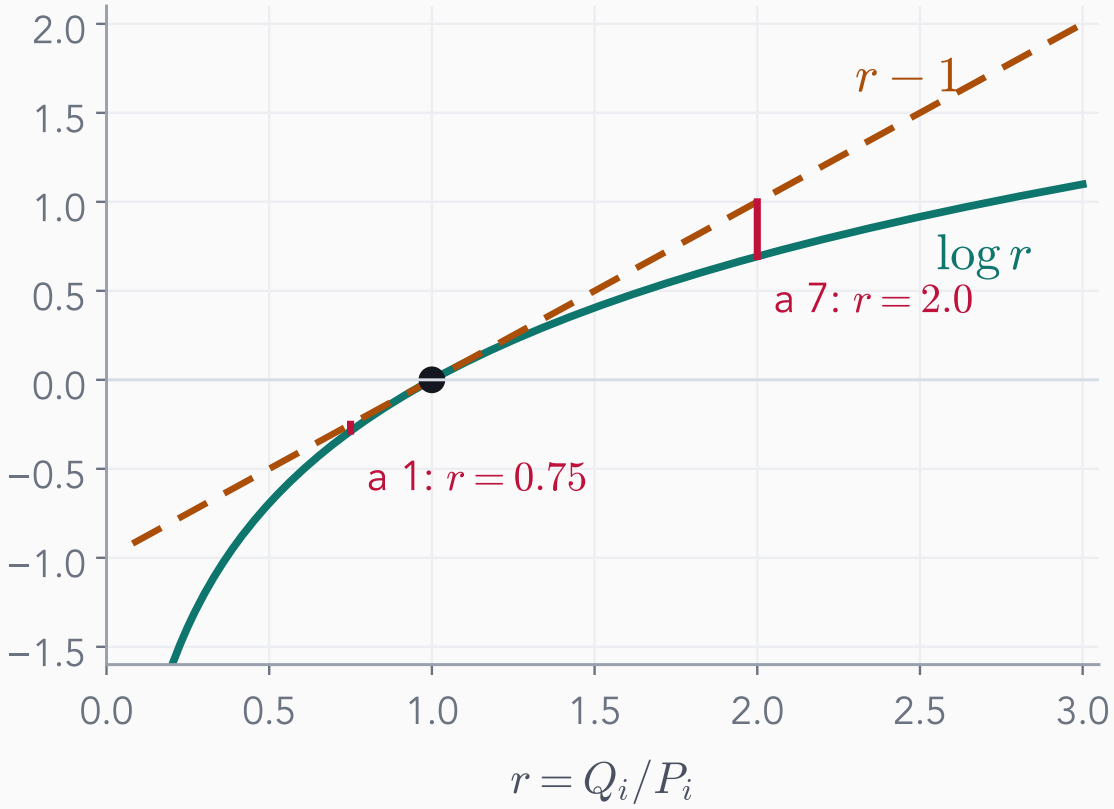
# One picture: $\log r$ lies under its tangent




Let $r = Q_i/P_i$. The curve $\log r$ never rises above the line $r - 1$.

$$\mathrm{KL} = \sum_i P_i \left(-\log r_i\right) \;\geq\; \sum_i P_i \left(1 - r_i\right)$$

# One picture: $\log r$ lies under its tangent




Let $r = Q_i/P_i$. The curve $\log r$ never rises above the line $r - 1$.

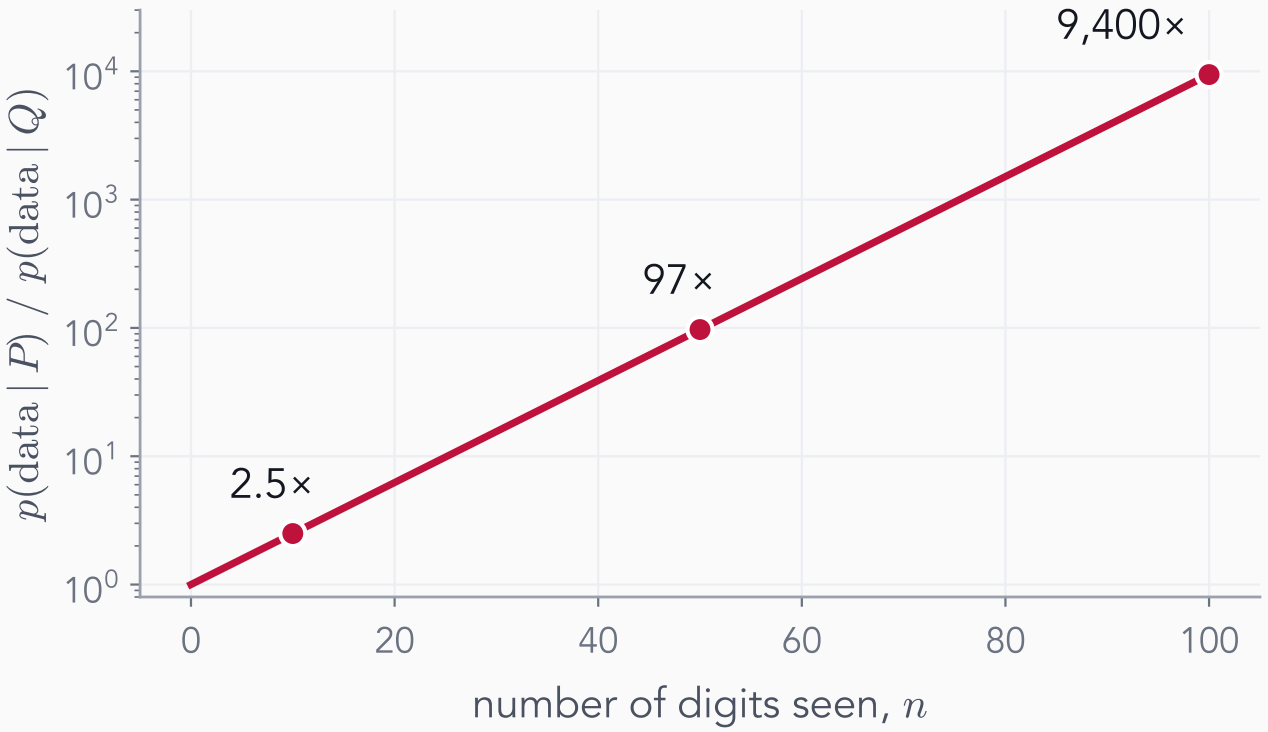
$$\mathrm{KL} = \sum_i P_i \left(-\log r_i\right) \;\geq\; \sum_i P_i \left(1 - r_i\right)$$

$$= \sum_i P_i - \sum_i Q_i = 1 - 1 = 0$$

**Equality only when every $r_i = 1$: $Q = P$. No model beats the truth on the truth's own data.**

WHAT THE NUMBER MEANS

# KL is how fast evidence piles up




$$\frac{p(\text{data} \mid P)}{p(\text{data} \mid Q)} \approx e^{n\,\mathrm{KL}(P\|Q)}$$

After 10 digits the truth is 2.5× more likely; after 50, 97×; after 100, 9,400×.

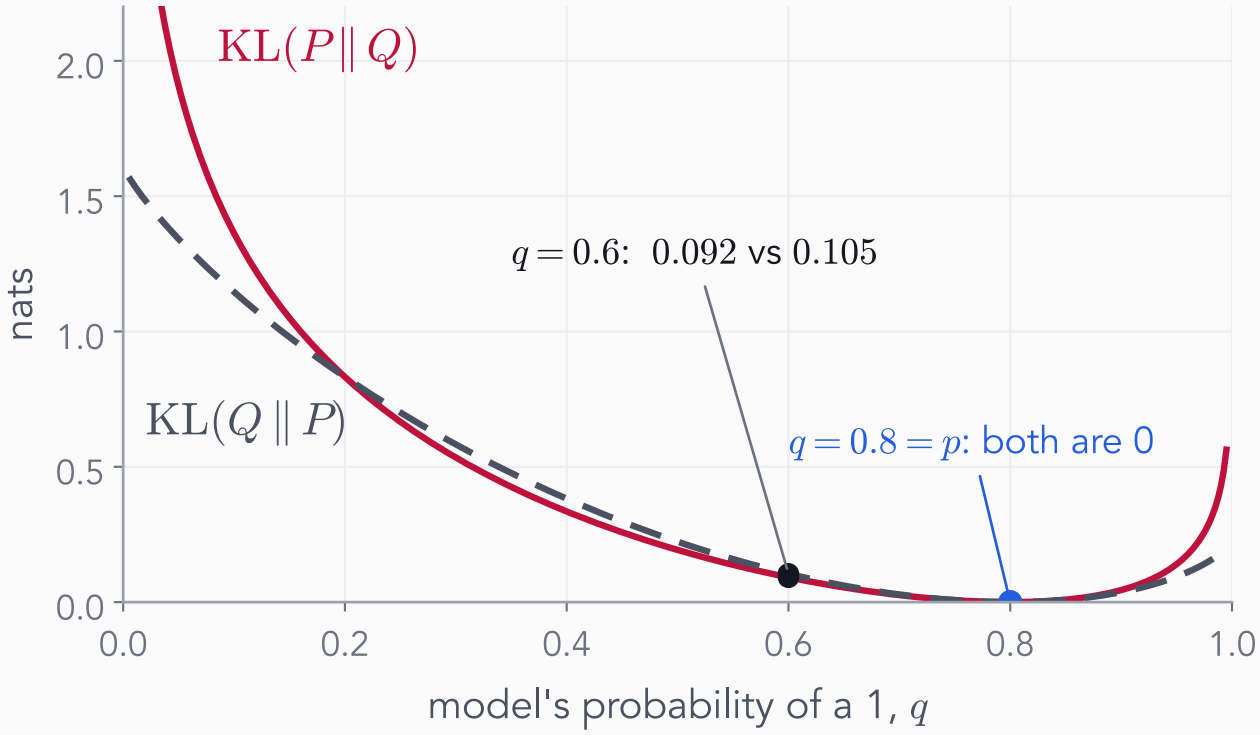
**A small KL means the two models are hard to tell apart from data; a large one means a few samples settle it.**

NOT A DISTANCE

# Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.1726 + 0.2773 = 0.1046$$



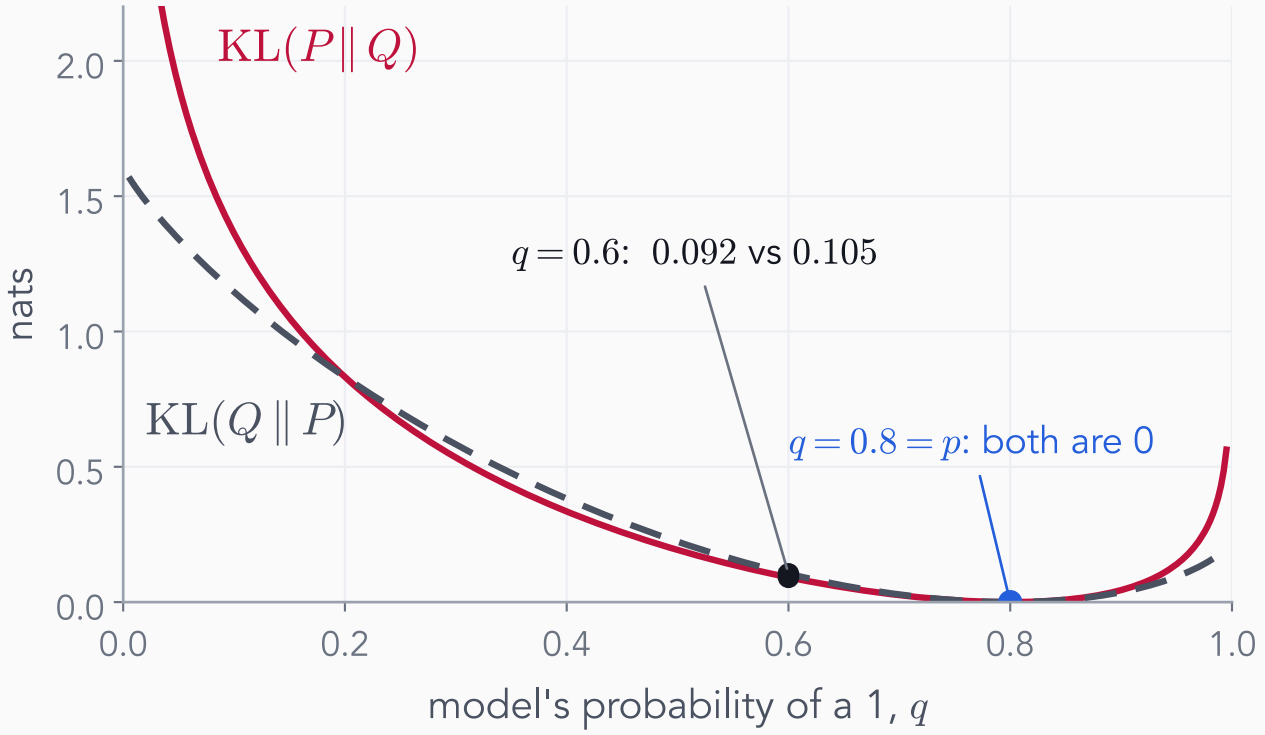
NOT A DISTANCE

# Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.1726 + 0.2773 = 0.1046$$

$0.105 \neq 0.092$: KL is not symmetric. "How wrong is $Q$ about $P$" differs from "how wrong is $P$ about $Q$".



NOT A DISTANCE

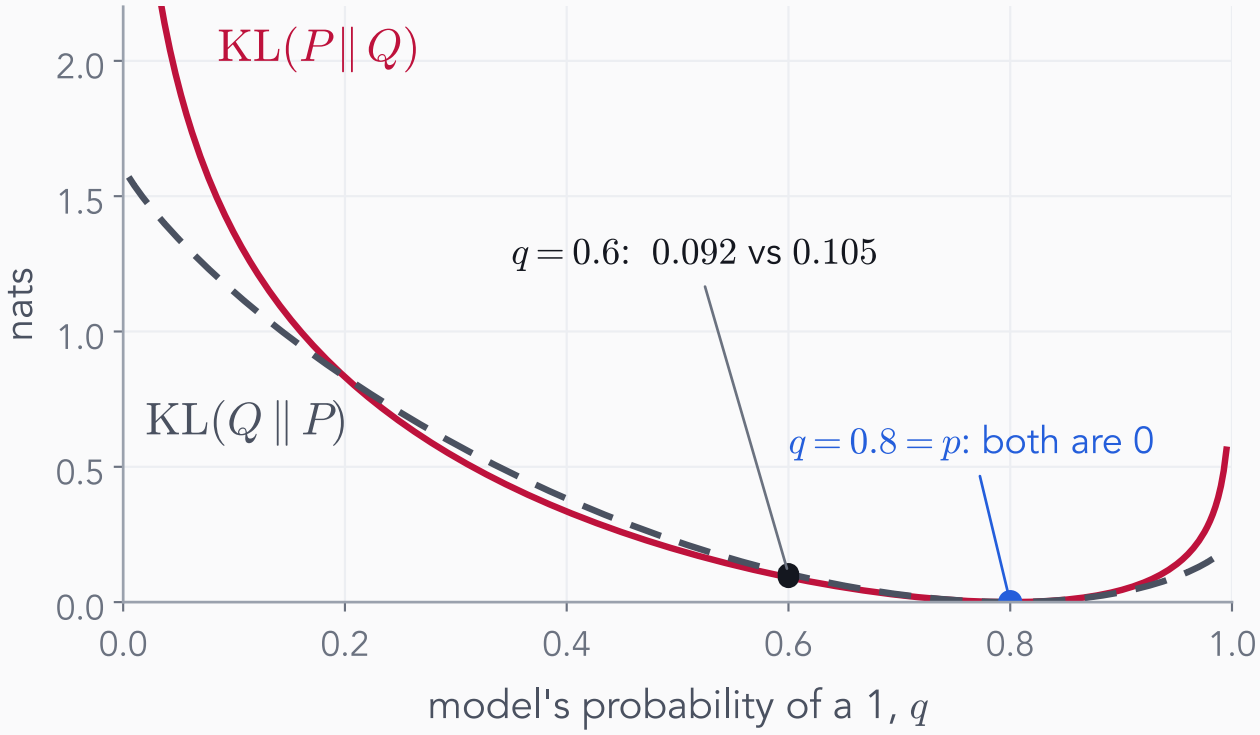
# Swap the arguments, get a different number

$$\mathrm{KL}(Q\|P) = 0.6\log\frac{0.6}{0.8} + 0.4\log\frac{0.4}{0.2}$$

$$= -0.1726 + 0.2773 = 0.1046$$

$0.105 \neq 0.092$: KL is not symmetric. "How wrong is $Q$ about $P$" differs from "how wrong is $P$ about $Q$".

Push $q \to 0$ and $\mathrm{KL}(P\|Q) \to \infty$. Never give zero probability to something that happens.



THE ORDER MATTERS, SHARPLY

# A model that "forgets" 7s

Truth $P = (0.8, 0.2)$. A model $Q' = (0.99, 0.01)$ that almost never expects a 7.

THE ORDER MATTERS, SHARPLY

# A model that "forgets" 7s

Truth $P = (0.8, 0.2)$. A model $Q' = (0.99, 0.01)$ that almost never expects a 7.

$$\mathrm{KL}(P \| Q') = 0.43$$

real 7s keep showing up, and each costs $\log(0.2/0.01) = 3.0$ nats

THE ORDER MATTERS, SHARPLY

# A model that "forgets" 7s

Truth $P = (0.8, 0.2)$. A model $Q' = (0.99, 0.01)$ that almost never expects a 7.

$$\mathrm{KL}(P\|Q') = 0.43$$

real 7s keep showing up, and each costs $\log(0.2/0.01) = 3.0$ nats

$$\mathrm{KL}(Q'\|P) = 0.18$$

averaged under $Q'$, which almost never draws a 7, so the costly digit rarely counts

**$\mathrm{KL}(P\|Q)$ punishes a model for giving tiny probability to things that really happen.**

# Cross-entropy = entropy + KL

$$H(P, Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

# Cross-entropy = entropy + KL

$$H(P, Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

$$= -\sum_i P_i \log P_i + \sum_i P_i \big( \log P_i - \log Q_i \big)$$

add and subtract $\sum_i P_i \log P_i$

# Cross-entropy = entropy + KL

$$H(P, Q) = -\sum_i P_i \log Q_i$$

cross-entropy: the loss you train classifiers with

$$= -\sum_i P_i \log P_i + \sum_i P_i \big( \log P_i - \log Q_i \big)$$

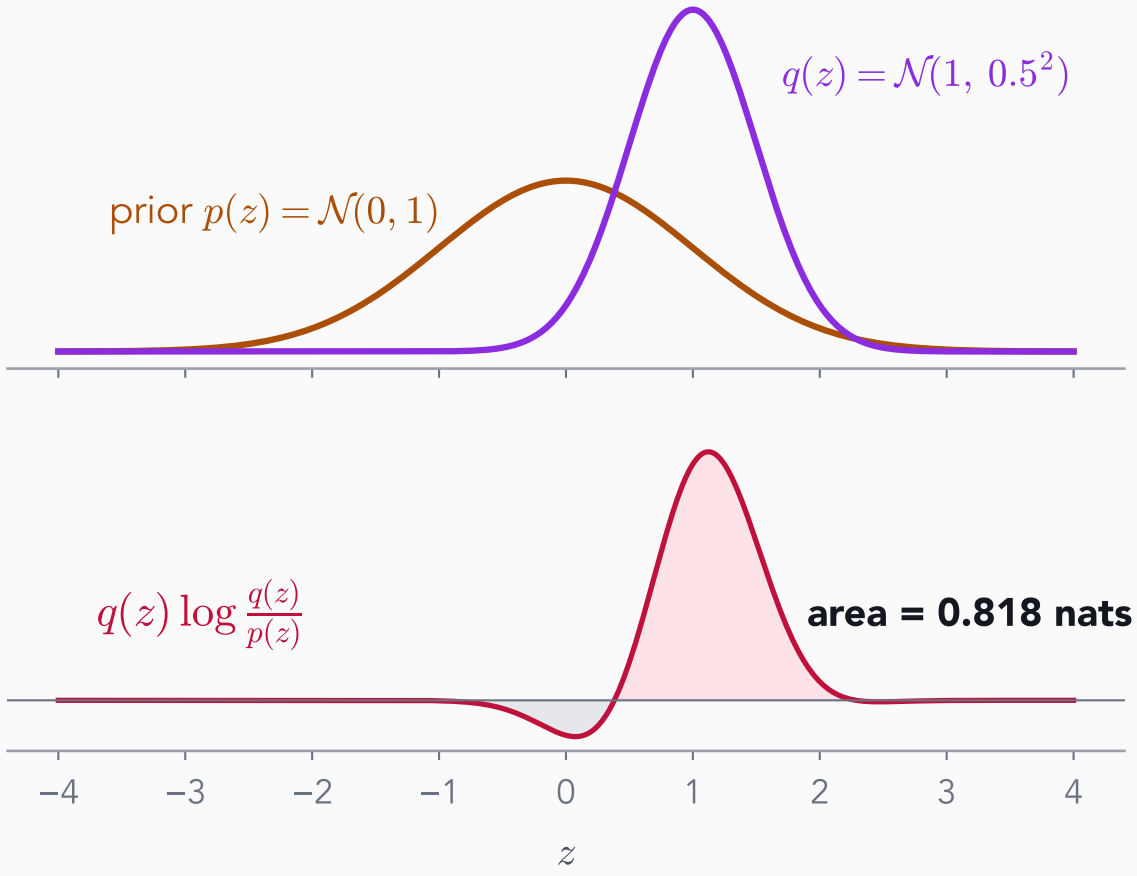
add and subtract $\sum_i P_i \log P_i$

$$= \underbrace{H(P)}_{\text{entropy: fixed by the data}} + \mathrm{KL}(P \,\|\, Q)$$

**Lowering cross-entropy can only lower the KL part. So does maximum likelihood: it minimises $\mathrm{KL}(\text{data} \,\|\, \text{model})$.**

THE VERSION THE VAE NEEDS

# KL between two bumps: an area

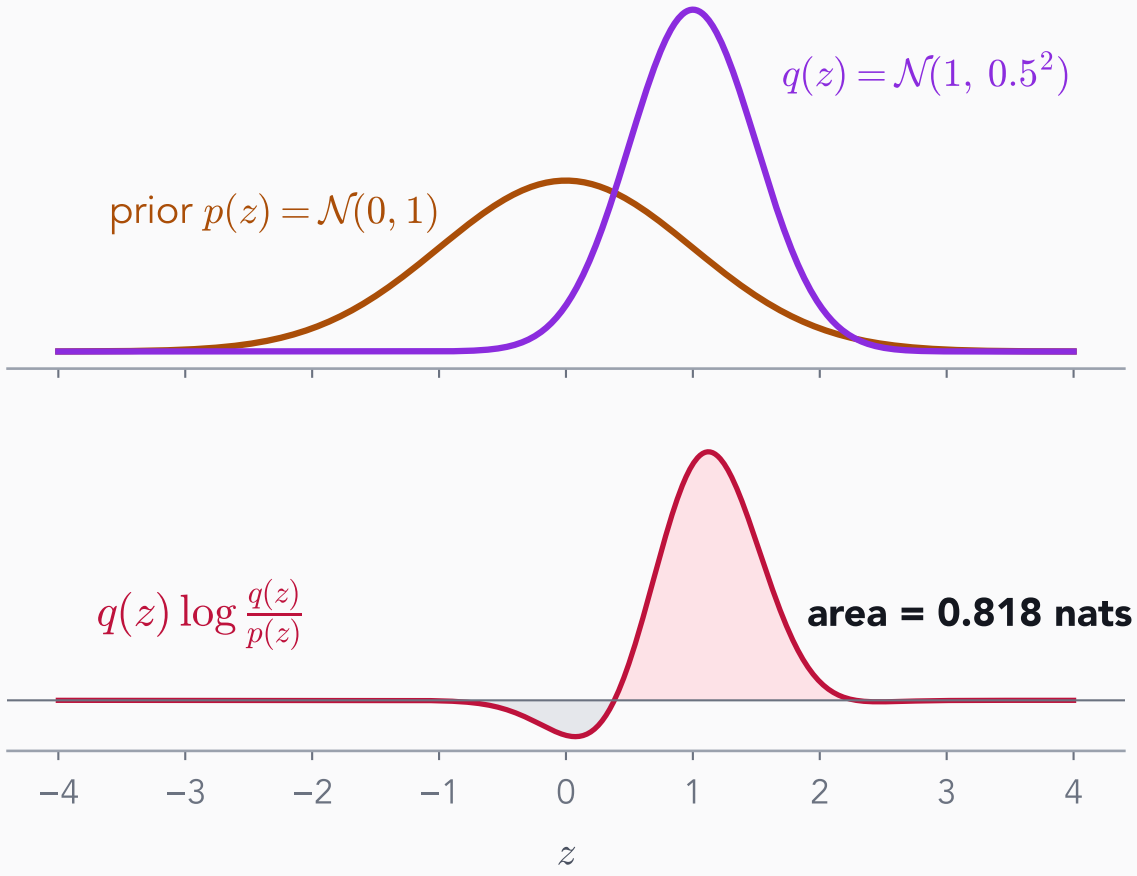



For continuous variables the sum becomes an integral:

$$\mathrm{KL}(q\|p) = \int q(z)\ \log \frac{q(z)}{p(z)}\ dz$$

THE VERSION THE VAE NEEDS

# KL between two bumps: an area



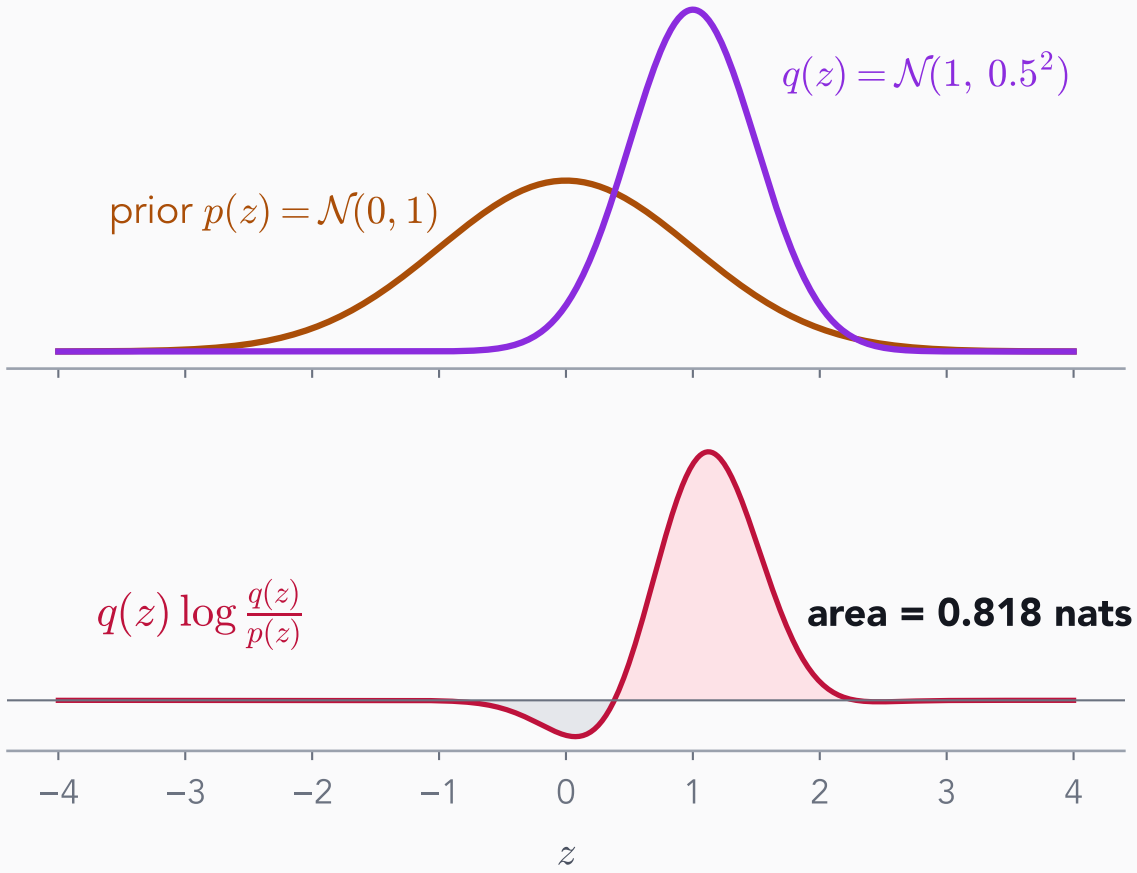

For continuous variables the sum becomes an integral:

$$\mathrm{KL}(q\|p) = \int q(z)\ \log \frac{q(z)}{p(z)}\, dz$$

Take a toy cloud $q = \mathcal{N}(1, 0.5^2)$ and $p = \mathcal{N}(0, 1)$, the prior. (Our 3's real $q$ is far narrower: Part 7.)

THE VERSION THE VAE NEEDS

# KL between two bumps: an area




For continuous variables the sum becomes an integral:

$$\mathrm{KL}(q\|p) = \int q(z)\ \log \frac{q(z)}{p(z)}\, dz$$

Take a toy cloud $q = \mathcal{N}(1, 0.5^2)$ and $p = \mathcal{N}(0, 1)$, the prior. (Our 3's real $q$ is far narrower: Part 7.)

The shaded area, computed numerically, is **0.818** nats.

**Can we get 0.818 without a computer? Yes: for two Gaussians the integral has a formula. Two steps.**

GAUSSIAN KL, STEP 1 OF 2

# Write out the log of the ratio

$$\log q(z) = -\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}$$

log of $\mathcal{N}(z; \mu, \sigma^2)$

GAUSSIAN KL, STEP 1 OF 2

# Write out the log of the ratio

$$\log q(z) = \underbrace{-\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}}_{\text{log of } \mathcal{N}(z;\mu,\sigma^2)}$$

$$\log p(z) = \underbrace{-\tfrac{1}{2}\log 2\pi - \frac{z^2}{2}}_{\text{log of } \mathcal{N}(z;0,1)}$$

GAUSSIAN KL, STEP 1 OF 2

# Write out the log of the ratio

$$\log q(z) = -\log \sigma - \tfrac{1}{2}\log 2\pi - \frac{(z-\mu)^2}{2\sigma^2}$$

log of $\mathcal{N}(z; \mu, \sigma^2)$

$$\log p(z) = -\tfrac{1}{2}\log 2\pi - \frac{z^2}{2}$$

log of $\mathcal{N}(z; 0, 1)$

$$\log \frac{q(z)}{p(z)} = -\log \sigma - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}$$

subtract: the $\frac{1}{2}\log 2\pi$ terms cancel

**Three terms. Next: average each one over $z \sim q$.**

GAUSSIAN KL, STEP 2 OF 2

# Average each term over the cloud $q$

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (mean squared + variance)

GAUSSIAN KL, STEP 2 OF 2

# Average each term over the cloud $q$

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (mean squared + variance)

$$\mathrm{KL} = -\log\sigma - \frac{\sigma^2}{2\sigma^2} + \frac{\mu^2+\sigma^2}{2} = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log\sigma^2\right)$$

GAUSSIAN KL, STEP 2 OF 2

# Average each term over the cloud $q$

$\mathbb{E}_q\left[(z-\mu)^2\right] = \sigma^2$ (that is the variance)

$\mathbb{E}_q\left[z^2\right] = \mu^2 + \sigma^2$ (mean squared + variance)

$$\mathrm{KL} = -\log\sigma - \frac{\sigma^2}{2\sigma^2} + \frac{\mu^2+\sigma^2}{2} = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log\sigma^2\right)$$

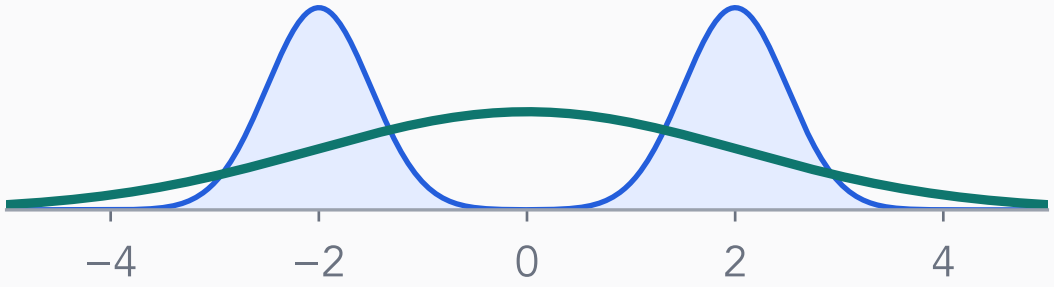
$$\mu = 1,\ \sigma = 0.5: \quad \tfrac{1}{2}\left(1 + 0.25 - 1 + 1.386\right) = 0.818\ \checkmark$$

**Per coordinate: moving the mean costs $\frac{1}{2}\mu^2$; a width other than 1 costs $\frac{1}{2}(\sigma^2 - 1 - \log\sigma^2)$, zero only at $\sigma = 1$. Several coordinates: add one term each.**

BACK TO OUR TWO BUMPS

# Fit one Gaussian: the KL direction decides what you get

best $q = \mathcal{N}(0.0,\ 2.06^2)$

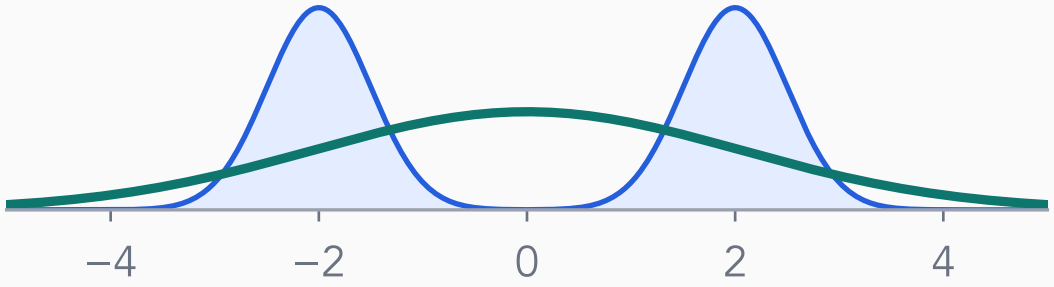



minimise $\mathrm{KL}(\mathrm{data} \,\|\, q)$: cover everything.
Part 1's maximum-likelihood fit, with infinite data.

BACK TO OUR TWO BUMPS

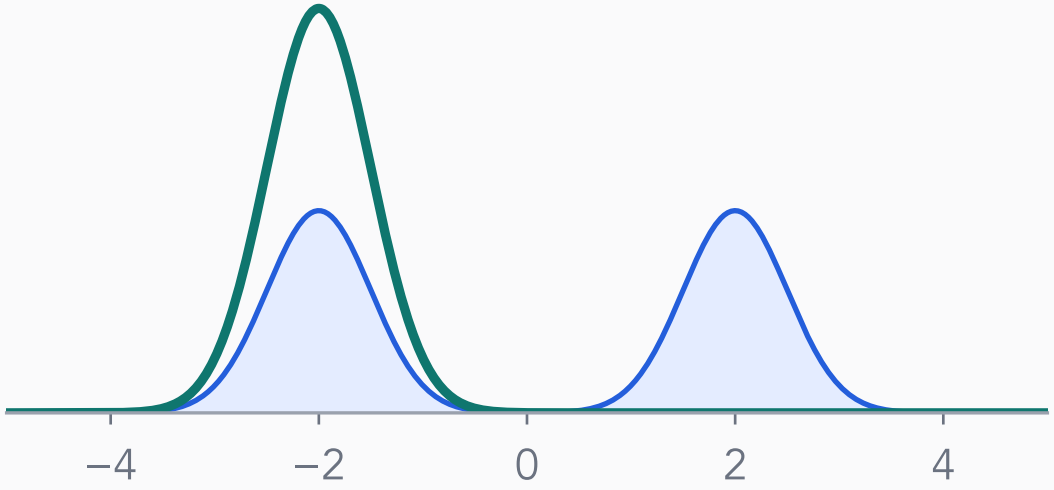
# Fit one Gaussian: the KL direction decides what you get

best $q = \mathcal{N}(0.0,\ 2.06^2)$




minimise $\mathrm{KL}(\mathrm{data} \,\|\, q)$: cover everything.
Part 1's maximum-likelihood fit, with infinite data.

best $q = \mathcal{N}(-2.0,\ 0.50^2)$




minimise $\mathrm{KL}(q \,\|\, \mathrm{data})$: never put mass where data is absent, so pick one bump.

**Mode-covering vs mode-seeking. Variational inference will minimise $\mathrm{KL}(q \,\|\, \mathrm{posterior})$: the second kind.**

END OF PART 4

# What we did, and what comes next

### WHAT WE DID

1. Surprise $= -\log p$; entropy = average surprise under the truth.
2. Cross-entropy = the same data scored by a model; KL = the extra.
3. $\mathrm{KL} \geq 0$, zero only when the two match, not symmetric.
4. For Gaussians: $\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$ per coordinate.

END OF PART 4

# What we did, and what comes next

### WHAT WE DID

1. Surprise $= -\log p$; entropy = average surprise under the truth.
2. Cross-entropy = the same data scored by a model; KL = the extra.
3. $\mathrm{KL} \geq 0$, zero only when the two match, not symmetric.
4. For Gaussians: $\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log \sigma^2)$ per coordinate.

### NEXT: MAKE $q$ CLOSE TO THE POSTERIOR

Minimise $\mathrm{KL}(q \,\|\, p_\theta(z \mid x))$. It contains the intractable $p_\theta(x)$; Part 5 rearranges it into a loss that does not.

PART 5 OF 7

# Variational inference, and the ELBO

Part 3 wanted a simple $q$ close to the posterior of our 3. Part 4 gave a number for "close". Write the goal down: one rearrangement turns it into a loss we can compute, the ELBO. Then run it, and see why it leads to an encoder.


✓ Distributions and fitting

✓ Autoencoders

✓ The probabilistic view

✓ KL divergence

5 **Variational inference and the ELBO**

6 Reparameterisation

7 The VAE on MNIST

THE GOAL, WRITTEN DOWN

# Find the $q$ with the smallest KL to the posterior

$$q^\star = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big( q(z) \,\|\, p_\theta(z \mid x) \big)$$

**$\mathcal{Q}$: the shapes we allow**

Diagonal Gaussians. For our 2-number code: 4 numbers, a centre $m$ and two widths $s$.

THE GOAL, WRITTEN DOWN

# Find the $q$ with the smallest KL to the posterior

$$q^\star = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big( q(z) \,\|\, p_\theta(z \mid x) \big)$$

### $\mathcal{Q}$: the shapes we allow

Diagonal Gaussians. For our 2-number code: 4 numbers, a centre $m$ and two widths $s$.

### Why $q$ first?

KL averages under its first argument. $q$ is ours, so we can draw from it. The other order would need draws from the posterior: the thing we cannot get.

THE GOAL, WRITTEN DOWN

# Find the $q$ with the smallest KL to the posterior

$$q^\star = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big( q(z) \,\|\, p_\theta(z \mid x) \big)$$

### $\mathcal{Q}$: the shapes we allow

Diagonal Gaussians. For our 2-number code: 4 numbers, a centre $m$ and two widths $s$.

### Why $q$ first?

KL averages under its first argument. $q$ is ours, so we can draw from it. The other order would need draws from the posterior: the thing we cannot get.

### The catch

$p_\theta(z \mid x) = p_\theta(x, z) \,/\, p_\theta(x)$, and $p_\theta(x)$ is the integral we cannot do. We cannot even evaluate this KL.

**Four short moves get $p_\theta(x)$ out of the way.**

# Write out the KL

$$\mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big)$$

$$= \mathbb{E}_{z\sim q}\Big[\, \log q(z) - \log p_\theta(z \mid x) \,\Big]$$

# Write out the KL

$$\mathrm{KL}\big(q(z) \,\|\, p_\theta(z \mid x)\big)$$

$$= \mathbb{E}_{z\sim q}\Big[\, \log q(z) - \log p_\theta(z \mid x) \,\Big]$$

The definition from Part 4: the average, under the first argument, of the log of the ratio. The highlighted term is the one we cannot compute.

# Bayes: the posterior is the joint over the evidence

$$\mathbb{E}_q\big[\log q(z) - \log p_\theta(z \mid x)\big]$$

$$= \mathbb{E}_q\big[\log q(z) - \log p_\theta(x, z)\big] + \log p_\theta(x)$$

# Bayes: the posterior is the joint over the evidence

$$\mathbb{E}_q\big[\log q(z) - \log p_\theta(z \mid x)\big]$$

$$= \mathbb{E}_q\big[\log q(z) - \log p_\theta(x, z)\big] + \log p_\theta(x)$$

$\log p_\theta(z \mid x) = \log p_\theta(x, z) - \log p_\theta(x)$. The evidence has no $z$ in it: the same number for every draw, so it comes out of the average.

**The joint $p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$ is easy: prior times decoder. All the difficulty is now in one number.**

# Move the evidence to the other side

$$\mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big) = \mathbb{E}_q\big[\log q(z) - \log p_\theta(x, z)\big] + \log p_\theta(x)$$

$$\log p_\theta(x) = \underbrace{\mathbb{E}_q\big[\log p_\theta(x, z) - \log q(z)\big]}_{\mathrm{ELBO}(q)} + \mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big)$$

# Move the evidence to the other side

$$\mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big) = \mathbb{E}_q\big[\log q(z) - \log p_\theta(x, z)\big] + \log p_\theta(x)$$

$$\log p_\theta(x) = \underbrace{\mathbb{E}_q\big[\log p_\theta(x, z) - \log q(z)\big]}_{\mathrm{ELBO}(q)} + \mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big)$$

Rearrange, and give the highlighted average a name: the **ELBO**. Every piece of it is computable: the joint is prior times decoder, and $q$ is ours.

**Why "Evidence Lower BOund"? The last move.**

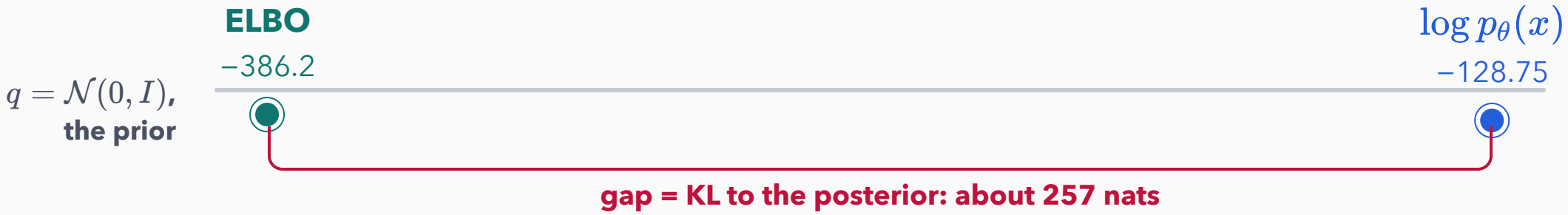
# The KL is never negative, so the ELBO is a lower bound

$$\log p_\theta(x) = \mathrm{ELBO}(q) + \underbrace{\mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big)}_{\geq 0} \;\geq\; \mathrm{ELBO}(q)$$

# The KL is never negative, so the ELBO is a lower bound

$$\log p_\theta(x) = \mathrm{ELBO}(q) + \underbrace{\mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big)}_{\geq 0} \;\geq\; \mathrm{ELBO}(q)$$

Our 3, two choices of $q$. The right end, $\log p_\theta(x)$, is fixed by the model; it does not move when $q$ changes.



# The KL is never negative, so the ELBO is a lower bound

$$\log p_\theta(x) = \mathrm{ELBO}(q) + \underbrace{\mathrm{KL}\big(q \,\|\, p_\theta(z \mid x)\big)}_{\geq 0} \;\geq\; \mathrm{ELBO}(q)$$

Our 3, two choices of $q$. The right end, $\log p_\theta(x)$, is fixed by the model; it does not move when $q$ changes.




> The total cannot move, so raising the ELBO must shrink the KL: maximise the ELBO and $q$ approaches the posterior, without ever computing $p_\theta(x)$.

# The joint is prior times decoder

$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p_\theta(x, z) - \log q(z)\big]$$

$$= \mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q(z)\Big]$$

# The joint is prior times decoder

$$\mathrm{ELBO}(q) = \mathbb{E}_q\big[\log p_\theta(x, z) - \log q(z)\big]$$

$$= \mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q(z)\Big]$$

$p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$: the generative story from Part 3. The log of a product is a sum of logs.

> **Why unpack? To see which parts are the decoder and which are the prior, and to get two terms with names.**

# The average of a sum is the sum of the averages

$$\mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q(z)\Big]$$

$$= \mathbb{E}_q\big[\log p_\theta(x \mid z)\big] + \mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

# The average of a sum is the sum of the averages

$$\mathbb{E}_q\Big[\log p_\theta(x \mid z) + \log p(z) - \log q(z)\Big]$$

$$= \mathbb{E}_q\big[\log p_\theta(x \mid z)\big] + \mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

$\mathbb{E}[a+b] = \mathbb{E}[a] + \mathbb{E}[b]$. Group the decoder term on its own, and the two terms that only involve $z$ together.

# Recognise the second average: a KL to the prior

$$\mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

$$= -\,\mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right] = -\mathrm{KL}\big(q(z)\,\|\,p(z)\big)$$

# Recognise the second average: a KL to the prior

$$\mathbb{E}_q\big[\log p(z) - \log q(z)\big]$$

$$= -\mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right] = -\mathrm{KL}\big(q(z) \,\|\, p(z)\big)$$

Flip the sign to flip the fraction. Then use the definition: $\mathrm{KL}(q\|p) = \mathbb{E}_{z\sim q}\big[\log \frac{q(z)}{p(z)}\big]$, the average extra surprise.

**And for Gaussians we already have its formula: $\frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2)$.**

PUT THE PIECES TOGETHER

# The ELBO, in symbols and in words

$$\text{ELBO} = \underbrace{\mathbb{E}_{z\sim q}\big[\log p_\theta(x \mid z)\big]}_{\text{rebuild } x \text{ from codes in its cloud}} - \underbrace{\mathrm{KL}\big(q(z) \,\|\, p(z)\big)}_{\text{keep the cloud near } \mathcal{N}(0,I)}$$

PUT THE PIECES TOGETHER

# The ELBO, in symbols and in words

$$\text{ELBO} = \underbrace{\mathbb{E}_{z\sim q}\big[\log p_\theta(x \mid z)\big]}_{\text{rebuild } x \text{ from codes in its cloud}} - \underbrace{\mathrm{KL}\big(q(z) \,\|\, p(z)\big)}_{\text{keep the cloud near } \mathcal{N}(0,I)}$$

First term: the autoencoder's reconstruction score, with the code drawn from the cloud.

Second term: the Gaussian KL from Part 4, in closed form.

**The two wishes from the end of Part 2, derived rather than invented: rebuild well, and keep codes where the prior can find them.**

CHECK IT WITH NUMBERS YOU CAN DO BY HAND

# Neat or messy writer? A two-value latent

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation $x$: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

CHECK IT WITH NUMBERS YOU CAN DO BY HAND

# Neat or messy writer? A two-value latent

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation $x$: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

evidence $p(x) = 0.5$ · true posterior $(0.8,\ 0.2)$

CHECK IT WITH NUMBERS YOU CAN DO BY HAND

# Neat or messy writer? A two-value latent

$z \in \{\text{neat}, \text{messy}\}$, prior $\frac{1}{2}, \frac{1}{2}$. Observation $x$: a thick stroke.

$$p(x \mid \text{neat}) = 0.8, \quad p(x \mid \text{messy}) = 0.2$$

evidence $p(x) = 0.5$ · true posterior $(0.8,\ 0.2)$

Our guess $q = (q_0,\ 1 - q_0)$:

$q_0$ = `0.60`

| quantity | value |
|---|---|
| $\mathbb{E}_q[\log p(x \mid z)]$ | −0.778 |
| $-\,\mathrm{KL}(q \,\|\, \text{prior})$ | −0.020 |
| ELBO | **−0.798** |
| $+\,\mathrm{KL}(q \,\|\, \text{posterior})$ | 0.105 |
| $\log p(x)$ | **−0.693** |

**Move $q$: the ELBO and the gap change, their sum never does. At $q$ = posterior $(0.8)$ the gap is 0.**

VARIATIONAL INFERENCE BY HAND, FOR OUR 3

# Four numbers, gradient ascent on the ELBO

```
x = our_3                                  # 784 pixels, fixed
m     = torch.zeros(2, requires_grad=True)   # q's centre
log_s = torch.zeros(2, requires_grad=True)   # q's log-widths
opt = torch.optim.Adam([m, log_s], lr=0.03)

for step in range(800):
    z = m + log_s.exp() * torch.randn(64, 2)    # 64 draws from q
    rec = -bce(decoder(z), x).sum(1).mean()    # E_q[log p(x|z)]
    kl  = 0.5 * (m**2 + (2*log_s).exp() - 1 - 2*log_s).sum()
    loss = -(rec - kl)                           # minus the ELBO
    opt.zero_grad(); loss.backward(); opt.step()
```

**Only $q$ is learned.** The decoder is a trained one, kept fixed; $x$ is our 3. Start at $q = \mathcal{N}(0, I)$: $m = 0$, $s = 1$.

# Four numbers, gradient ascent on the ELBO

```
x = our_3                                  # 784 pixels, fixed
m     = torch.zeros(2, requires_grad=True)   # q's centre
log_s = torch.zeros(2, requires_grad=True)   # q's log-widths
opt = torch.optim.Adam([m, log_s], lr=0.03)

for step in range(800):
    z = m + log_s.exp() * torch.randn(64, 2)     # 64 draws from q
    rec = -bce(decoder(z), x).sum(1).mean()    # E_q[log p(x|z)]
    kl  = 0.5 * (m**2 + (2*log_s).exp() - 1 - 2*log_s).sum()
    loss = -(rec - kl)                           # minus the ELBO
    opt.zero_grad(); loss.backward(); opt.step()
```

**Draw codes from $q$:** centre plus width times standard noise, exactly how Part 1 sampled from a bump. Part 6 explains why gradients flow through this line.

# Four numbers, gradient ascent on the ELBO

```
x = our_3                                  # 784 pixels, fixed
m     = torch.zeros(2, requires_grad=True)   # q's centre
log_s = torch.zeros(2, requires_grad=True)   # q's log-widths
opt = torch.optim.Adam([m, log_s], lr=0.03)

for step in range(800):
    z = m + log_s.exp() * torch.randn(64, 2)    # 64 draws from q
    rec = -bce(decoder(z), x).sum(1).mean()    # E_q[log p(x|z)]
    kl  = 0.5 * (m**2 + (2*log_s).exp() - 1 - 2*log_s).sum()
    loss = -(rec - kl)                           # minus the ELBO
    opt.zero_grad(); loss.backward(); opt.step()
```

**The two terms:** reconstruction, averaged over the 64 draws; the KL to the prior by the Gaussian formula from Part 4.

VARIATIONAL INFERENCE BY HAND, FOR OUR 3

# Four numbers, gradient ascent on the ELBO

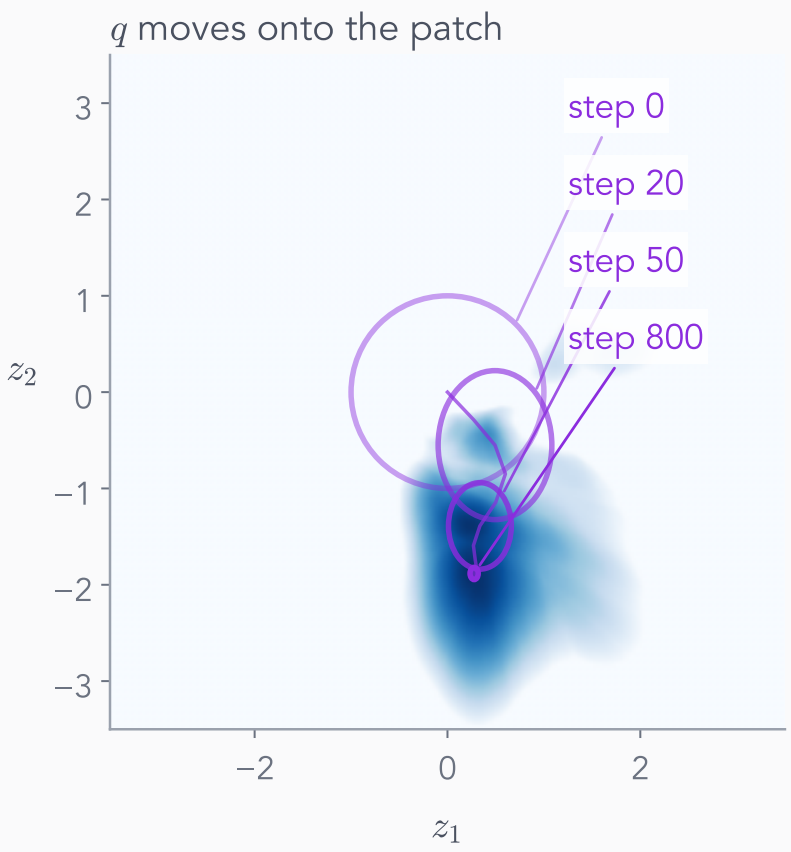
```
x = our_3                                  # 784 pixels, fixed
m     = torch.zeros(2, requires_grad=True)   # q's centre
log_s = torch.zeros(2, requires_grad=True)   # q's log-widths
opt = torch.optim.Adam([m, log_s], lr=0.03)

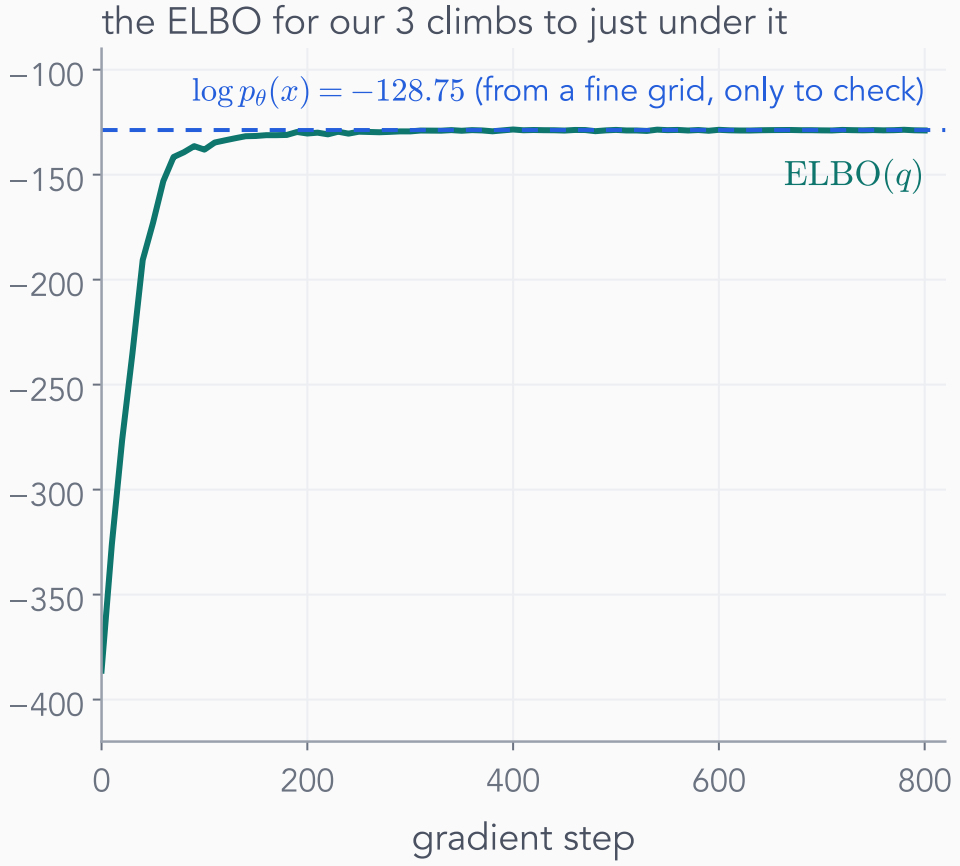
for step in range(800):
    z = m + log_s.exp() * torch.randn(64, 2)    # 64 draws from q
    rec = -bce(decoder(z), x).sum(1).mean()    # E_q[log p(x|z)]
    kl  = 0.5 * (m**2 + (2*log_s).exp() - 1 - 2*log_s).sum()
    loss = -(rec - kl)                              # minus the ELBO
    opt.zero_grad(); loss.backward(); opt.step()
```

**The same three-line loop** as fitting one Gaussian in Part 1. The ELBO rises from −386.0 to −128.8.

WATCH $q$ TRAVEL

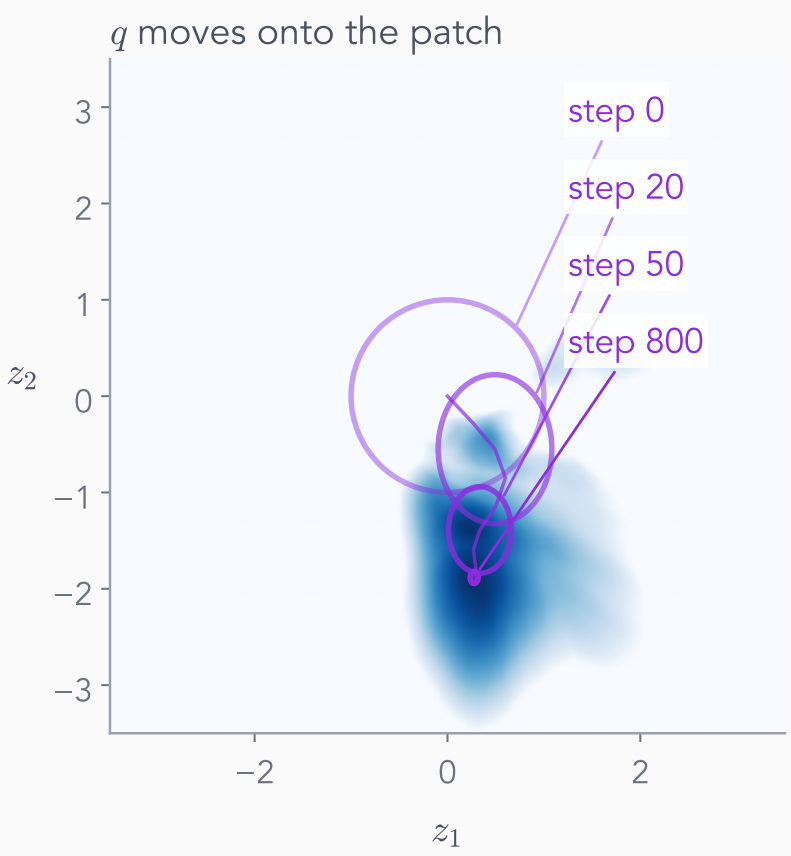
# From $\mathcal{N}(0, I)$ to our 3's patch in 800 steps

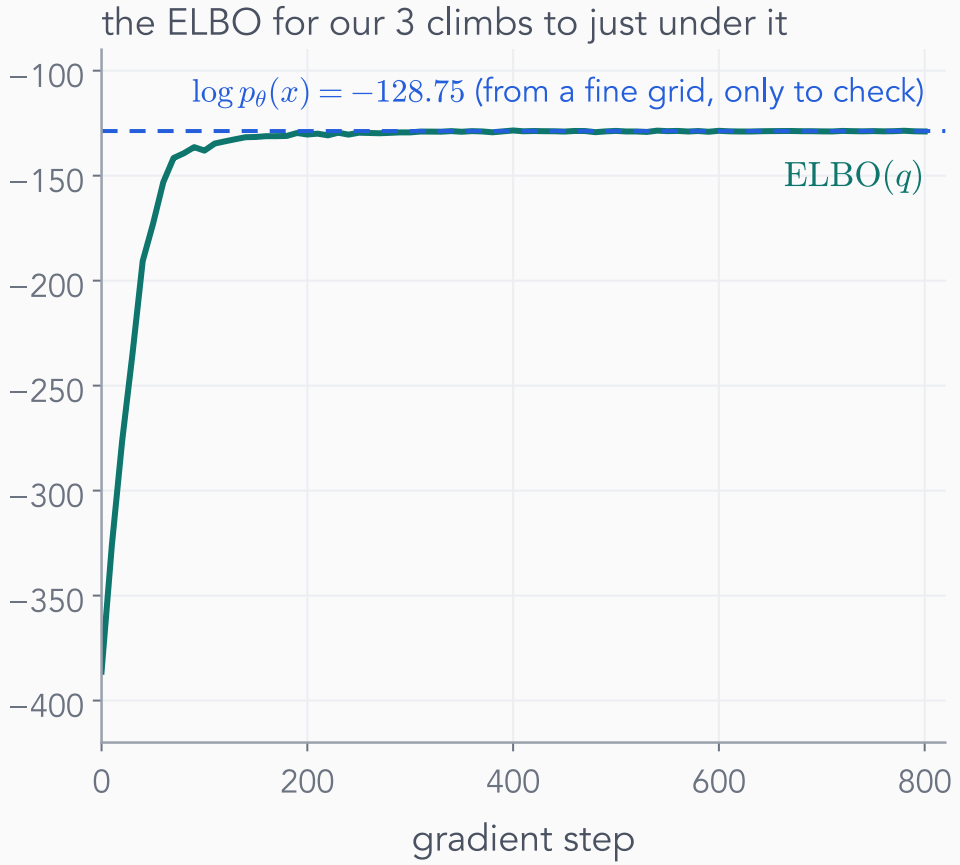


WATCH $q$ TRAVEL

# From $\mathcal{N}(0, I)$ to our 3's patch in 800 steps




**Final ELBO −128.83, just under $\log p_\theta(x) = -128.75$. The gap, $\mathrm{KL}(q \,\|\, \text{posterior})$, is 0.08 nats: $q$ is almost exactly right.**

INSIDE THOSE 800 STEPS

# The two terms pull against each other

| step | reconstruction $\mathbb{E}_q[\log p_\theta(x \mid z)]$ | $-\,\mathrm{KL}(q \,\|\, \text{prior})$ | ELBO |
|---|---|---|---|
| $0 \cdot q$ is the prior | −386.2 | — 0.0 | −386.2 |
| | | | |
| | | | |

INSIDE THOSE 800 STEPS

# The two terms pull against each other

| step | reconstruction $\mathbb{E}_q[\log p_\theta(x \mid z)]$ | $-\,\mathrm{KL}(q \,\|\, \text{prior})$ | ELBO |
|---|---|---|---|
| $0 \cdot q$ is the prior | −386.2 | — 0.0 | −386.2 |
| 20 | −275.9 | — 0.53 | −276.4 |
| 50 | −170.9 | — 2.1 | −173.0 |

INSIDE THOSE 800 STEPS

# The two terms pull against each other

| step | reconstruction $\mathbb{E}_q[\log p_\theta(x \mid z)]$ | $-\,\mathrm{KL}(q \,\|\, \text{prior})$ | ELBO |
|---|---|---|---|
| $0 \cdot q$ is the prior | −386.2 | — 0.0 | −386.2 |
| 20 | −275.9 | — 0.53 | −276.4 |
| 50 | −170.9 | — 2.1 | −173.0 |
| **800** | **−122.2** | — **6.73** | **−128.9** |

INSIDE THOSE 800 STEPS

# The two terms pull against each other

| step | reconstruction $\mathbb{E}_q[\log p_\theta(x \mid z)]$ | $-\,\mathrm{KL}(q \,\|\, \text{prior})$ | ELBO |
|---|---|---|---|
| $0 \cdot q$ is the prior | −386.2 | — 0.0 | −386.2 |
| 20 | −275.9 | — 0.53 | −276.4 |
| 50 | −170.9 | — 2.1 | −173.0 |
| **800** | **−122.2** | **— 6.73** | **−128.9** |

Reconstruction gains about 264 nats: codes from $q$ now decode to our 3.

The KL to the prior grows from 0 to 6.73: a narrow cloud far from the origin costs something.

**The ELBO is a trade: a $q$ precise enough to rebuild the image, no more precise than it needs to be.**

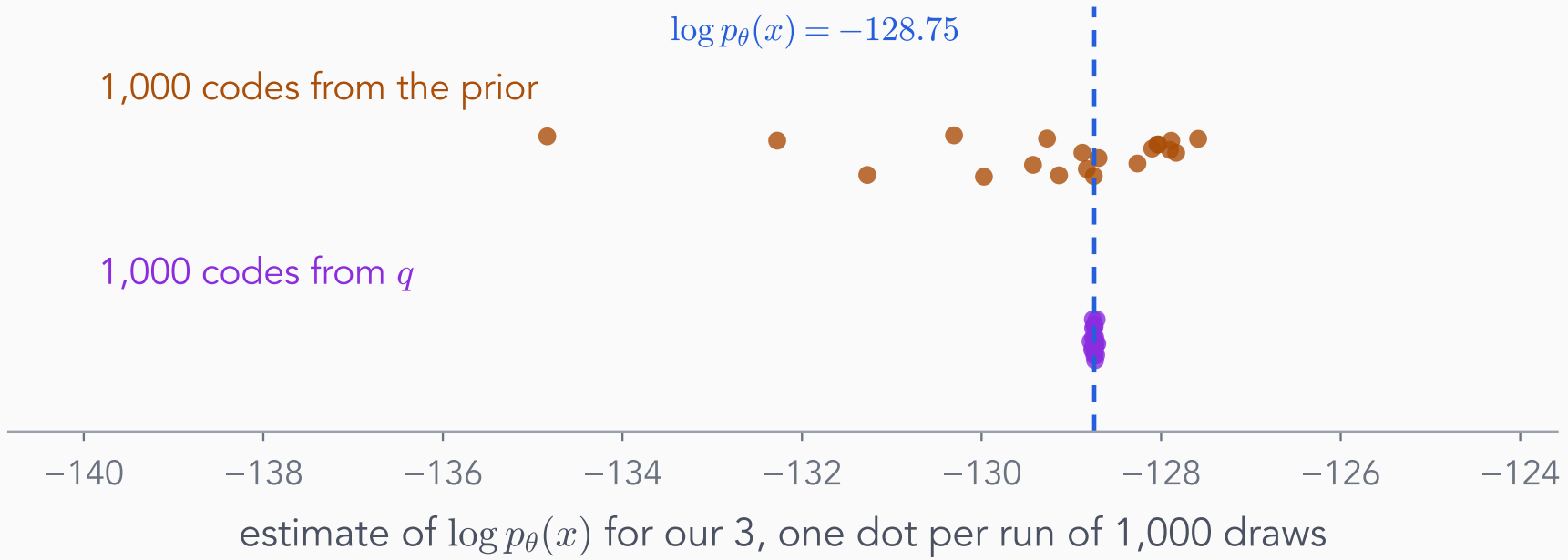
A BONUS: WITH $q$, EVEN $p_\theta(x)$ BECOMES EASY TO ESTIMATE

# Sample codes where the patch is, then reweight

$$p_\theta(x) = \int q(z)\,\frac{p(z)\,p_\theta(x \mid z)}{q(z)}\,dz = \mathbb{E}_{z\sim q}\left[\frac{p_\theta(x,z)}{q(z)}\right] \approx \frac{1}{S}\sum_{s=1}^{S}\frac{p_\theta(x,z_s)}{q(z_s)}$$

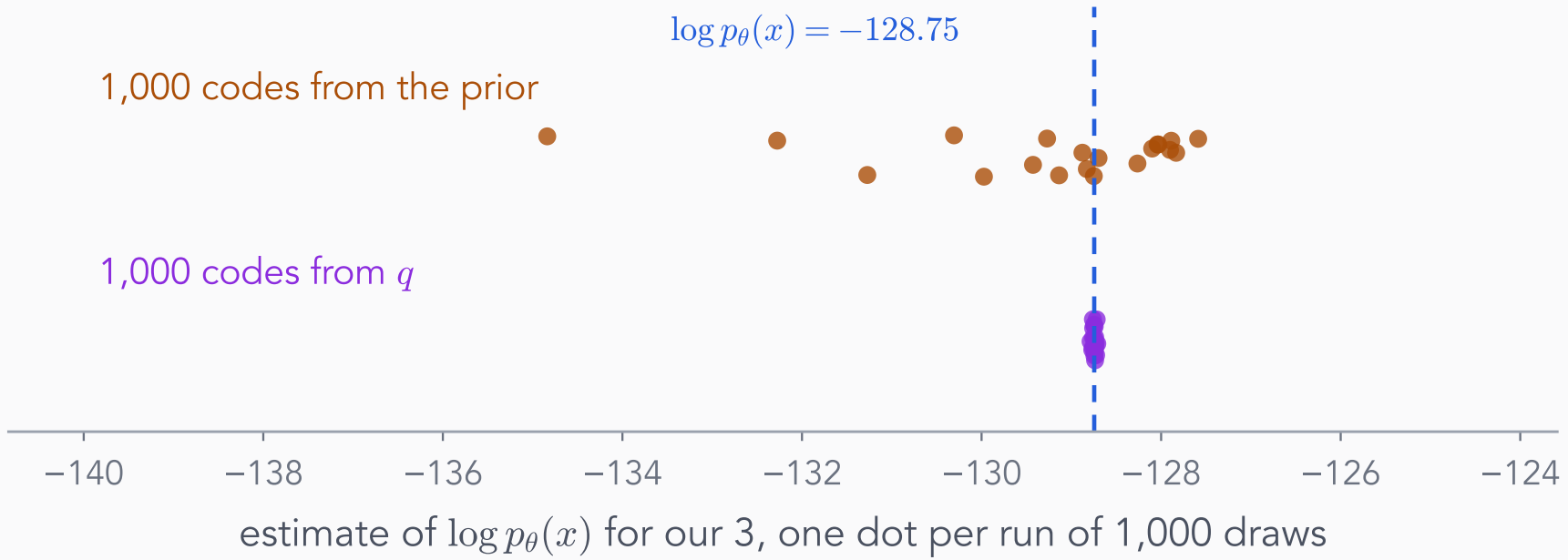
# Sample codes where the patch is, then reweight

$$p_\theta(x) = \int q(z)\,\frac{p(z)\,p_\theta(x \mid z)}{q(z)}\,dz = \mathbb{E}_{z\sim q}\left[\frac{p_\theta(x,z)}{q(z)}\right] \approx \frac{1}{S}\sum_{s=1}^{S}\frac{p_\theta(x,z_s)}{q(z_s)}$$



# Sample codes where the patch is, then reweight

$$p_\theta(x) = \int q(z)\,\frac{p(z)\,p_\theta(x \mid z)}{q(z)}\,dz = \mathbb{E}_{z\sim q}\left[\frac{p_\theta(x,z)}{q(z)}\right] \approx \frac{1}{S}\sum_{s=1}^{S}\frac{p_\theta(x,z_s)}{q(z_s)}$$

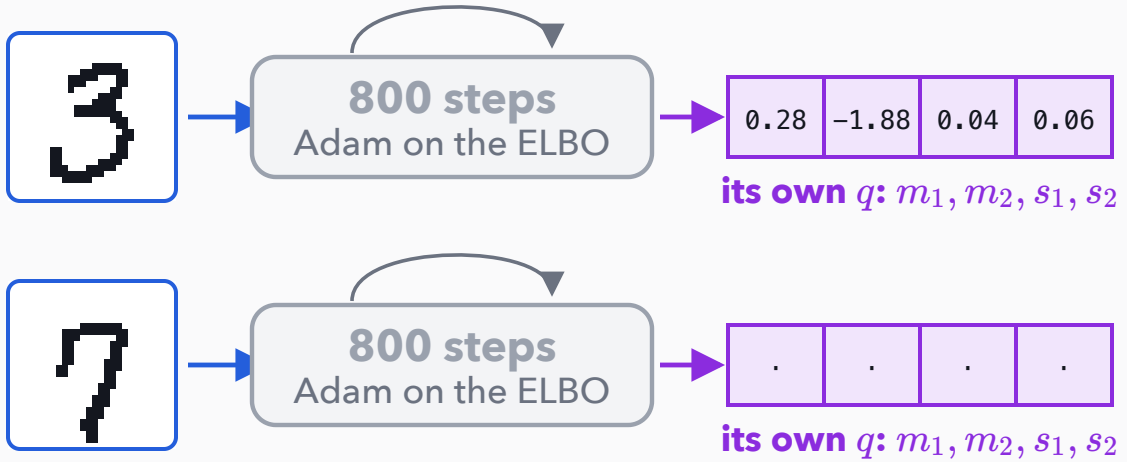



**20 runs of 1,000 draws each. From the prior, the estimates scatter from −134.8 to −127.6. From $q$: −128.74, give or take 0.017.**

800 STEPS FOR ONE IMAGE

# Amortised inference: learn to predict $q$

**VI by hand: one optimisation per image**




× 60,000 training images, redone each time the decoder changes, and again for every new image

800 STEPS FOR ONE IMAGE

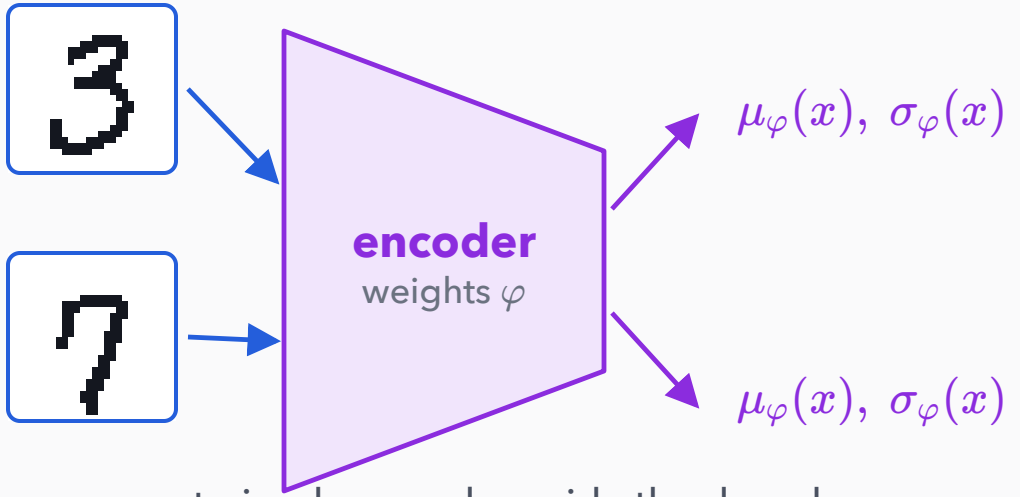
# Amortised inference: learn to predict $q$

**VI by hand: one optimisation per image**




× 60,000 training images, redone each time the decoder changes, and again for every new image

**Amortised: one network predicts q**




trained once, alongside the decoder;
**then any image, even a new one: one forward pass**

800 STEPS FOR ONE IMAGE

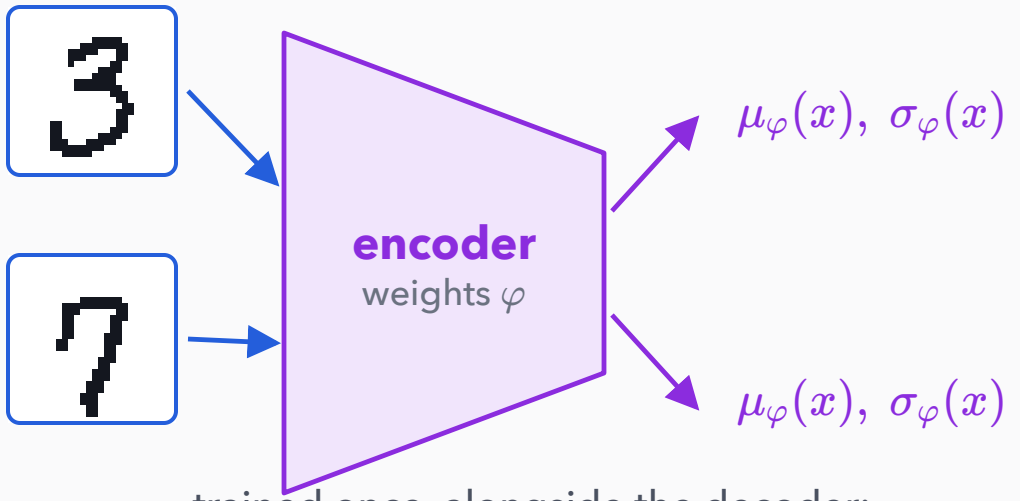
# Amortised inference: learn to predict $q$

**VI by hand: one optimisation per image**




× 60,000 training images, redone each time the decoder changes, and again for every new image

**Amortised: one network predicts q**




trained once, alongside the decoder;
**then any image, even a new one: one forward pass**

> Our 3: ELBO −128.83 by hand in 800 steps, −129.3 from the encoder in one pass. Pay once during training; a little worse, thousands of times cheaper.

PUT THE PIECES TOGETHER

# Encoder + decoder + ELBO = the variational autoencoder

$$\max_{\theta,\varphi} \sum_{i=1}^{N} \left( \mathbb{E}_{z \sim q_\varphi(z|x_i)} \left[ \log p_\theta(x_i \mid z) \right] - \mathrm{KL}\left( q_\varphi(z \mid x_i) \,\|\, p(z) \right) \right)$$

PUT THE PIECES TOGETHER

# Encoder + decoder + ELBO = the variational autoencoder

$$\max_{\theta,\varphi} \sum_{i=1}^{N} \left( \mathbb{E}_{z \sim q_\varphi(z|x_i)} \left[ \log p_\theta(x_i \mid z) \right] - \mathrm{KL}\left( q_\varphi(z \mid x_i) \,\|\, p(z) \right) \right)$$

**"Variational"**

$q$ always depended on the image; now we write it $q_\varphi(z \mid x)$: the best Gaussian under the ELBO, predicted by a network.

PUT THE PIECES TOGETHER

# Encoder + decoder + ELBO = the variational autoencoder

$$\max_{\theta,\,\varphi} \sum_{i=1}^{N} \left( \mathbb{E}_{z \sim q_\varphi(z|x_i)} \left[ \log p_\theta(x_i \mid z) \right] - \mathrm{KL}\left( q_\varphi(z \mid x_i) \,\|\, p(z) \right) \right)$$

**"Variational"**

$q$ always depended on the image; now we write it $q_\varphi(z \mid x)$: the best Gaussian under the ELBO, predicted by a network.

**"Autoencoder"**

image → encoder → code → decoder → image: the shape from Part 2, now with a cloud for a code.

PUT THE PIECES TOGETHER

# Encoder + decoder + ELBO = the variational autoencoder

$$\max_{\theta,\,\varphi} \sum_{i=1}^{N} \left( \mathbb{E}_{z \sim q_\varphi(z \mid x_i)} \left[ \log p_\theta(x_i \mid z) \right] - \mathrm{KL}\left( q_\varphi(z \mid x_i) \,\|\, p(z) \right) \right)$$

### "Variational"

$q$ always depended on the image; now we write it $q_\varphi(z \mid x)$: the best Gaussian under the ELBO, predicted by a network.

### "Autoencoder"

image → encoder → code → decoder → image: the shape from Part 2, now with a cloud for a code.

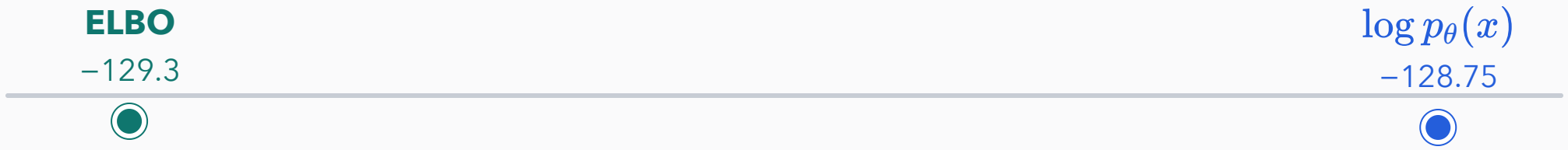
### Trained together

Pushing on $\theta$ raises the ELBO mostly by making the data more probable: a better model. Pushing on $\varphi$ shrinks the gap: a better $q$.

> **Kingma & Welling (2014) and Rezende et al. (2014): one network for the likelihood, one for the posterior, one loss for both.**

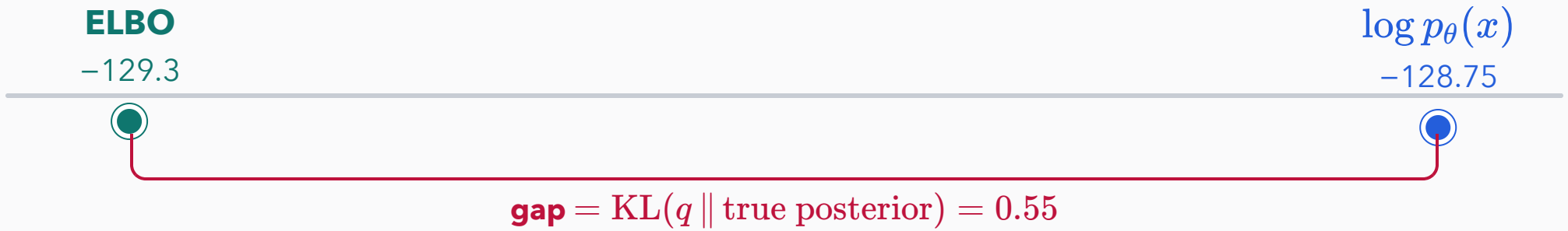
THE GAP FOR THE ENCODER'S $q$, MEASURED ON OUR 3

# Two different KLs, two different jobs



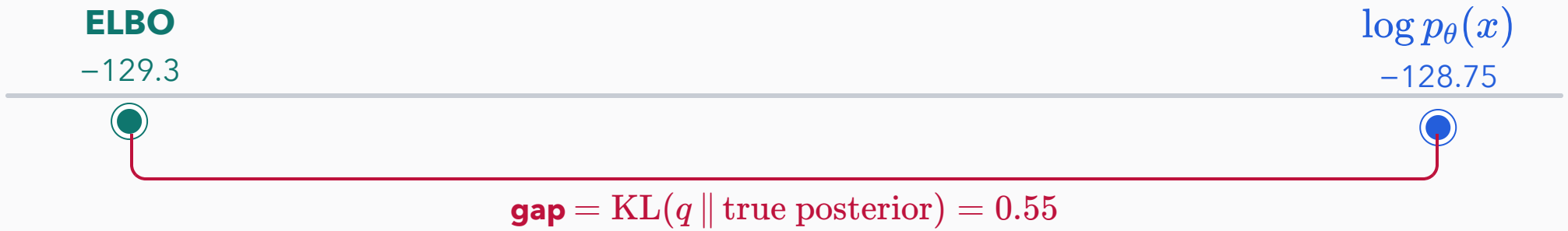
THE GAP FOR THE ENCODER'S $q$, MEASURED ON OUR 3

# Two different KLs, two different jobs



THE GAP FOR THE ENCODER'S $q$, MEASURED ON OUR 3

# Two different KLs, two different jobs




**In the loss:** $\mathrm{KL}(q \,\|\, \text{prior}) = 6.97$ nats.
Computable by formula.

**In the gap:** $\mathrm{KL}(q \,\|\, \text{true posterior}) = 0.55$ nats.
Needs $p_\theta(x)$; we only got it from the grid.

> **VI by hand closed the gap to 0.08 nats in 800 steps; the encoder's single pass leaves 0.55. That small loss is the price of amortising.**

END OF PART 5

# What we did, and what comes next

### WHAT WE DID

1. Goal: minimise $\mathrm{KL}(q \,\|\, \text{posterior})$. It contains $p_\theta(x)$.
2. Rearranged: $\log p_\theta(x) = \mathrm{ELBO} + \mathrm{KL}$. Maximising the ELBO minimises the KL.
3. ELBO = reconstruction $-\ \mathrm{KL}(q \,\|\, \text{prior})$: both computable.
4. VI by hand fits our 3's $q$ in 800 steps; an encoder predicts it in one pass. That is the VAE.

END OF PART 5

# What we did, and what comes next

### WHAT WE DID

1. Goal: minimise $\mathrm{KL}(q \,\|\, \text{posterior})$. It contains $p_\theta(x)$.
2. Rearranged: $\log p_\theta(x) = \mathrm{ELBO} + \mathrm{KL}$. Maximising the ELBO minimises the KL.
3. ELBO = reconstruction $-\ \mathrm{KL}(q \,\|\, \text{prior})$: both computable.
4. VI by hand fits our 3's $q$ in 800 steps; an encoder predicts it in one pass. That is the VAE.

### NEXT: GRADIENTS THROUGH A RANDOM DRAW

The loss draws $z$ from the encoder's cloud. Part 6: why writing the draw as $\mu + \sigma\varepsilon$ lets gradients reach the encoder.

PART 6 OF 7

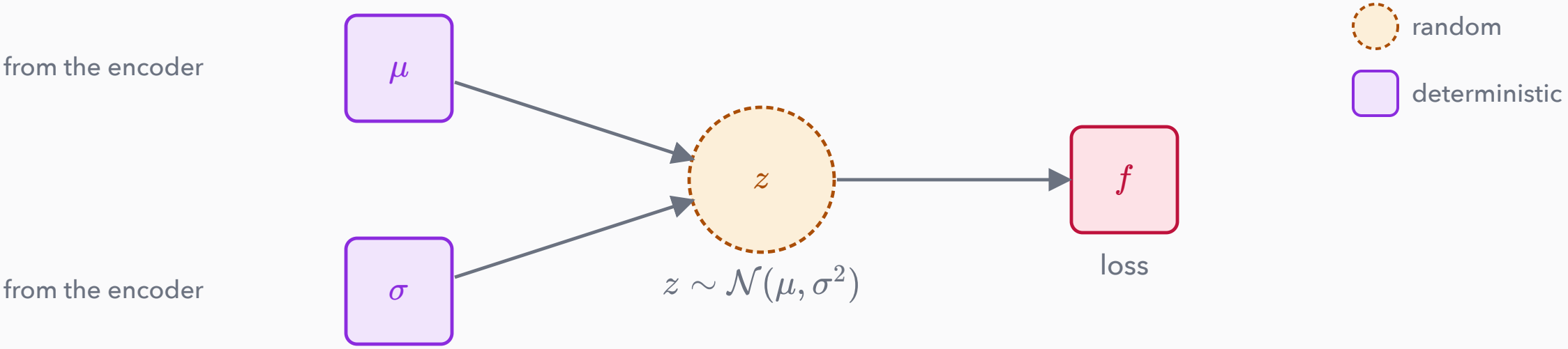
# Backpropagating through a random draw

VI by hand wrote each draw as $m + s \cdot \varepsilon$ without comment. The VAE loss also uses a $z$ drawn from the encoder's cloud, and to train the encoder, gradients must get from the loss back to $\mu$ and $\sigma$ through that draw. Here is why the way we write the draw is the whole trick.


✓ Distributions and fitting
✓ Autoencoders
✓ The probabilistic view
✓ KL divergence
✓ Variational inference and the ELBO
6 **Reparameterisation**
7 The VAE on MNIST

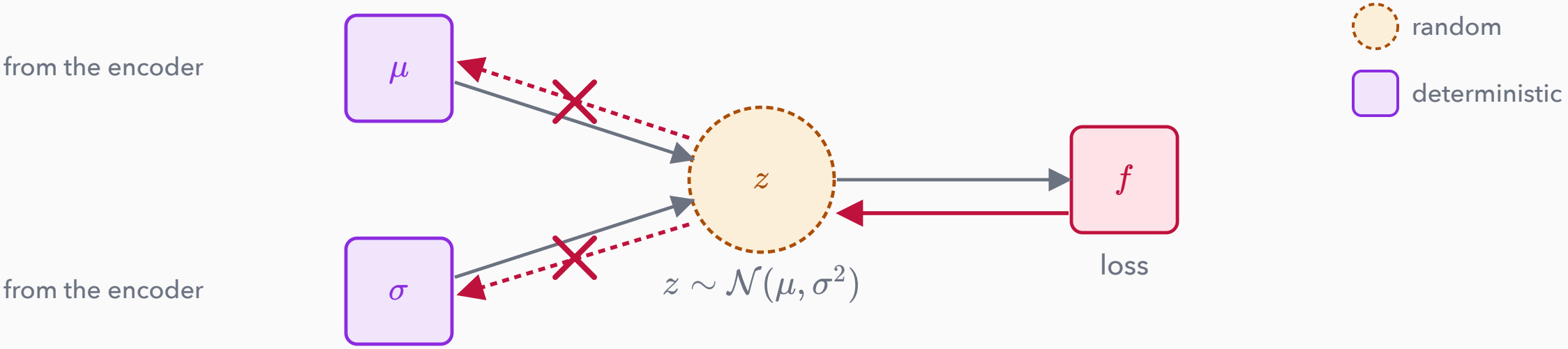
THE PROBLEM

# A sampling step sits in the middle of the graph



THE PROBLEM

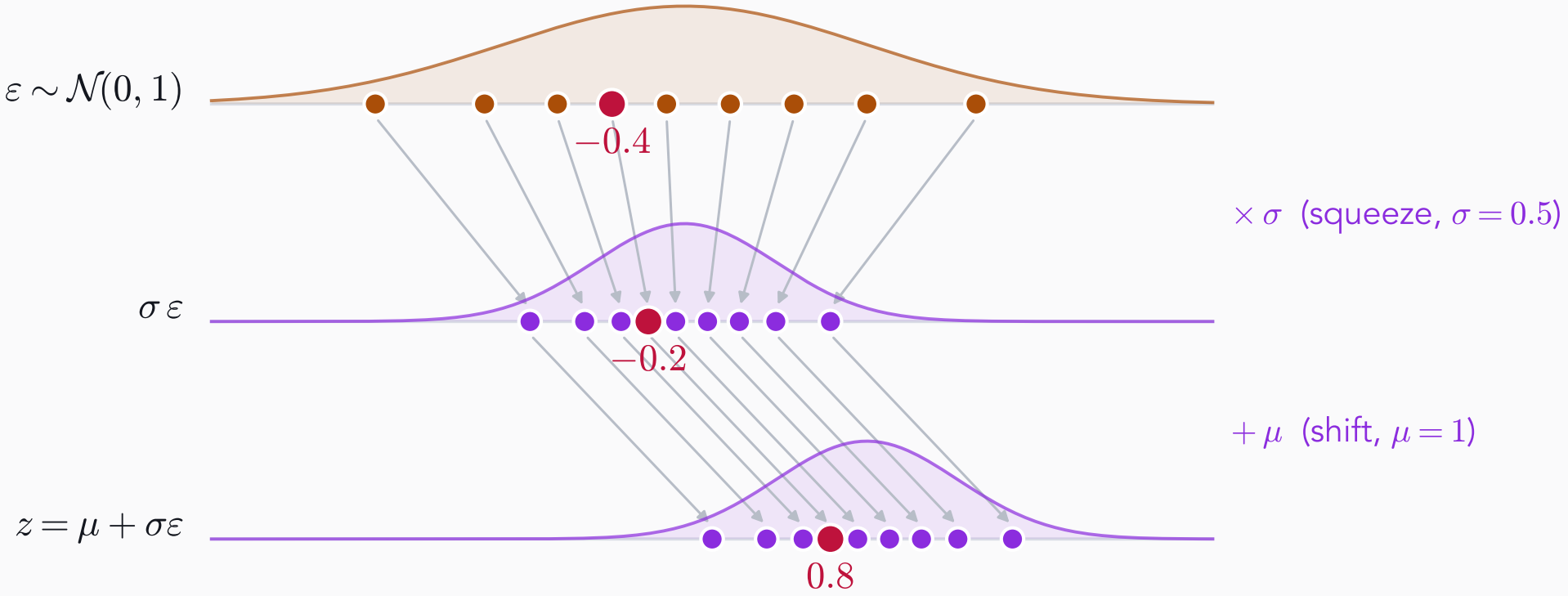
# A sampling step sits in the middle of the graph




**the loss knows how $z$ should move, but $z$ is a fresh random draw: how does it change when $\mu$ or $\sigma$ changes a little?**

THE TRICK

# Draw standard noise first, then stretch and shift it

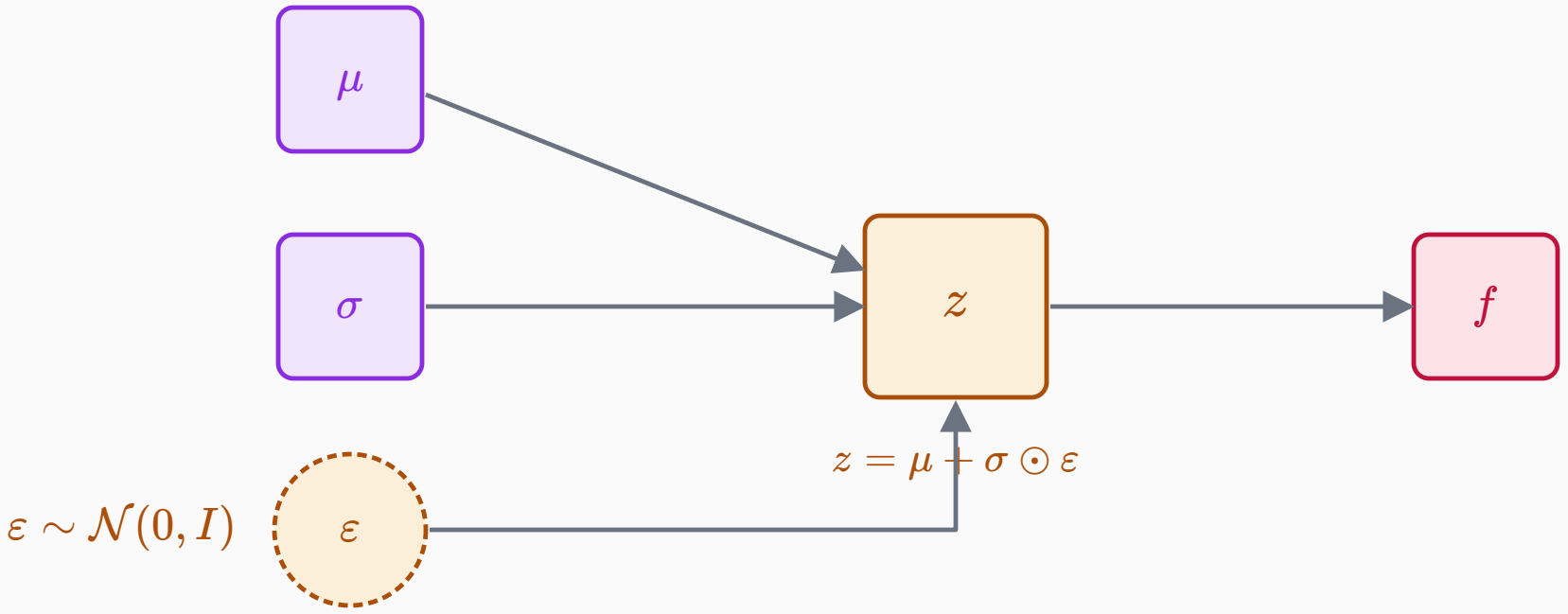



$$z = \mu + \sigma\,\varepsilon, \quad \varepsilon \sim \mathcal{N}(0,1) \quad \Longrightarrow \quad z \sim \mathcal{N}(\mu, \sigma^2)$$

> **Same distribution as before (the $\mu = 1, \sigma = 0.5$ cloud of the Gaussian KL slide). But the randomness now enters as an input, $\varepsilon$, that does not depend on any weights.**

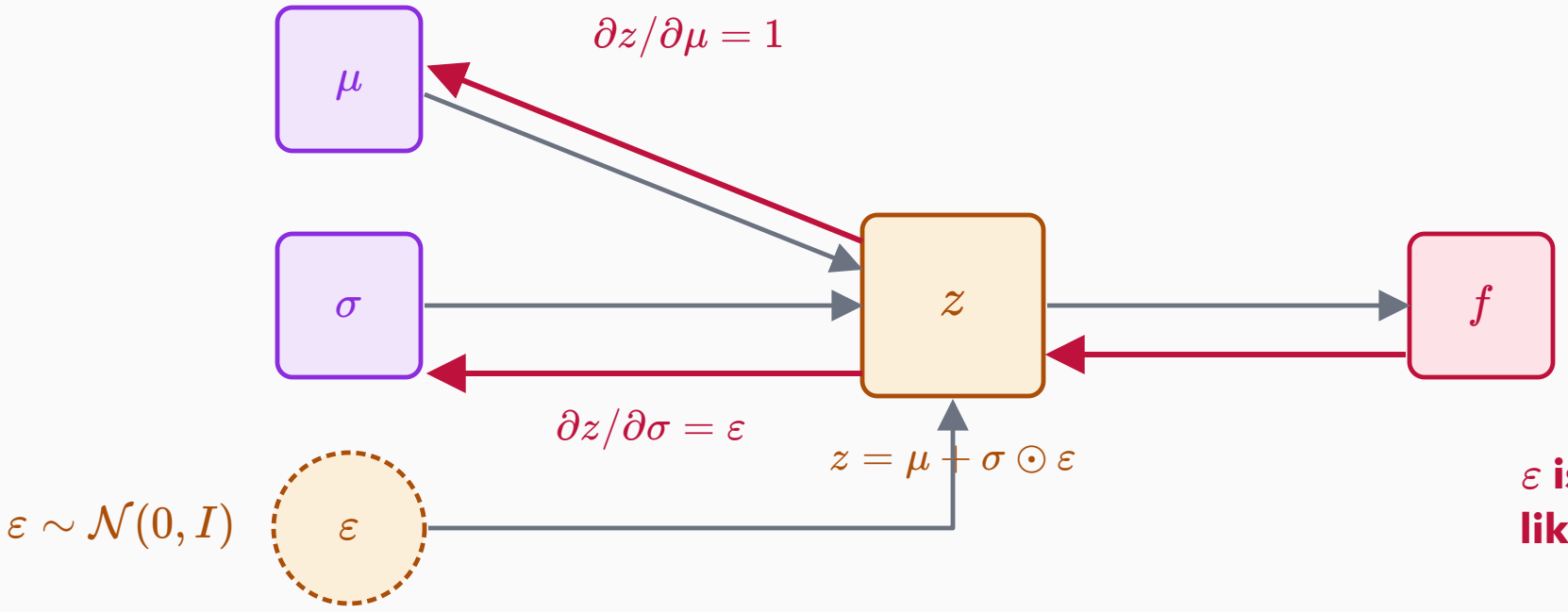
THE FIX

# Now the path from the loss to $\mu$ and $\sigma$ is ordinary arithmetic



THE FIX

# Now the path from the loss to $\mu$ and $\sigma$ is ordinary arithmetic




**$\varepsilon$ is just an input, like a pixel value**

# Take $f(z) = z^2$ as the loss

$$z = \mu + \sigma\varepsilon \qquad \Rightarrow \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon, \qquad \frac{\partial f}{\partial z} = 2z$$

# Take $f(z) = z^2$ as the loss

$$z = \mu + \sigma\varepsilon \qquad \Rightarrow \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon, \qquad \frac{\partial f}{\partial z} = 2z$$

$$\frac{\partial f}{\partial \mu} = \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \mu} = 2z \cdot 1$$

# Take $f(z) = z^2$ as the loss

$$z = \mu + \sigma\varepsilon \qquad \Rightarrow \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon, \qquad \frac{\partial f}{\partial z} = 2z$$

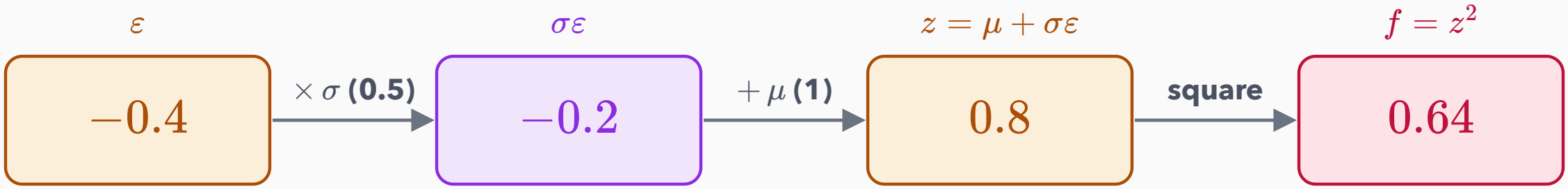
$$\frac{\partial f}{\partial \mu} = \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \mu} = 2z \cdot 1$$

$$\frac{\partial f}{\partial \sigma} = \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \sigma} = 2z \cdot \varepsilon$$

The noise value $\varepsilon$ appears in the $\sigma$-gradient: a draw far from the centre pushes $\sigma$ harder.

## NOW PLUG IN NUMBERS

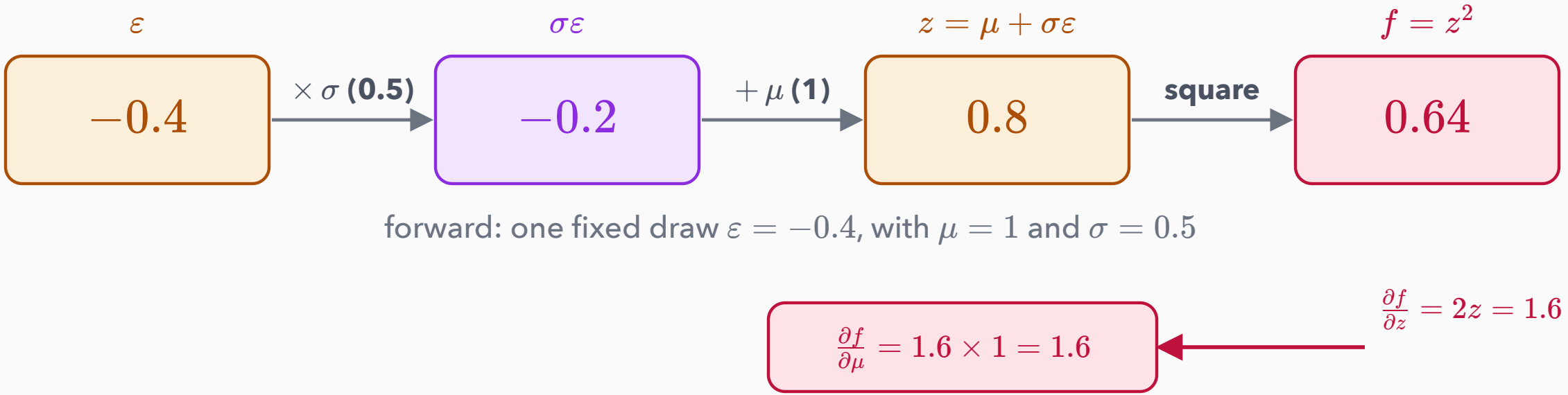
$$\mu = 1,\ \sigma = 0.5,\ \varepsilon = -0.4$$




forward: one fixed draw $\varepsilon = -0.4$, with $\mu = 1$ and $\sigma = 0.5$

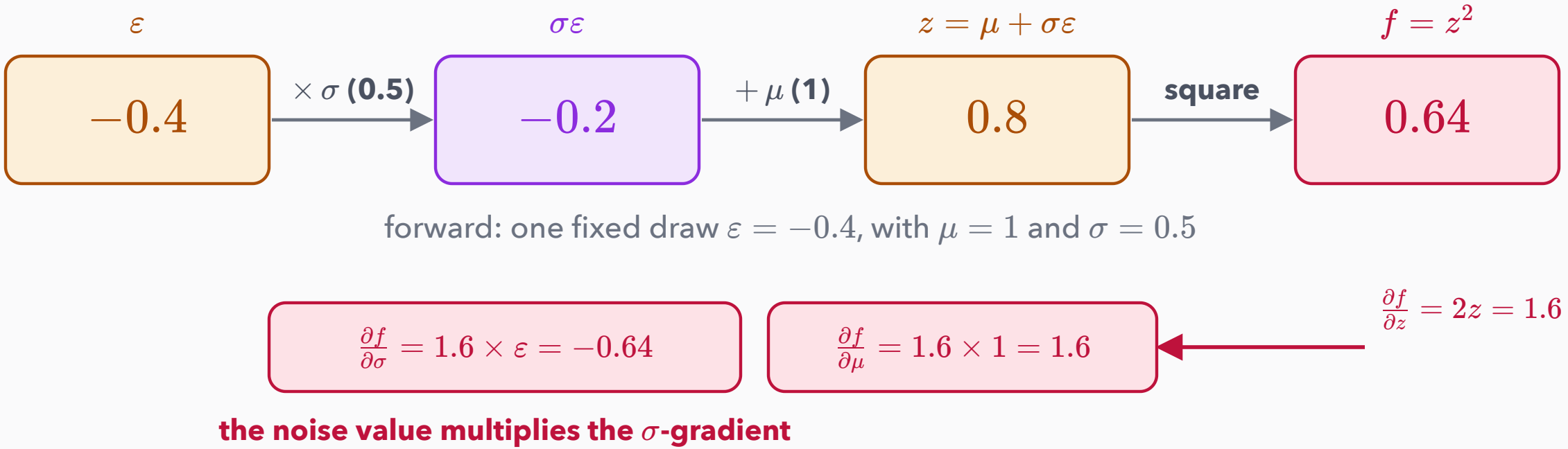
## NOW PLUG IN NUMBERS

$$\mu = 1,\ \sigma = 0.5,\ \varepsilon = -0.4$$



## NOW PLUG IN NUMBERS

$$\mu = 1,\ \sigma = 0.5,\ \varepsilon = -0.4$$



ONE MORE LINK IN THE CHAIN

# The network outputs $v = \log \sigma^2$, not $\sigma$

$$\sigma = e^{v/2}$$

$v$ can be any real number; $\sigma$ is always positive

ONE MORE LINK IN THE CHAIN

# The network outputs $v = \log \sigma^2$, not $\sigma$

$$\sigma = e^{v/2}$$

$v$ can be any real number; $\sigma$ is always positive

$$\frac{\partial \sigma}{\partial v} = \frac{1}{2} e^{v/2} = \frac{\sigma}{2} = 0.25$$

derivative of $e^{v/2}$

ONE MORE LINK IN THE CHAIN

# The network outputs $v = \log \sigma^2$, not $\sigma$

$$\sigma = e^{v/2}$$

$v$ can be any real number; $\sigma$ is always positive

$$\frac{\partial \sigma}{\partial v} = \tfrac{1}{2} e^{v/2} = \frac{\sigma}{2} = 0.25$$

derivative of $e^{v/2}$

$$\frac{\partial f}{\partial v} = \frac{\partial f}{\partial \sigma} \cdot \frac{\partial \sigma}{\partial v} = -0.64 \times 0.25 = -0.16$$

one more chain-rule factor

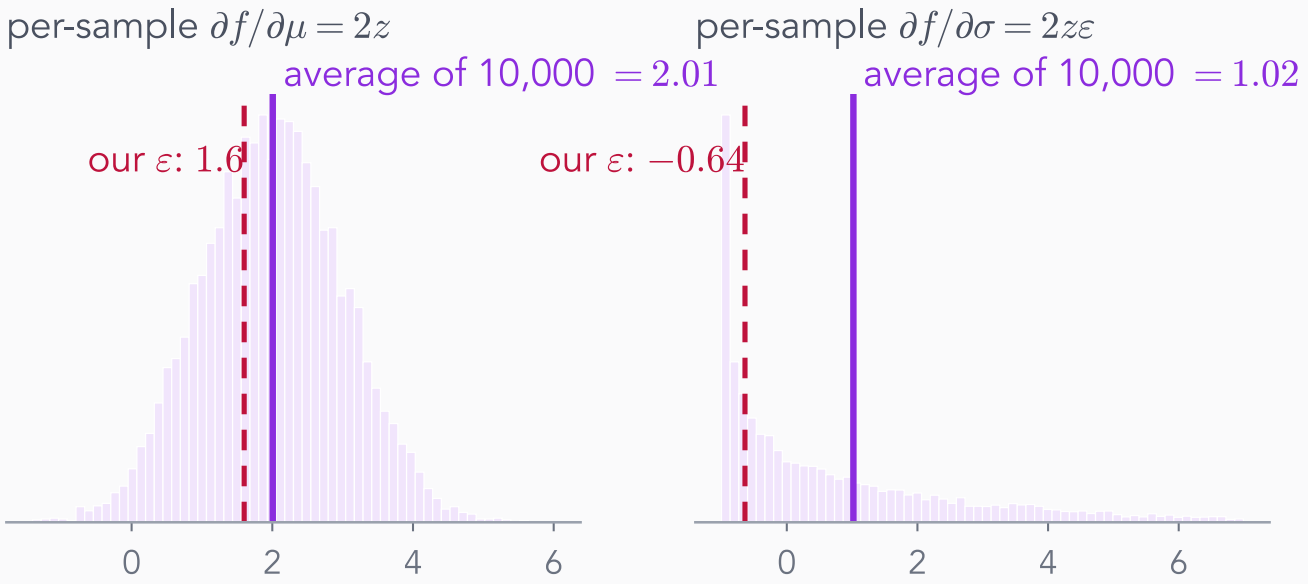
**Autograd does exactly this. The point of doing it by hand once: no step is magic.**

# One draw is noisy; the average is exactly right

$$\mathbb{E}_\varepsilon\big[f\big] = \mathbb{E}\big[(\mu+\sigma\varepsilon)^2\big] = \mu^2+\sigma^2 \quad \Rightarrow \quad \frac{\partial}{\partial\mu}\mathbb{E}[f] = 2\mu = 2, \qquad \frac{\partial}{\partial\sigma}\mathbb{E}[f] = 2\sigma = 1$$

# One draw is noisy; the average is exactly right

$$\mathbb{E}_\varepsilon\big[f\big] = \mathbb{E}\big[(\mu+\sigma\varepsilon)^2\big] = \mu^2+\sigma^2 \quad \Rightarrow \quad \frac{\partial}{\partial\mu}\mathbb{E}[f] = 2\mu = 2, \qquad \frac{\partial}{\partial\sigma}\mathbb{E}[f] = 2\sigma = 1$$




**Our single $\varepsilon$ gave 1.6 and $-0.64$. Averaged over 10000 draws: 2.01 and 1.02. SGD only needs the right answer on average.**

IN PYTORCH

# Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)     # sigma = exp(v / 2)
eps = torch.randn_like(std)       # standard normal, no grad
z   = mu + std * eps              # differentiable in mu, std
```

IN PYTORCH

# Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)      # sigma = exp(v / 2)
eps = torch.randn_like(std)        # standard normal, no grad
z   = mu + std * eps               # differentiable in mu, std
```

Why output $\log \sigma^2$? It can be any real number; $\sigma$ must be positive.

IN PYTORCH

# Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)     # sigma = exp(v / 2)
eps = torch.randn_like(std)       # standard normal, no grad
z   = mu + std * eps              # differentiable in mu, std
```

Why output $\log \sigma^2$? It can be any real number; $\sigma$ must be positive.

IN PYTORCH

# Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)      # sigma = exp(v / 2)
eps = torch.randn_like(std)        # standard normal, no grad
z   = mu + std * eps               # differentiable in mu, std
```

Why output $\log \sigma^2$? It can be any real number; $\sigma$ must be positive.

`Normal(mu, std).rsample()` does the same thing; `.sample()` drops the gradient.

For our 3: $\mu = (0.3,\ -1.93)$, $\varepsilon = (0.62,\ -1.15)$ gives $z = (0.33,\ -1.99)$.

END OF PART 6

# What we did, and what comes next

### WHAT WE DID

1. Sampling $z$ directly cuts the gradient path to $\mu$ and $\sigma$.
2. Write $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$: same distribution, randomness as an input.
3. $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$; one draw is noisy, the average is exact.
4. In PyTorch: three lines, or `rsample()`.

END OF PART 6

# What we did, and what comes next

### WHAT WE DID

1. Sampling $z$ directly cuts the gradient path to $\mu$ and $\sigma$.
2. Write $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$: same distribution, randomness as an input.
3. $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$; one draw is noisy, the average is exact.
4. In PyTorch: three lines, or `rsample()`.

### NEXT: EVERYTHING TOGETHER

Encoder, cloud, reparameterised sample, decoder, ELBO: train it on MNIST and follow our 3 through every step.

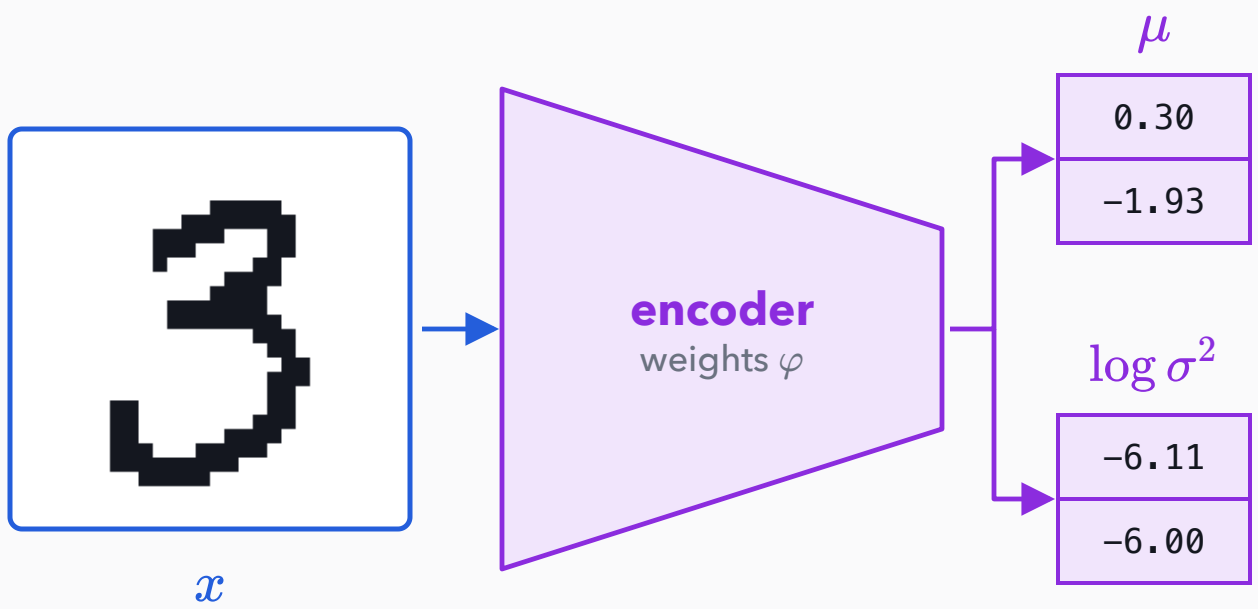
PART 7 OF 7

# The variational autoencoder, end to end

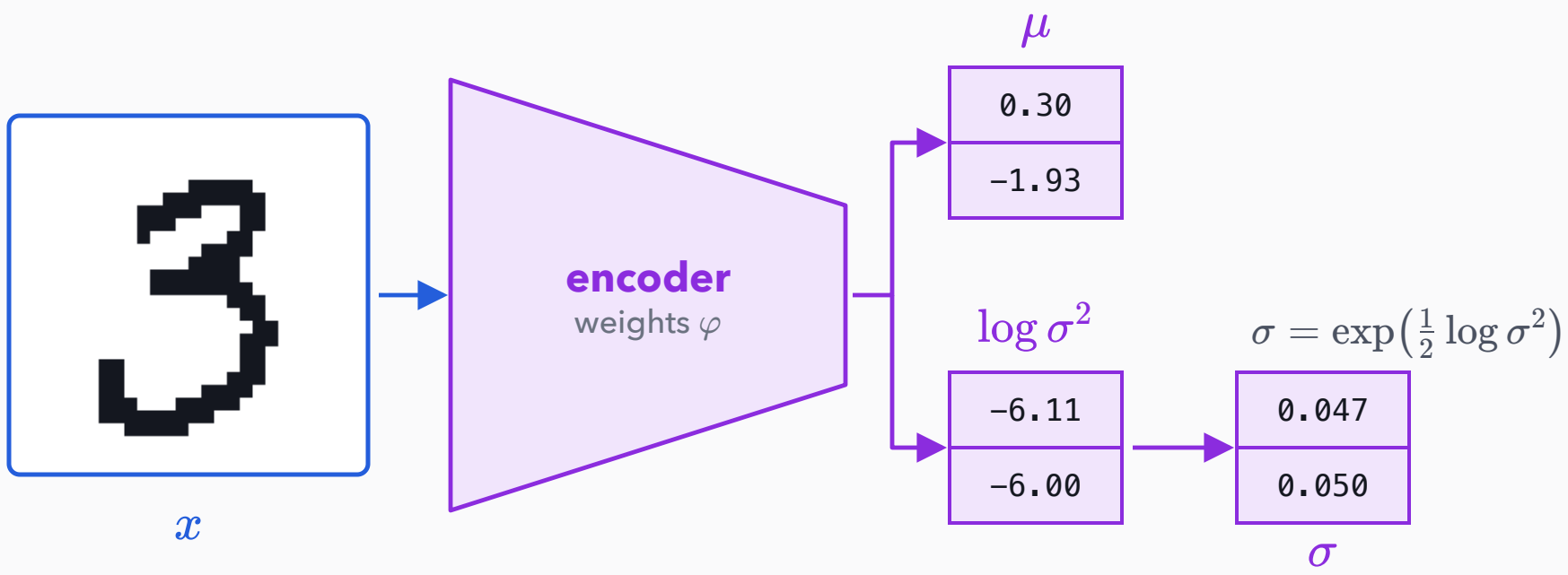
Every piece is in place: encoder, cloud, KL, ELBO, reparameterisation. Follow our 3 through the trained network, then generate.


- ✓ Distributions and fitting
- ✓ Autoencoders
- ✓ The probabilistic view
- ✓ KL divergence
- ✓ Variational inference and the ELBO
- ✓ Reparameterisation
- 7 **The VAE on MNIST**

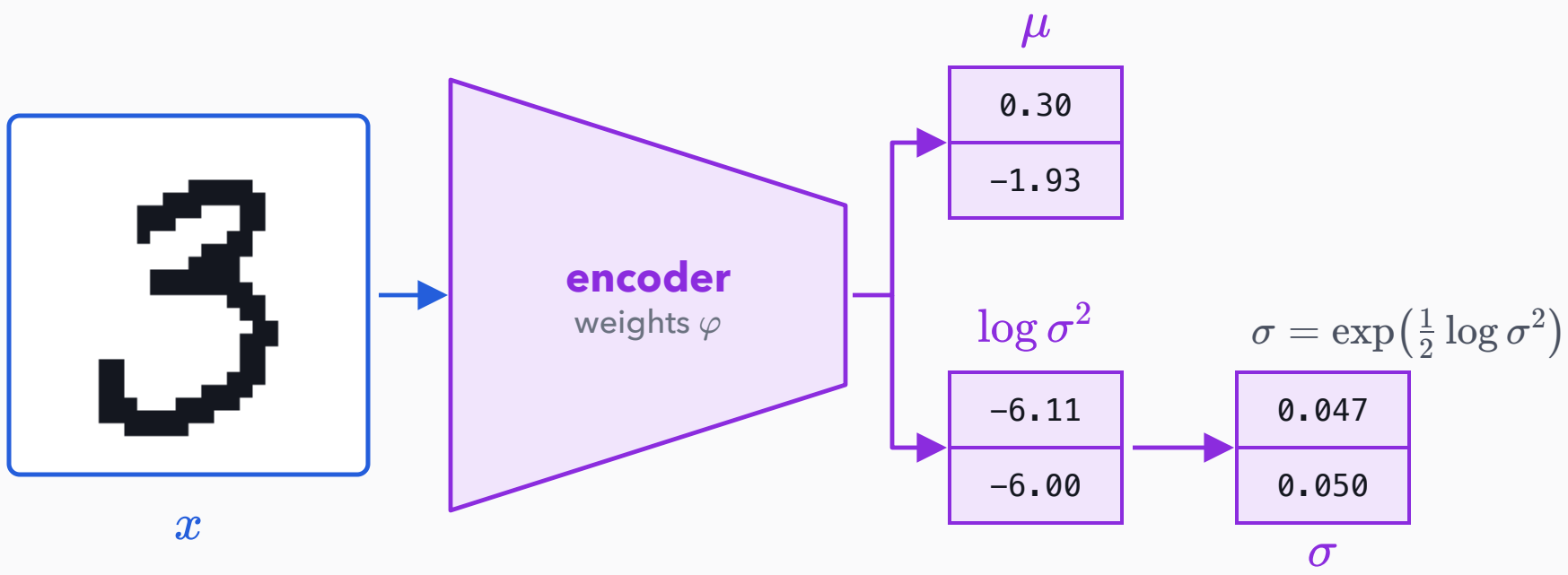
# Encode: the image becomes a cloud



# Encode: the image becomes a cloud



# Encode: the image becomes a cloud

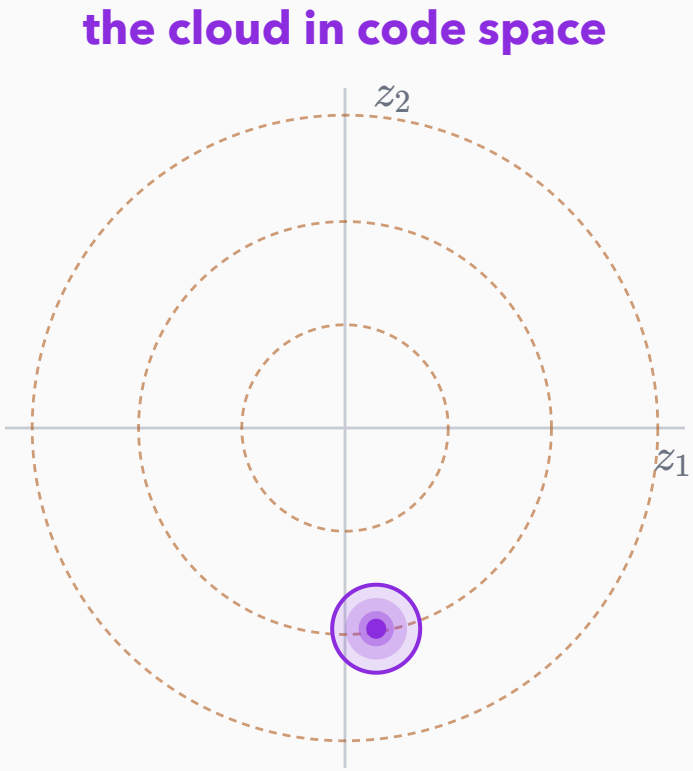



**a cloud centred at $\mu$ with width $\sigma$:**

$$q_\varphi(z \mid x) = \mathcal{N}(\mu, \sigma^2)$$

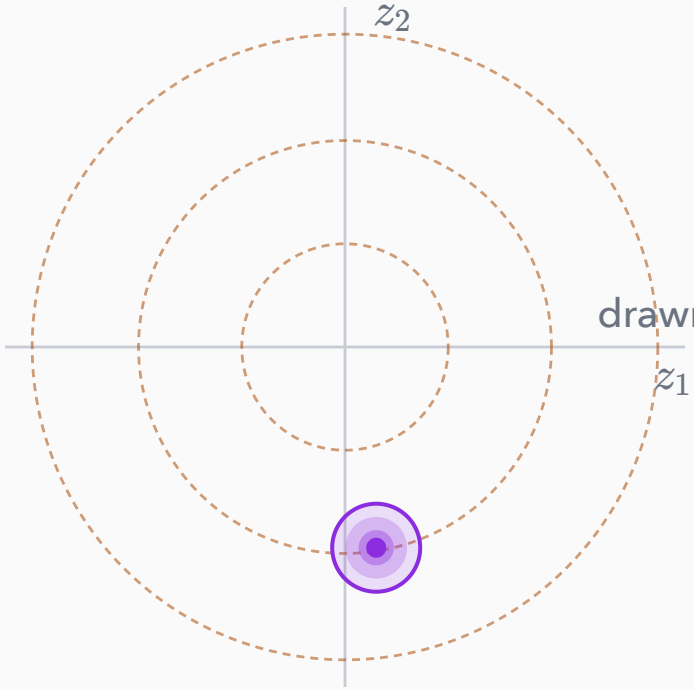
$\sigma \approx 0.05$ against the prior's $\sigma = 1$: the encoder is very sure where our 3 belongs.

# Sample: one code from inside the cloud



# Sample: one code from inside the cloud

**the cloud in code space**





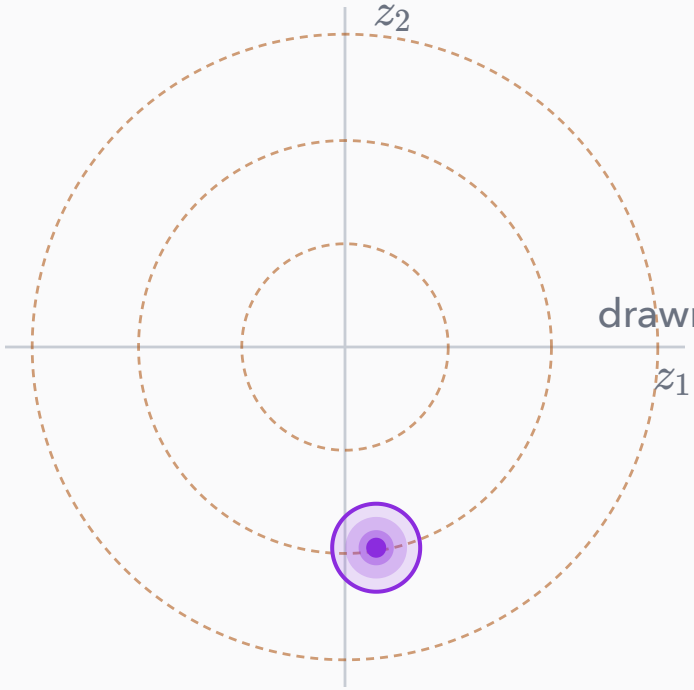


$\varepsilon = (0.62,\ -1.15)$

drawn from a standard normal once

# Sample: one code from inside the cloud

**the cloud in code space**




$\varepsilon$

$\varepsilon = (0.62,\ -1.15)$

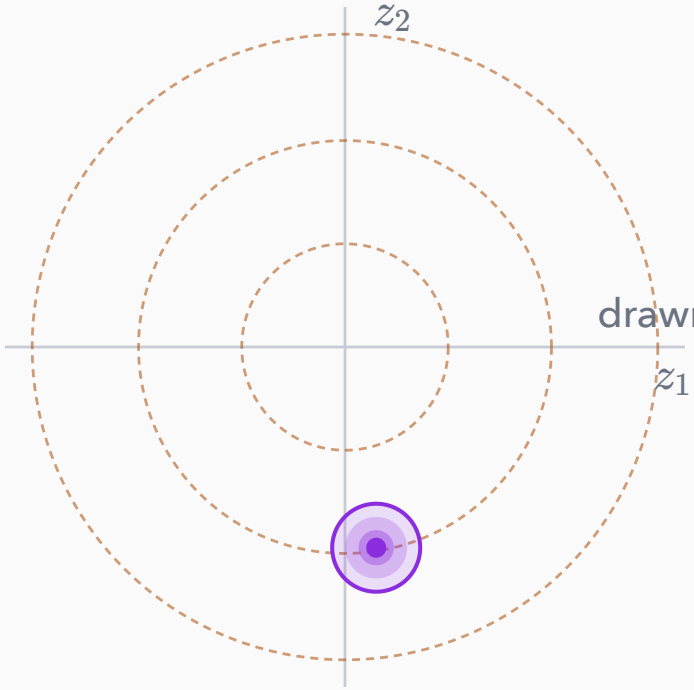
drawn from a standard normal once

$$z_1 = \mu_1 + \sigma_1 \varepsilon_1$$

$$= 0.30 + 0.047 \times 0.62 = 0.33$$

# Sample: one code from inside the cloud

**the cloud in code space**




$\varepsilon$

$\varepsilon = (0.62,\ -1.15)$

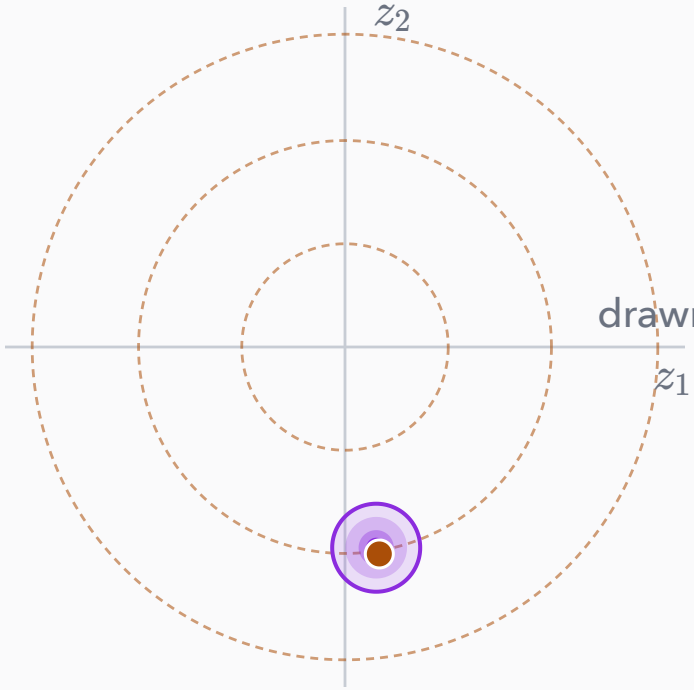
drawn from a standard normal once

$$z_1 = \mu_1 + \sigma_1 \varepsilon_1$$
$$= 0.30 + 0.047 \times 0.62 = 0.33$$

$$z_2 = \mu_2 + \sigma_2 \varepsilon_2$$
$$= -1.93 + 0.050 \times (-1.15) = -1.99$$

# Sample: one code from inside the cloud

**the cloud in code space**




$\varepsilon$

$\varepsilon = (0.62,\ -1.15)$

drawn from a standard normal once

$$z_1 = \mu_1 + \sigma_1 \varepsilon_1$$
$$= 0.30 + 0.047 \times 0.62 = 0.33$$

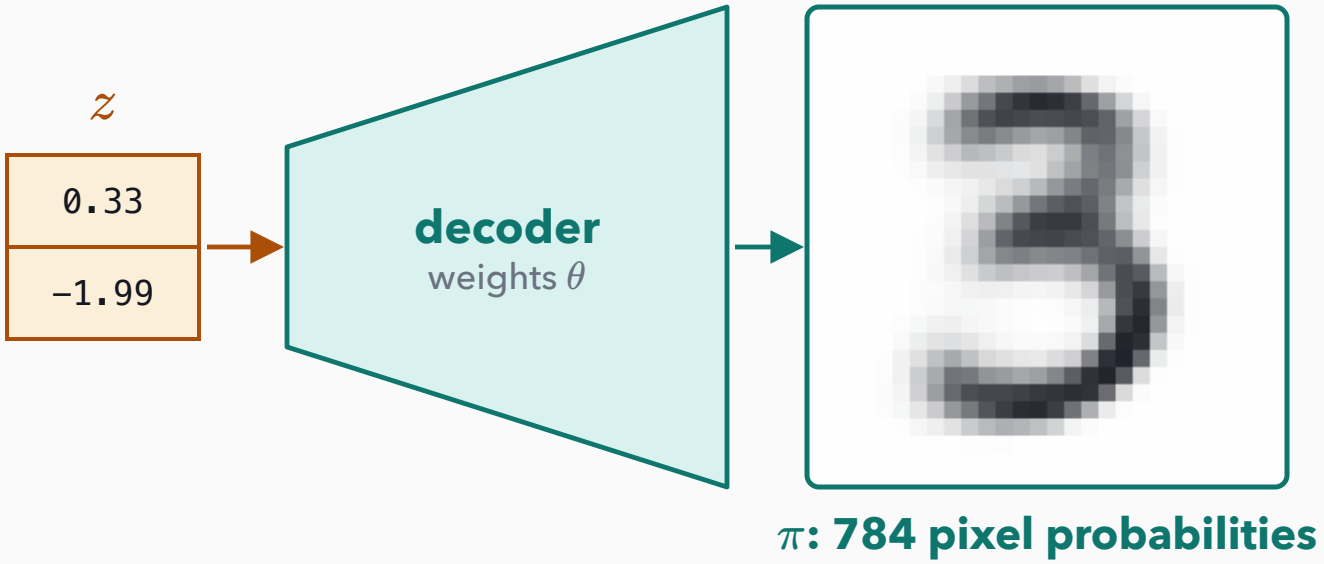
$$z_2 = \mu_2 + \sigma_2 \varepsilon_2$$
$$= -1.93 + 0.050 \times (-1.15) = -1.99$$

$z =$

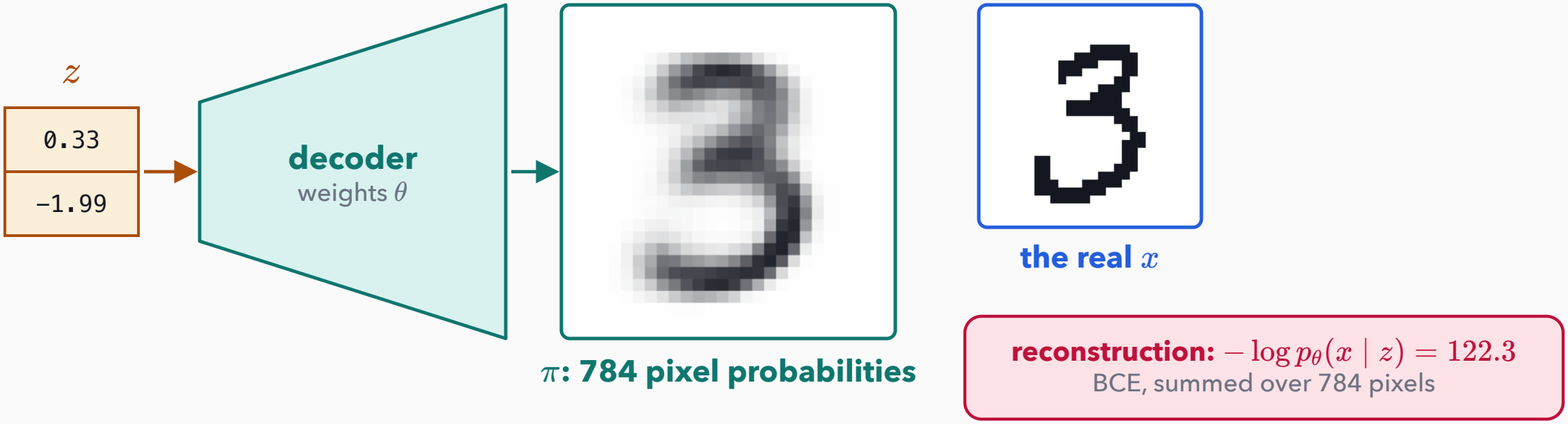
| 0.33 | -1.99 |
|---|---|

**one code from inside the cloud**

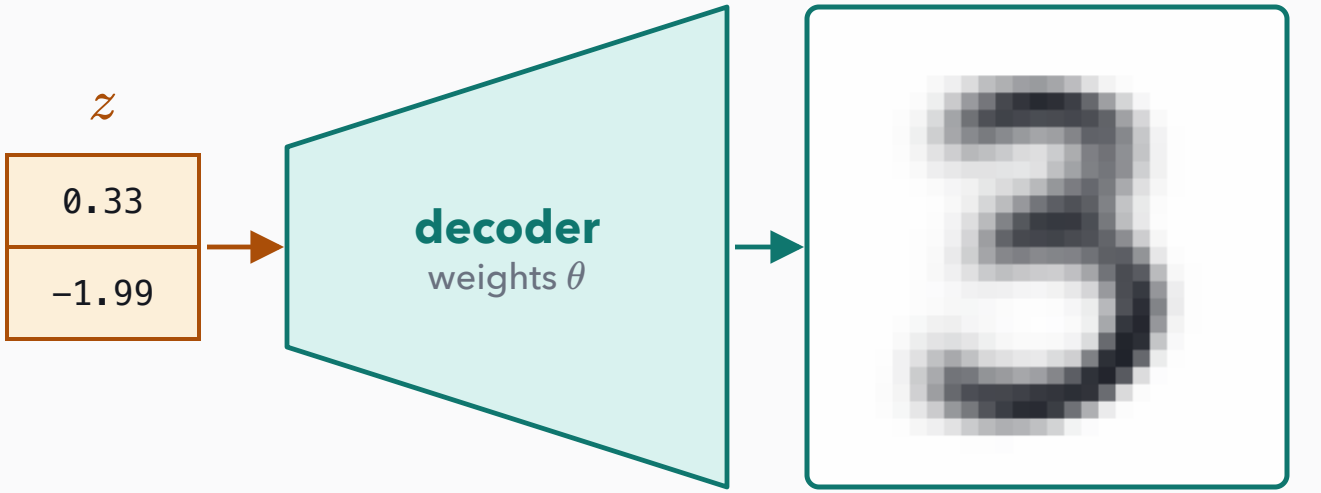
# Decode, then score against the real 3



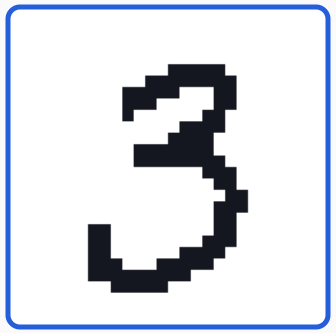
# Decode, then score against the real 3



# Decode, then score against the real 3




$\pi$: 784 pixel probabilities



the real $x$

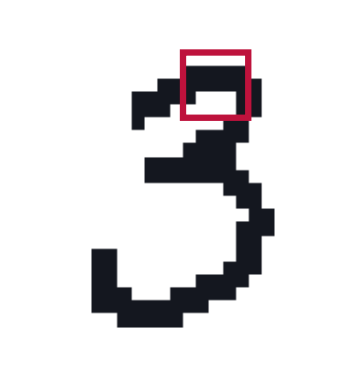
**reconstruction:** $-\log p_\theta(x \mid z) = 122.3$
BCE, summed over 784 pixels

$+\,\mathrm{KL}\big(q_\varphi(z \mid x) \,\|\, \mathcal{N}(0, I)\big) = 6.97$

**Loss for our 3:** $122.3 + 6.97 = 129.3$ **nats. Backpropagate it into both** $\theta$ **and** $\varphi$**.**

STEP 3, ZOOMED IN: THE RECONSTRUCTION TERM

# Each pixel is a coin flip with probability $\pi_i$



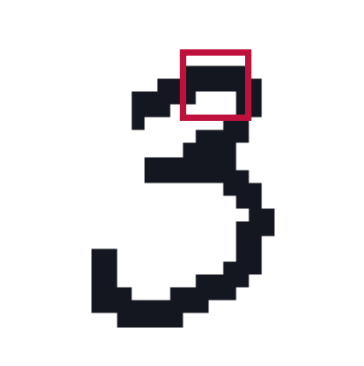
our 3, one 5 × 5 patch

observed $x_i$

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 0 | 1 |

STEP 3, ZOOMED IN: THE RECONSTRUCTION TERM

# Each pixel is a coin flip with probability $\pi_i$



our 3, one 5 × 5 patch

observed $x_i$

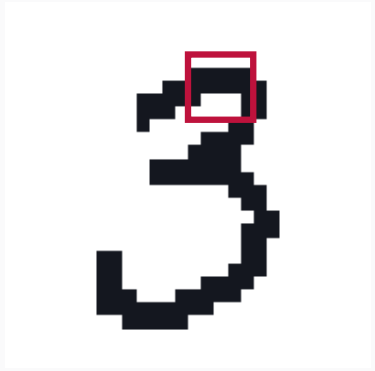
| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 0 | 1 |

decoder's $\pi_i$

| | | | | |
|---|---|---|---|---|
| 0.38 | 0.28 | 0.13 | 0.05 | 0.02 |
| 0.89 | 0.82 | 0.65 | 0.41 | 0.18 |
| 0.77 | 0.79 | 0.76 | 0.67 | 0.40 |
| 0.41 | 0.52 | 0.67 | 0.66 | 0.51 |
| 0.22 | 0.35 | 0.55 | 0.60 | 0.48 |

STEP 3, ZOOMED IN: THE RECONSTRUCTION TERM

# Each pixel is a coin flip with probability $\pi_i$



our 3, one 5 × 5 patch

observed $x_i$

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 0 | 1 |

decoder's $\pi_i$

| | | | | |
|---|---|---|---|---|
| 0.38 | 0.28 | 0.13 | 0.05 | 0.02 |
| 0.89 | 0.82 | 0.65 | 0.41 | 0.18 |
| 0.77 | 0.79 | 0.76 | 0.67 | 0.40 |
| 0.41 | 0.52 | 0.67 | 0.66 | 0.51 |
| 0.22 | 0.35 | 0.55 | 0.60 | 0.48 |

$-\log p(x_i \mid z)$

| | | | | |
|---|---|---|---|---|
| 0.48 | 0.33 | 0.14 | 0.05 | 0.02 |
| 0.12 | 0.20 | 0.43 | 0.89 | 1.71 |
| 0.26 | 0.24 | 0.27 | 0.40 | 0.92 |
| 0.89 | 0.73 | 1.11 | 1.08 | 0.67 |
| 0.25 | 0.43 | 0.80 | 0.92 | 0.73 |

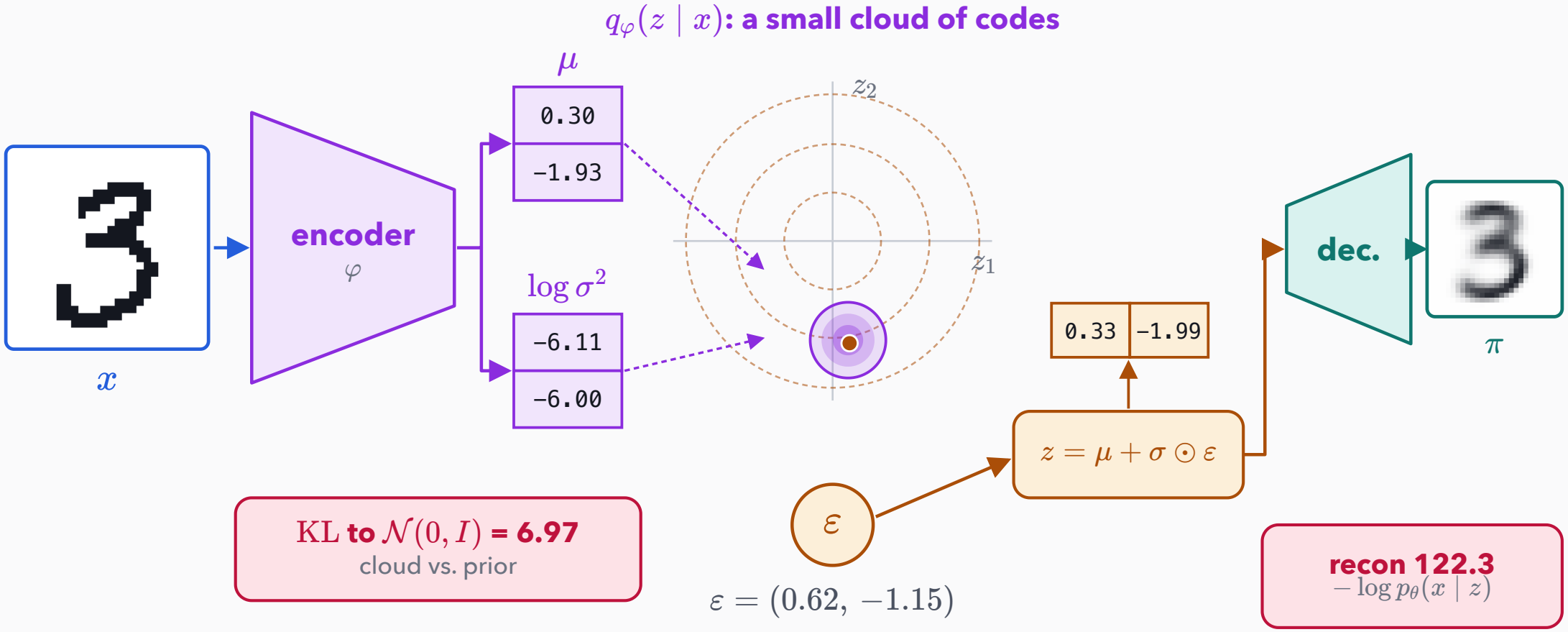
**this 5 × 5 patch: 14.1 nats**

$$-\log p_\theta(x \mid z) = -\sum_{i=1}^{784} \Big[ x_i \log \pi_i + (1 - x_i) \log(1 - \pi_i) \Big] = 122.3 \text{ nats}$$

> **Binary cross-entropy is not a design choice: it is the Bernoulli log-likelihood. Gaussian pixels would give squared error.**

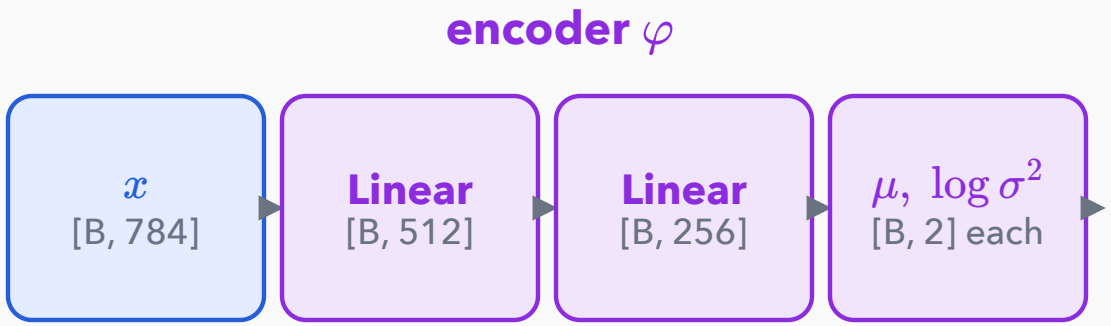
ALL THREE STEPS ON ONE PAGE

# The whole VAE, for our 3



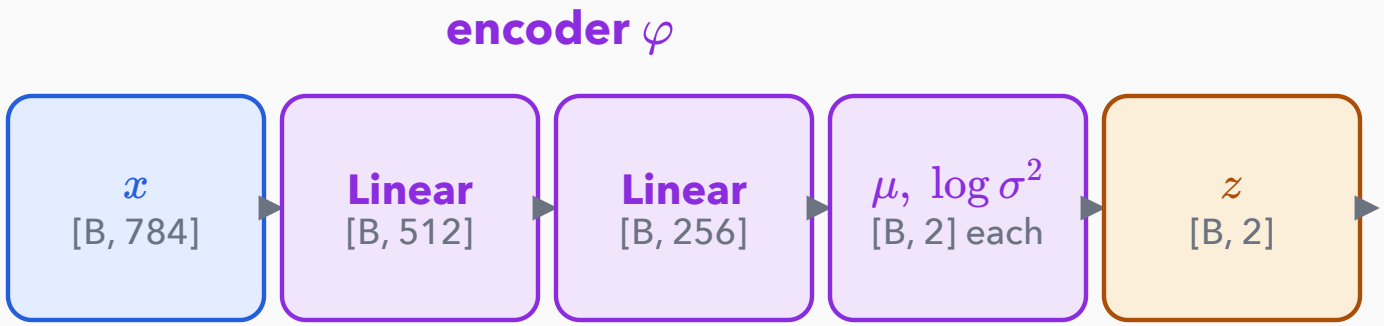
THE SAME NETWORK AS TENSORS

# Shapes, layer by layer



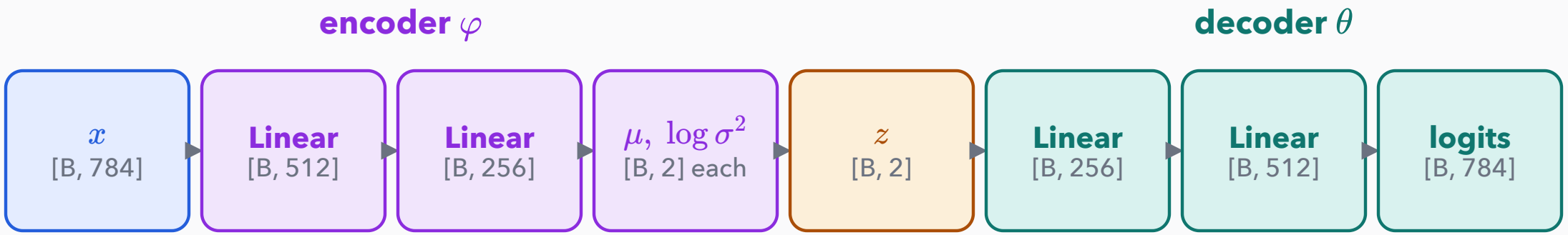
THE SAME NETWORK AS TENSORS

# Shapes, layer by layer



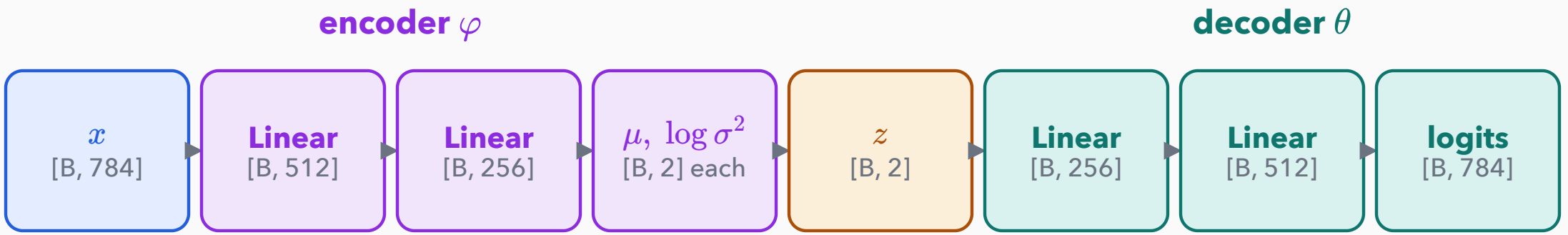
THE SAME NETWORK AS TENSORS

# Shapes, layer by layer



THE SAME NETWORK AS TENSORS

# Shapes, layer by layer




B = batch size. Only the 2 in the middle is the code; $\varepsilon$ has the same shape as $\mu$.

Loss per image: BCE summed over 784 pixels + KL summed over 2 coordinates.
Then average over the B images in the batch.

THE KL TERM FOR OUR 3

# A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j} \left( \mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right)$$

| coordinate | $\mu_j$ | $\sigma_j$ | $\mu_j^2$ | $\sigma_j^2 - 1$ | $-\log \sigma_j^2$ | $\frac{1}{2}(\text{sum})$ |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |

THE KL TERM FOR OUR 3

# A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j} \left( \mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right)$$

| coordinate | $\mu_j$ | $\sigma_j$ | $\mu_j^2$ | $\sigma_j^2 - 1$ | $-\log \sigma_j^2$ | $\frac{1}{2}(\text{sum})$ |
|---|---|---|---|---|---|---|
| $z_1$ | 0.3 | 0.047 | 0.09 | $-1.00$ | 6.11 | **2.6** |

THE KL TERM FOR OUR 3

# A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j} \left( \mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right)$$

| coordinate | $\mu_j$ | $\sigma_j$ | $\mu_j^2$ | $\sigma_j^2 - 1$ | $-\log \sigma_j^2$ | $\frac{1}{2}(\text{sum})$ |
|---|---|---|---|---|---|---|
| $z_1$ | 0.3 | 0.047 | 0.09 | $-1.00$ | 6.11 | **2.6** |
| $z_2$ | −1.93 | 0.05 | 3.72 | $-1.00$ | 6.00 | **4.37** |

THE KL TERM FOR OUR 3

# A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2}\sum_{j}\left(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2\right)$$

| coordinate | $\mu_j$ | $\sigma_j$ | $\mu_j^2$ | $\sigma_j^2 - 1$ | $-\log\sigma_j^2$ | $\frac{1}{2}(\text{sum})$ |
|---|---|---|---|---|---|---|
| $z_1$ | 0.3 | 0.047 | 0.09 | $-1.00$ | 6.11 | **2.6** |
| $z_2$ | −1.93 | 0.05 | 3.72 | $-1.00$ | 6.00 | **4.37** |
| **total** | | | | | | **6.97 nats** |

$$-\log\sigma_1^2 = -\log(0.047^2) = -\log 0.0022 = 6.11$$

> **Most of the cost is $-\log\sigma^2$: the price of being precise. Reconstruction wants precision; the KL resists.**

ONE TRAINING STEP

# The whole algorithm fits on a slide

```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

Encode, reparameterise, decode.

ONE TRAINING STEP

# The whole algorithm fits on a slide

```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

Sum the BCE over all 784 pixels of each image.

ONE TRAINING STEP

# The whole algorithm fits on a slide

```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

Sum the KL over the 2 code coordinates of each image.

ONE TRAINING STEP

# The whole algorithm fits on a slide

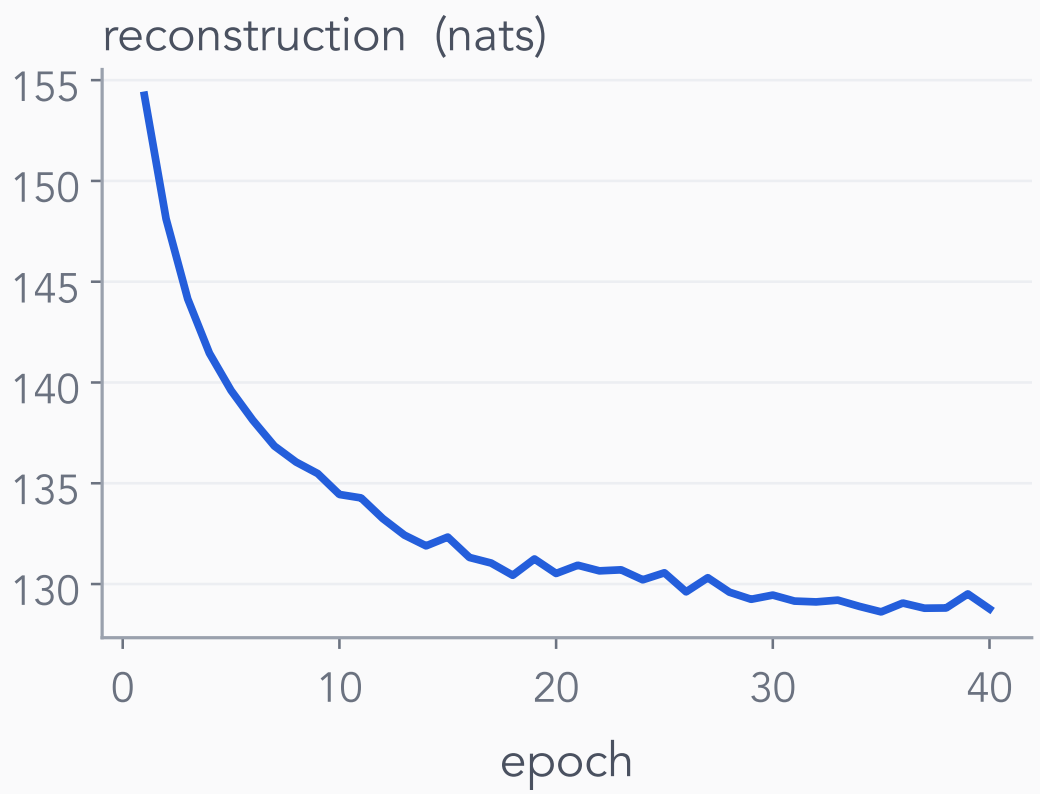
```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

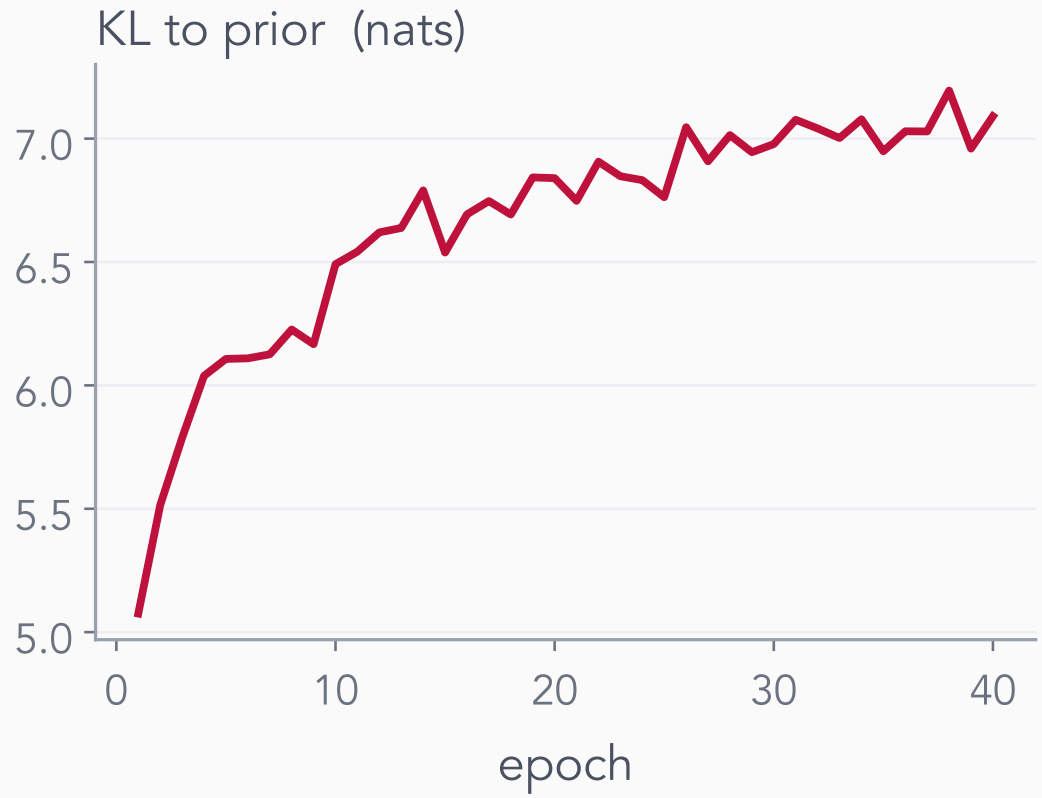
Add per image, average over the batch, step both encoder and decoder.

Averaging BCE over pixels instead of summing silently makes the KL 784× stronger.

TRAINING RUN, TEST SET

# Reconstruction falls, KL rises, then both settle

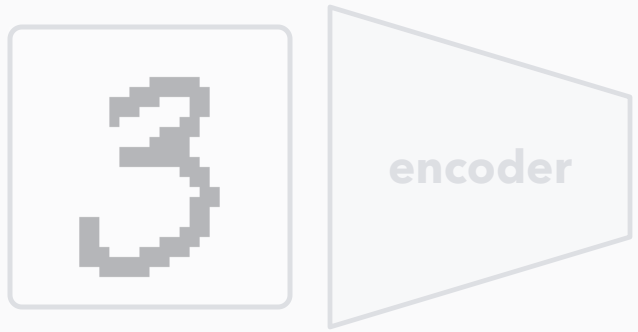



**After 40 epochs: reconstruction 128.8 + KL 7.1 = 135.9 nats per test image.**

THE PAYOFF

# Generate: random code → decoder. No encoder.

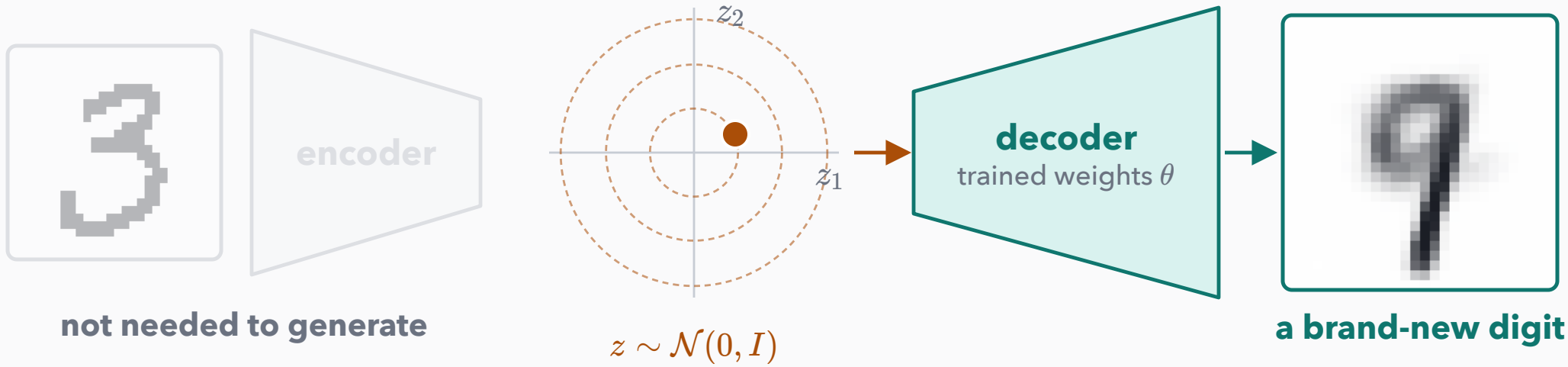



not needed to generate

> This is the whole generator. The encoder was scaffolding for training.

THE PAYOFF

# Generate: random code → decoder. No encoder.




This is the whole generator. The encoder was scaffolding for training.

48 RANDOM CODES EACH

# Samples from two VAEs



2-number code · reconstruction 128.8

48 RANDOM CODES EACH

# Samples from two VAEs



2-number code · reconstruction 128.8

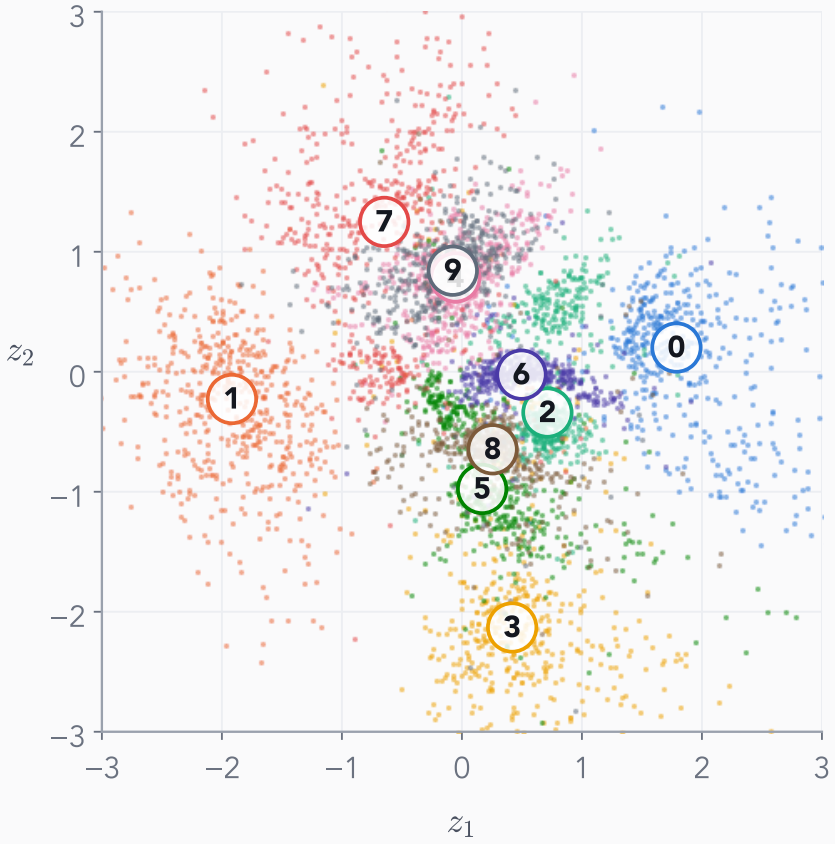


16-number code · reconstruction 57.8

**More code dimensions: sharper and more varied, but some samples are odd. Neither is perfect.**

BECAUSE THE CODE IS 2-D, WE CAN DRAW THE WHOLE MAP

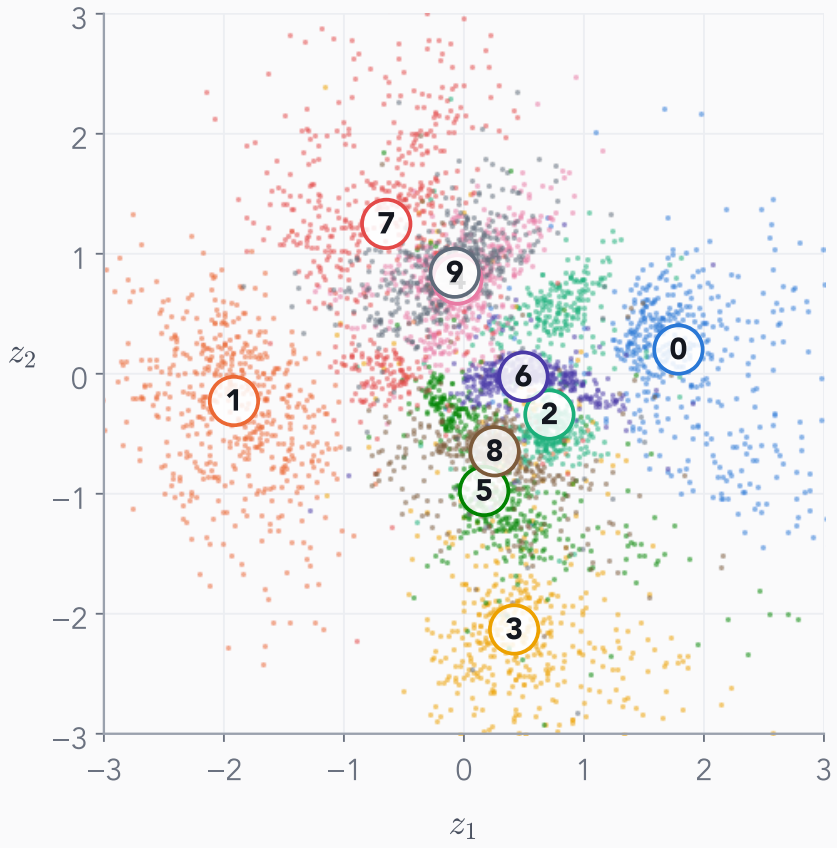
# Every point in the code space is a digit



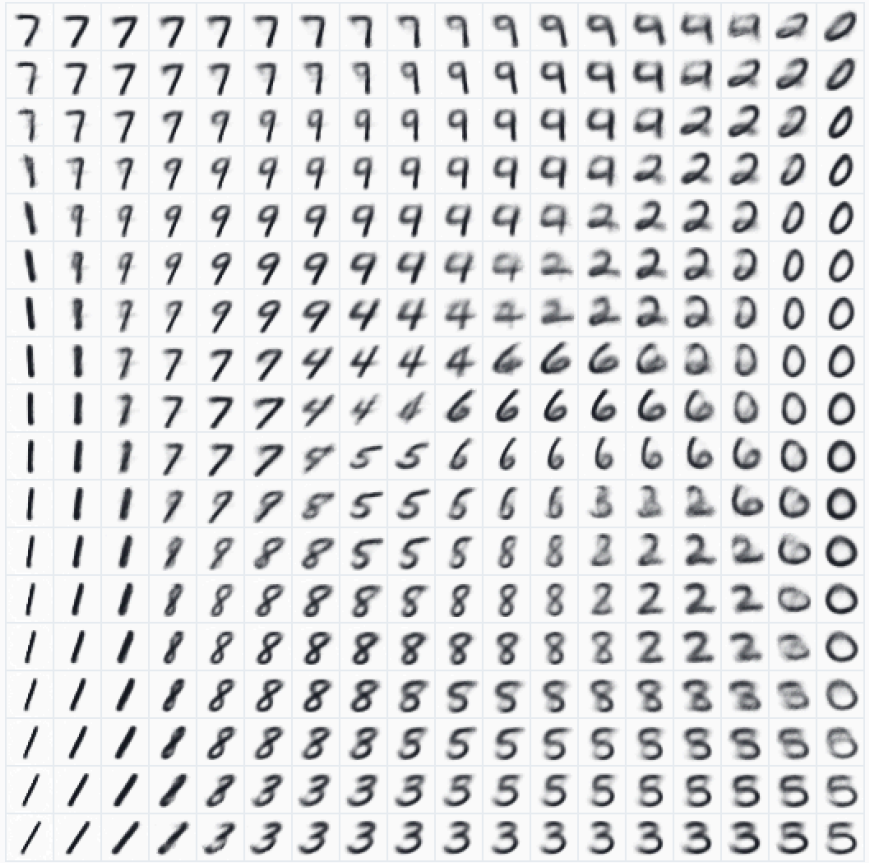

where test digits land

BECAUSE THE CODE IS 2-D, WE CAN DRAW THE WHOLE MAP

# Every point in the code space is a digit




where test digits land

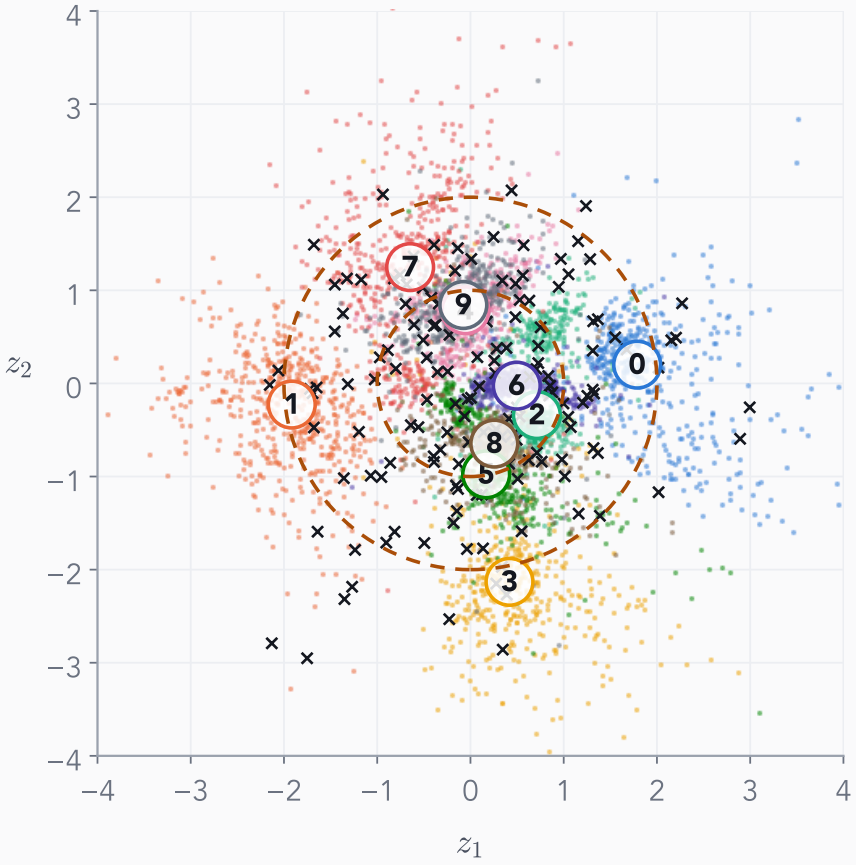


decoded on an 18 × 18 grid, spaced by quantiles of $\mathcal{N}(0, 1)$

**Neighbouring codes give similar digits, and every region decodes to something plausible.**

TRY IT: THE TRAINED VAE, IN YOUR BROWSER

# The same app, now for the VAE




Now random codes from $\mathcal{N}(0, I)$ land among the dots, and almost every one decodes to a digit.

THE COVER, EXPLAINED

# Walk in a straight line from our 3 to a 7



encode both, take 12 evenly spaced points between their $\mu$'s, decode each (16-number VAE)

THE COVER, EXPLAINED

# Walk in a straight line from our 3 to a 7



encode both, take 12 evenly spaced points between their $\mu$'s, decode each (16-number VAE)

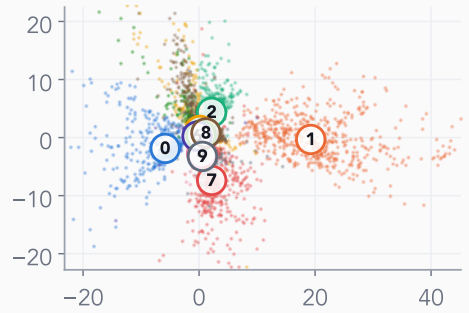


the same walk in the 2-number VAE: the line passes through the 5, 8, 4 and 9 regions on the map

THE TUG OF WAR, MADE VISIBLE

# No KL, normal KL, too much KL

**autoencoder (no KL)**



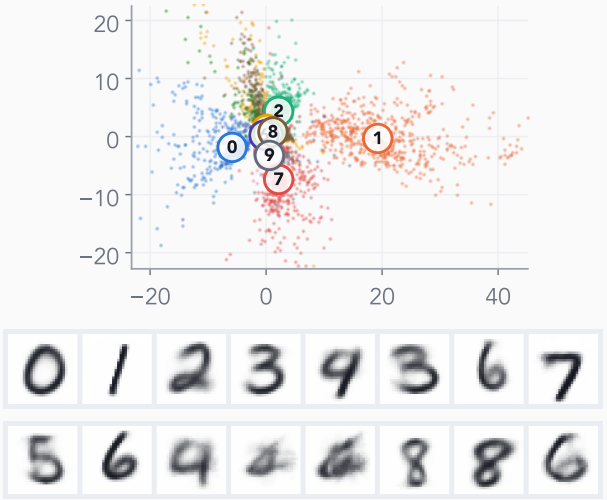



recon 128.1 · no KL term

THE TUG OF WAR, MADE VISIBLE

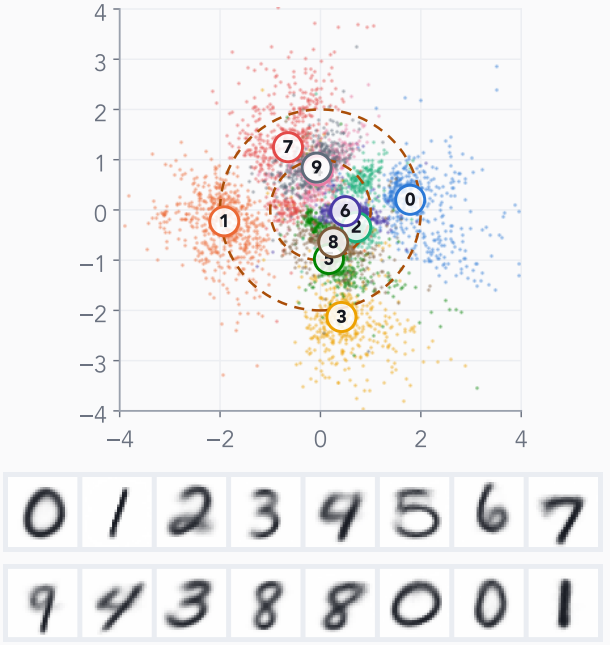
# No KL, normal KL, too much KL

**autoencoder (no KL)**




recon 128.1 · no KL term

**VAE, loss = recon + KL**

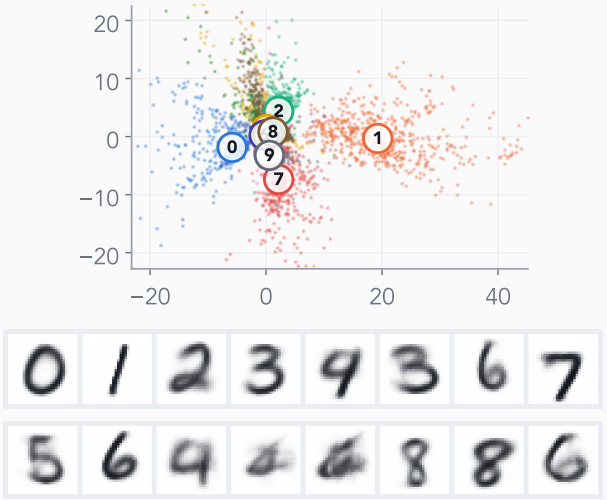



recon 128.8 · KL 7.1

THE TUG OF WAR, MADE VISIBLE

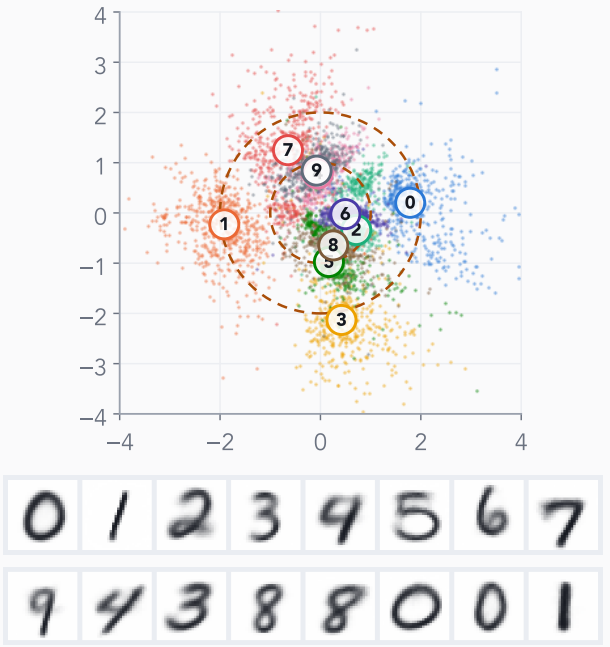
# No KL, normal KL, too much KL

### autoencoder (no KL)



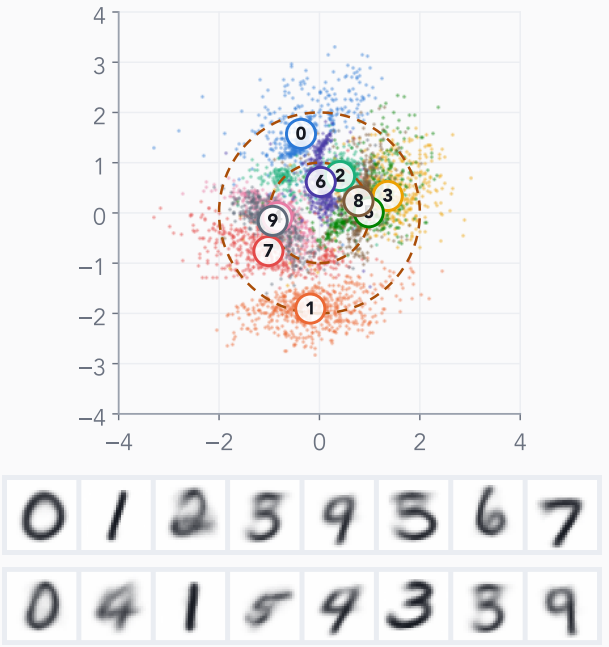
recon 128.1 · no KL term

### VAE, loss = recon + KL



recon 128.8 · KL 7.1

### recon + 8 × KL



recon 140.4 · KL 3.9

> **Rows: code space · reconstructions · random samples. Too little KL: samples fail. Too much: everything blurs together.**

AN HONEST LOOK

# Why are VAE samples blurry?

**1. It fits $\mathrm{KL}(\text{data} \,\|\, \text{model})$.** Like the wide Gaussian on our bumps, it must cover every training digit, so when unsure it spreads probability.



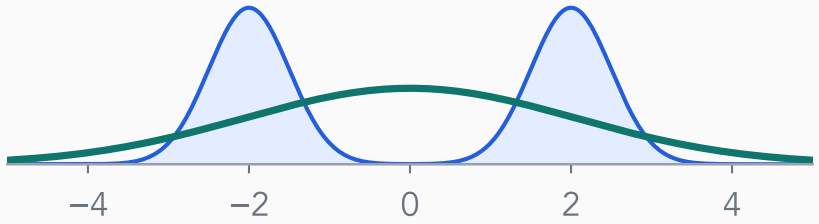
AN HONEST LOOK

# Why are VAE samples blurry?

**1. It fits $\mathrm{KL}(\text{data} \,\|\, \text{model})$.** Like the wide Gaussian on our bumps, it must cover every training digit, so when unsure it spreads probability.

**2. Pixels are independent given $z$.** The decoder outputs one probability per pixel. If a stroke could be here or one pixel over, the best $\pi$ is grey in both places.

best $q = \mathcal{N}(0.0,\ 2.06^2)$



AN HONEST LOOK

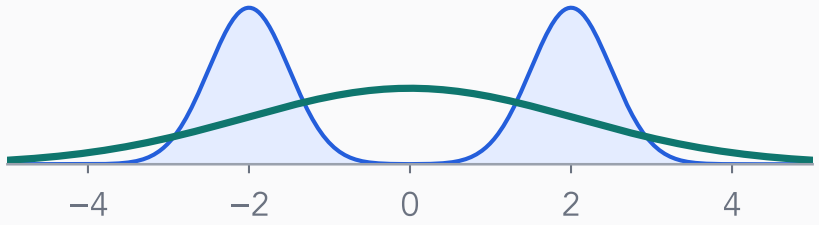
# Why are VAE samples blurry?

**1. It fits $\mathrm{KL}(\text{data} \,\|\, \text{model})$.** Like the wide Gaussian on our bumps, it must cover every training digit, so when unsure it spreads probability.

**2. Pixels are independent given $z$.** The decoder outputs one probability per pixel. If a stroke could be here or one pixel over, the best $\pi$ is grey in both places.

**3. We show $\pi$, the mean,** not a sample. A sample would be speckled instead of grey.

best $q = \mathcal{N}(0.0,\ 2.06^2)$






$\pi$ (the mean)



a sample

**Not a law of nature: bigger codes, better decoders and better likelihoods all sharpen VAEs.**

END OF PART 7

# What we did, and what comes next

## WHAT WE DID

1. Our 3: encode to a cloud, sample a code, decode, score: loss $= 129.3$.
2. Training: reconstruction falls, KL rises, both settle.
3. Generation needs only the prior and the decoder.
4. The KL weight trades sharp reconstructions against a usable code space; samples blur.

END OF PART 7

# What we did, and what comes next

### WHAT WE DID

1. Our 3: encode to a cloud, sample a code, decode, score: loss $= 129.3$.
2. Training: reconstruction falls, KL rises, both settle.
3. Generation needs only the prior and the decoder.
4. The KL weight trades sharp reconstructions against a usable code space; samples blur.

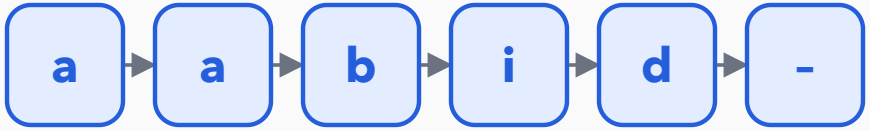
### NEXT LECTURE: A DIFFERENT IDEA

VAEs maximise a bound on $p_\theta(x)$. GANs skip $p_\theta(x)$ entirely: a second network learns to judge real from fake.

THREE WAYS TO GENERATE

# Same goal, three different training signals

**autoregressive**
one token at a time

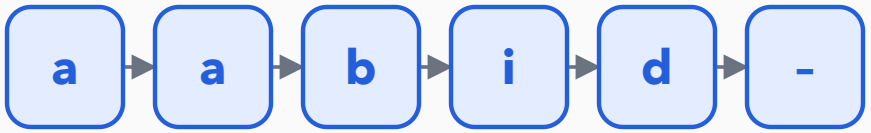



**exact likelihood**
slow sampling

THREE WAYS TO GENERATE

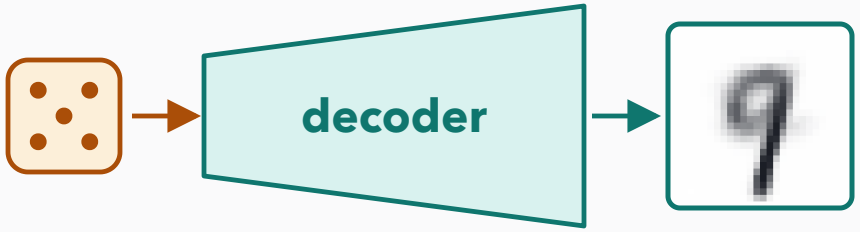
# Same goal, three different training signals

**autoregressive**
one token at a time




**exact likelihood**
slow sampling

**VAE**
random z, decode once

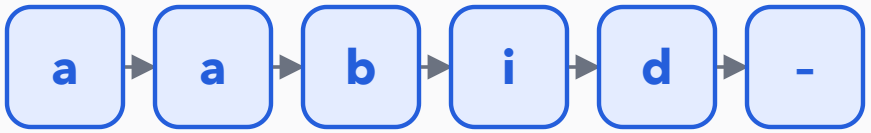



**trained on the ELBO**
tends to blur

THREE WAYS TO GENERATE

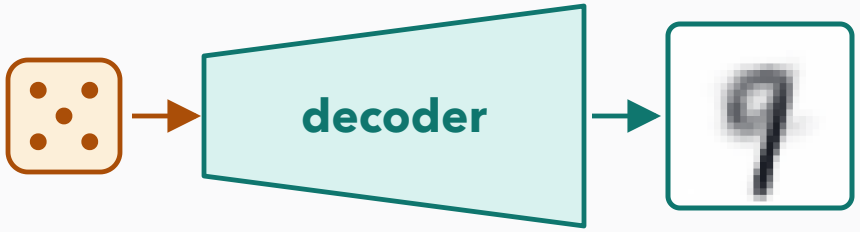
# Same goal, three different training signals

**autoregressive**
one token at a time



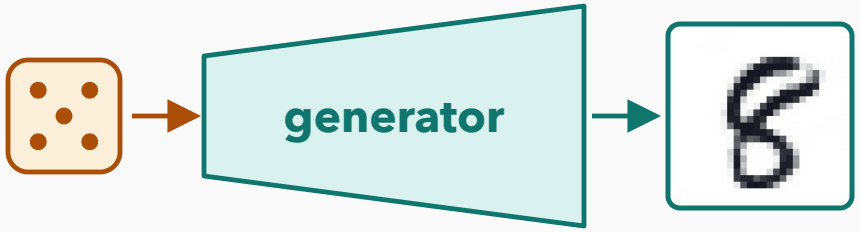

**exact likelihood**
slow sampling

**VAE**
random z, decode once




**trained on the ELBO**
tends to blur

**GAN**
random z, generate once




**next lecture**
trained against a judge

RETURN TO THE OPENING

# What you can now explain



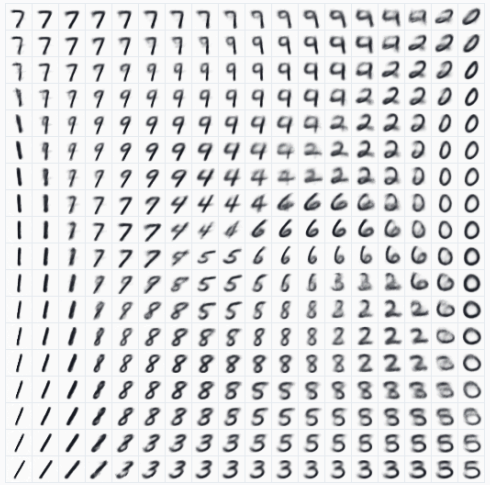
the cover: encode a 3 and a 7, walk a straight line between their codes, decode each point.

RETURN TO THE OPENING

# What you can now explain



the cover: encode a 3 and a 7, walk a straight line between their codes, decode each point.



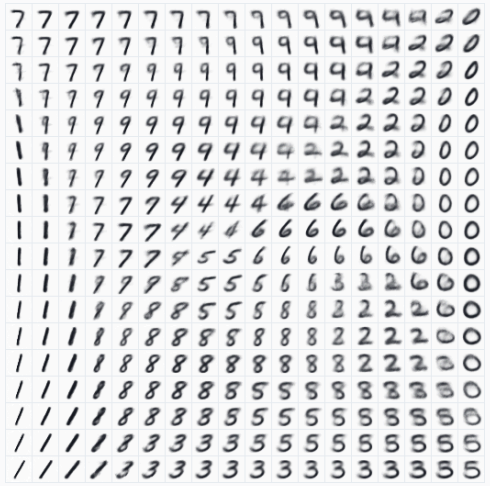
every code in $\mathcal{N}(0, I)$ decodes to a digit: recon + KL, derived from $\mathrm{KL}(q \,\|\, \text{posterior})$.

RETURN TO THE OPENING

# What you can now explain



the cover: encode a 3 and a 7, walk a straight line between their codes, decode each point.



every code in $\mathcal{N}(0, I)$ decodes to a digit: recon + KL, derived from $\mathrm{KL}(q \,\|\, \text{posterior})$.



a conditional VAE: give the label to the encoder and the decoder, and choose which digit to draw.

BEFORE YOU LEAVE

# Three exit questions

1. To generate a new digit with a trained VAE, which parts of the network do you use?

2. Two different KL divergences appear in the VAE story. What does each compare, and which one is in the loss?

3. We never compute $p_\theta(x)$. Why does maximising the ELBO still pull $q$ towards the true posterior?

BEFORE YOU LEAVE

# Three exit questions

1. To generate a new digit with a trained VAE, which parts of the network do you use?

   The prior and the decoder only: $z \sim \mathcal{N}(0, I)$, then decode. The encoder is for training.

2. Two different KL divergences appear in the VAE story. What does each compare, and which one is in the loss?

   $\mathrm{KL}(q \,\|\, \text{prior})$ is in the loss (6.97 for our 3). $\mathrm{KL}(q \,\|\, \text{true posterior})$ is the gap between the ELBO and $\log p_\theta(x)$ (0.55).

3. We never compute $p_\theta(x)$. Why does maximising the ELBO still pull $q$ towards the true posterior?

   $\log p_\theta(x) = \mathrm{ELBO} + \mathrm{KL}(q \,\|\, \text{posterior})$, and the left side does not depend on $q$. Whatever the ELBO gains, the KL loses.

AFTER THE GAN LECTURE

# What if we generate by removing noise, a little at a time?



our 3 → add a little Gaussian noise, again and again → pure noise

> **Adding noise is easy and gives free training pairs. Can a network learn to undo one small step, then walk all the way back from pure noise?**

APPENDIX

# A second route to the ELBO

Part 5 reached the ELBO from $\mathrm{KL}(q \,\|\, \text{posterior})$. Many textbooks reach it from $\log p_\theta(x)$ with Jensen's inequality instead. Same bound, one more tool.

# Start from what we want

$$\log p_\theta(x) = \log \int p_\theta(x, z)\, dz$$

Maximum likelihood wants this number large for our 3, and for every image in the data. The integral over all codes is the part we cannot compute.

**A second route to the bound of Part 5, starting from $\log p_\theta(x)$ instead of the KL.**

# Multiply and divide by the encoder's cloud

$$\log \int p_\theta(x, z)\, dz$$

$$= \log \int q_\varphi(z \mid x)\, \frac{p_\theta(x, z)}{q_\varphi(z \mid x)}\, dz$$

# Multiply and divide by the encoder's cloud

$$\log \int p_\theta(x, z)\, dz$$

$$= \log \int q_\varphi(z \mid x)\, \frac{p_\theta(x, z)}{q_\varphi(z \mid x)}\, dz$$

Multiplying and dividing by the same positive number changes nothing. We are free to choose any $q$; we choose the encoder's Gaussian, which is positive everywhere.

**Why bother? Because the next move turns this integral into an average under $q$, and $q$ knows where the patch is.**

# Read the integral as an average under $q$

$$\log \int q_\varphi(z \mid x)\, \frac{p_\theta(x, z)}{q_\varphi(z \mid x)}\, dz$$

$$= \log \mathbb{E}_{z \sim q_\varphi(z|x)} \left[ \frac{p_\theta(x, z)}{q_\varphi(z \mid x)} \right]$$

# Read the integral as an average under $q$

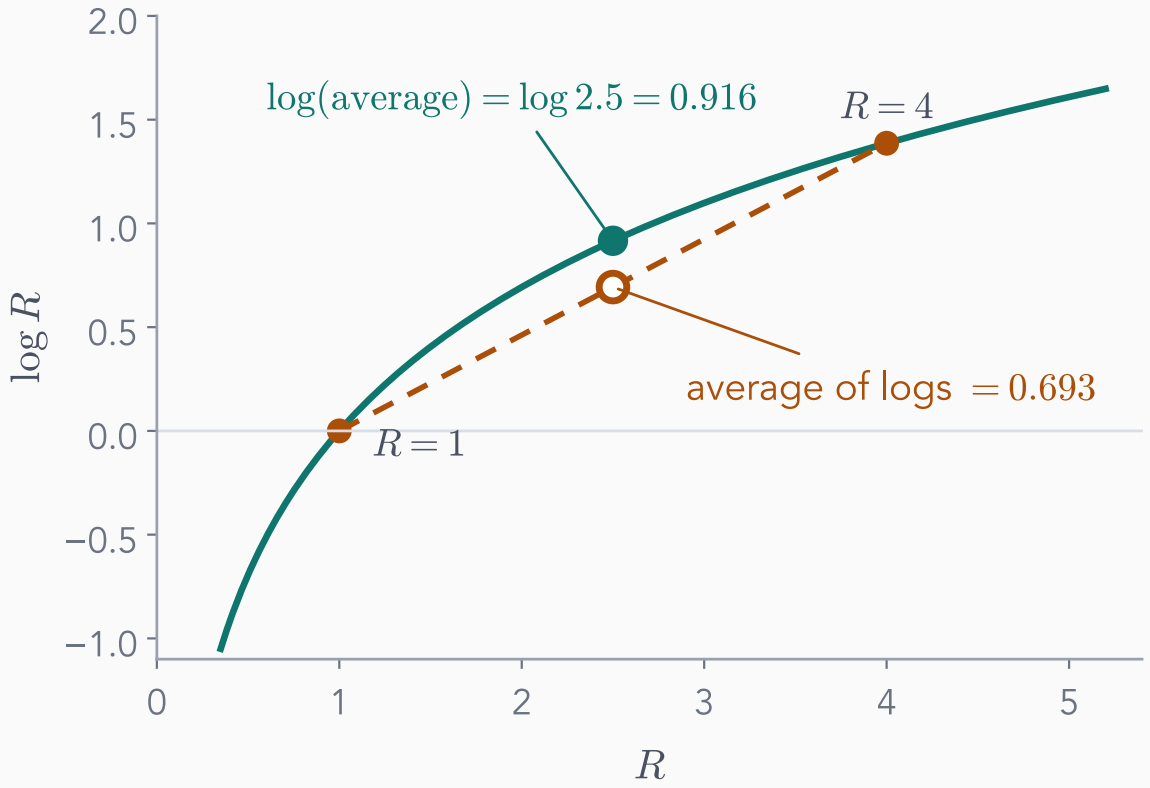
$$\log \int q_\varphi(z \mid x)\, \frac{p_\theta(x, z)}{q_\varphi(z \mid x)}\, dz$$

$$= \log \mathbb{E}_{z \sim q_\varphi(z|x)} \left[ \frac{p_\theta(x, z)}{q_\varphi(z \mid x)} \right]$$

From the $\mathbb{E}$ slide: $\int q(z)\, f(z)\, dz = \mathbb{E}_{z \sim q}[f(z)]$. Here $f(z)$ is the ratio $p_\theta(x, z)/q_\varphi(z \mid x)$: one number for each code.

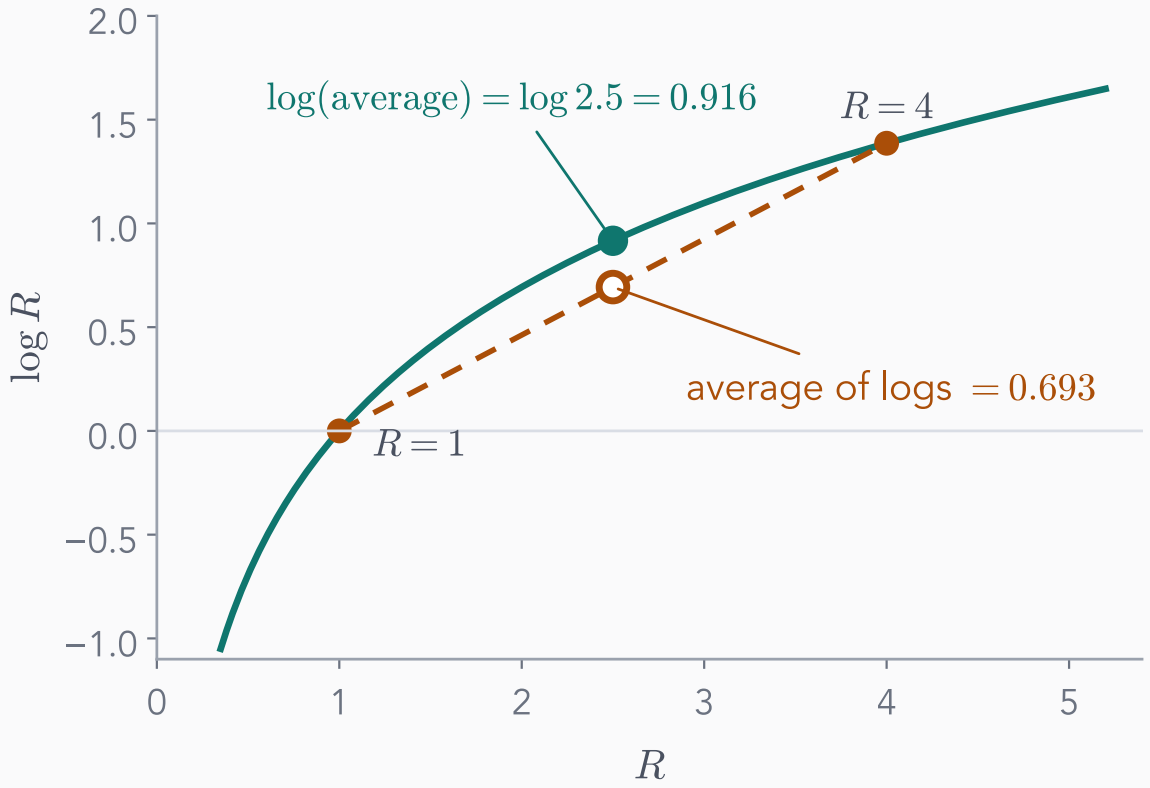
**Still exact. We now average over codes drawn from the encoder's cloud, right where the patch is.**

# Jensen's inequality, in one picture




log curves downwards (it is concave), so every chord lies below the curve.

# Jensen's inequality, in one picture

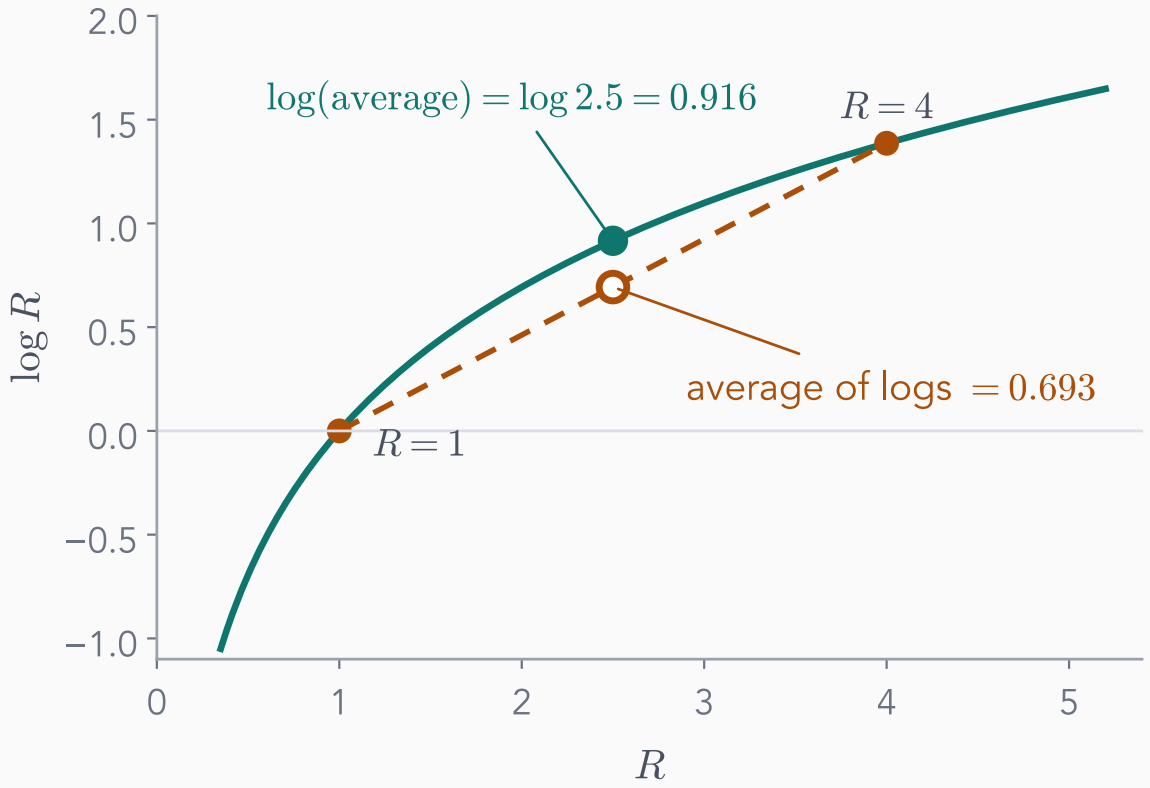



log curves downwards (it is concave), so every chord lies below the curve.

Two values, $R = 1$ and $R = 4$:

$$\underbrace{\log 2.5}_{\log \text{ of the average}} = 0.916 \geq \underbrace{\tfrac{1}{2}(0 + 1.386)}_{\text{average of the logs}} = 0.693$$

# Jensen's inequality, in one picture




log curves downwards (it is concave), so every chord lies below the curve.

Two values, $R = 1$ and $R = 4$:

$$\underbrace{\log 2.5}_{\log \text{ of the average}} = 0.916 \geq \underbrace{\tfrac{1}{2}(0 + 1.386)}_{\text{average of the logs}} = 0.693$$

$$\log \mathbb{E}[R] \geq \mathbb{E}[\log R]$$

# Swap the log and the average: the only inequality

$$\log p_\theta(x) = \log \mathbb{E}_q\left[\frac{p_\theta(x,z)}{q_\varphi(z \mid x)}\right]$$

$$\log p_\theta(x) \geq \mathbb{E}_q\left[\log \frac{p_\theta(x,z)}{q_\varphi(z \mid x)}\right] =: \mathrm{ELBO}$$

# Swap the log and the average: the only inequality

$$\log p_\theta(x) = \log \mathbb{E}_q\left[\frac{p_\theta(x,z)}{q_\varphi(z \mid x)}\right]$$

$$\log p_\theta(x) \geq \mathbb{E}_q\left[\log \frac{p_\theta(x,z)}{q_\varphi(z \mid x)}\right] =: \mathrm{ELBO}$$

Jensen with $R = p_\theta(x,z)/q_\varphi(z \mid x)$. The right side is the **E**vidence **L**ower **BO**und: it can never exceed $\log p_\theta(x)$.

**The same ELBO as Part 5. This route shows it is a bound; Part 5's route also shows the gap is $\mathrm{KL}(q \,\|\, \text{posterior})$.**

FURTHER READING

# Where to go next

- Lilian Weng, *From Autoencoder to Beta-VAE* (lilianweng.github.io): the autoencoder family of Part 2
- Jaan Altosaar, *Tutorial: what is a variational autoencoder?*: the neural-network and probability views side by side
- Kingma & Welling, *Auto-Encoding Variational Bayes*, arXiv 1312.6114
- Blei, Kucukelbir & McAuliffe, *Variational Inference: A Review for Statisticians*, arXiv 1601.00670
- Bishop, *Pattern Recognition and Machine Learning*, §10.1: the ELBO + KL picture of Part 5
- Three YouTube videos on KL divergence that Part 4 draws on (video ids sjgZxuCm_8Q, KHVR587oW8I, tXE23653JrU)
- Stanford CS236 notes on VAEs (deepgenerativemodels.github.io)
- This lecture's code: `scripts/train_models.py` (all models), `scripts/make_figures.py` and `make_figures_extra.py` (every number on the slides)